République Algérienne Démocratique et Populaire
الجمهورية الجزائرية الديمقراطية الشعبية
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
وزارة التعليم العالي و البحث العلمي

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
(المعهد الوطني للتكوين في الإعلام الآلي سابقا)
École nationale Supérieure d'Informatique
ex. INI (Institut National de formation en Informatique)

Mémoire de fin d'études

Pour l'obtention du diplôme d'Ingénieur d'État en Informatique

Option : Systèmes Informatiques

Modèle de prédiction de la dégradation de la QoS dans des environnements dynamiques et mobiles : Application aux véhicules autonomes de nouvelles générations

---

Réalisé par :
M. Borhaneddine HAMADOU

Encadré par :
Mme. Lamia YESSAD (ESI)
M. Sami YANGUI (LAAS-CNRS, Toulouse)
Mme. Nora FACI (LIRIS, Lyon)

Soutenu le 24 Septembre 2023, Devant le jury composé de :

Mme. Narimene DAKICHE :	ESI	- Présidente
M. Elhachemi GUERROUT :	ESI	- Examinateur
Mme. Rymel BENABIDALLAH :	ESI	- Rapporteur

Promotion : 2022/2023

Dédicace

“

À mes parents, pour leur soutien indéfectible, leurs bénédictions sans fin et l’éducation qu’ils m’ont offert, vous êtes la source de mon inspiration. Je suis à jamais redevable de votre générosité et de votre amour.

À ma maman, une femme d’une force et d’une gentillesse inégalées, je vous suis éternellement reconnaissant pour la contribution considérable que vous avez apportée à mes études et à ma vie en général.

À mes frères et soeurs, pour leur amour constant, leur motivation inébranlable. Toujours prêts à me soutenir, à me conseiller et à m’encourager. Je vous suis profondément reconnaissant pour tout ce que vous avez fait pour moi.

À tous les professeurs qui sont intervenu au cours de ma formation.

À tous mes amis, avec qui j’ai partagé tant de souvenirs précieux et d’années formidables.

À tous mes chers camarades spéciaux, avec qui j’ai partagé de nombreuses années d’études, d’entraide et de souvenirs : Mohammed, Nassim, Ayoub, Sarah, Lynda, Mélissa, Wilem.

À toutes ces personnes qui m’ont aidé, m’ont soutenu et ont contribué à ma croissance sur tous les plans.

”

- Borhaneddine

Remerciements

Je commence par rendre grâce à Dieu, par la grâce de qui les bonnes actions sont accomplies ...

À Borhaneddine, version 2018

Je voudrais exprimer ma profonde gratitude envers moi-même, Borhaneddine avant 5 ans, en 2018, pour avoir eu le courage de choisir mon propre domaine malgré les pressions. J'ai montré une clairvoyance exceptionnelle en comprenant que le succès ne dépend pas des attentes des autres. Je suis impressionné par ma confiance inébranlable face aux doutes des autres. Peu importe leur opinion, mon objectif a toujours été d'avoir un impact positif sur les gens. Ma détermination est admirable. Une excellente nouvelle : ma vision de l'avenir était exacte. Mes années universitaires ont été légères, passionnantes et stimulantes, car j'ai suivi un domaine qui me passionnait et je savais que je me démarquerais en lui. J'ai développé mes compétences dans les domaines qui m'intéressent réellement et où je souhaite évoluer. Je mets en garde Borhaneddine de 2023 contre Borhaneddine de 2018, qui pourrait surpasser ma version actuelle. Je dois redoubler d'efforts, faire les bons choix, avoir confiance en moi et croire en mes capacités. Mon objectif est que Borhaneddine de 2028 m'écrive une lettre de remerciement !

À l'équipe SARA .. Mes encadrants

Je tiens à exprimer ma profonde gratitude envers l'ensemble du personnel du ***LAAS***, et en particulier envers l'équipe ***SARA***. Je tiens à remercier tout particulièrement ***M. Khalil DRIRA***, Responsable de l'équipe ***SARA***, pour m'avoir accueilli au sein de son équipe. Je souhaite également remercier chaleureusement ***Cédric***, le doctorant avec lequel j'ai passé de nombreuses heures à travailler sur le projet. Son aide précieuse m'a permis de mieux comprendre le projet et de m'intégrer pleinement dans l'équipe.

Je remercie aussi mes encadrants ***M. Sami YANGUI*** et ***Mme. Nora FACI*** pour la confiance et le suivi tout au long de mon stage, leurs critiques constructives et leurs conseils précieux. Merci à ***Mme. Lamia YESSAD***, mon encadrante de l'ESI pour ses vifs encouragements et les corrections apportées à mon travail.

À la famille de l'ESI .. Mes camarades

Merci à l'ensemble du corps professoral des deux écoles : ***ESI SBA*** et ***ESI Alger***, et principalement tous les professeurs pour la qualité de la formation prodiguée. Merci à tous mes camarades de classe depuis la première année en classe préparatoire jusqu'à aujourd'hui, avec qui j'ai passé ma formation. Merci également à mes amis toulousains et camarades de stage avec qui j'ai passé ces derniers mois de stage.

Résumé

Avec la croissance de l'Internet des Objets (IoT) et l'expansion des réseaux 5G, les villes intelligentes progressent rapidement, accompagnées de services intelligents comme la santé, l'agriculture et les véhicules autonomes. Cependant, ces services nécessitent une faible latence pour des interactions en temps réel et une grande capacité de stockage en raison des nombreuses données générées par les dispositifs connectés.

Face aux exigences de l'IoT moderne, le cloud computing traditionnel se révèle insuffisant même avec ses importantes ressources, car il opère au niveau central. Pour résoudre ces limitations, l'edge computing émerge, utilisant les ressources périphériques du réseau pour délocaliser certains services des applications IoT et les rendre plus près des utilisateurs.

Dans ce contexte, le choix du nœud edge revêt une importance majeure pour répondre aux besoins de qualité de service (QoS) des applications, notamment en ce qui concerne la latence. Cette étude se concentre sur les véhicules autonomes, qui nécessitent une faible latence pour des décisions rapides et précises. L'objectif est d'anticiper les dégradations de la QoS des nœuds edge, afin de sélectionner un nœud répondant aux exigences des véhicules autonomes.

La dégradation se caractérise par une valeur de QoS qui sort de la plage requise pour un fonctionnement optimal des véhicules autonomes. Ainsi, le coeur du travail consiste à prédire avec précision les valeurs futures de la QoS tout en tenant compte des contraintes temporelles. Cela implique le développement et l'évaluation de modèles de prédiction de la QoS, qui revêtent une importance cruciale en raison de la complexité des données, notamment dans le contexte des véhicules autonomes.

Pour cela, des modèles de prédiction des séries chronologiques basés sur l'apprentissage automatique sont élaborés, permettant d'estimer les valeurs de QoS concernant la communication réseau entre le véhicule et les nœuds edge, ainsi que la charge de travail pour chaque nœud edge potentiel.

Mots clés : iot, 5g, qos, villes intelligentes, véhicules autonomes, edge computing, prédiction des séries chronologiques, apprentissage automatique.

Abstract

With the rise of the Internet of Things (IoT) and the expansion of 5G networks, smart cities are rapidly advancing, accompanied by intelligent services such as healthcare, smart agriculture, and autonomous vehicles. However, these services require low latency for real-time interactions and substantial storage capacity due to the extensive data generated by connected devices.

Faced with the demanding requirements of modern IoT, traditional cloud computing proves inadequate even with its significant resources, as it operates at a central level. To address these limitations, edge computing is emerging, utilizing peripheral network resources to offload certain services from IoT applications and bring them closer to users.

In this context, the selection of the edge node becomes crucial in meeting the Quality of Service (QoS) needs of applications, particularly concerning latency. This study focuses on autonomous vehicles, which demand low latency for quick and precise decision-making. The goal is to anticipate QoS degradation of edge nodes to select a node that meets the requirements of autonomous vehicles.

Degradation is characterized by a QoS value falling outside the required range for optimal operation of autonomous vehicles. Thus, the core of the work is to accurately predict future QoS values while considering temporal constraints. This involves the development and evaluation of QoS prediction models, which are vital due to data complexity, especially in the context of autonomous vehicles.

To achieve this, machine learning-based time series prediction models are developed, enabling the estimation of Quality of Service (QoS) values related to network communication between the vehicle and edge nodes, as well as the workload for each potential edge node.

Keywords : iot, 5g, qos, smart cities, autonomous vehicles, edge computing, time-series forecasting, machine learning.

ملخص

مع ظهور إنترنت الأشياء (IoT) وانتشار شبكات 5G, تتقدم المدن الذكية بسرعة، مصحوبة بخدمات ذكية مثل الرعاية الصحية, الزراعة الذكية والمركبات الذاتية القيادة. ومع ذلك، تتطلب هذه الخدمات وقت إستجابة سريع جدا للتمكن من التفاعل في الوقت الحقيقي, وسعة تخزين كبيرة بسبب البيانات الواسعة التي تولدها الأجهزة المتصلة.

في مواجهة متطلبات IoT الحديثة، تثبت الحوسبة السحابية التقليدية عجزها حتى مع مواردها الكبيرة، حيث تعمل على المستوى المركزي. لمعالجة هذه القيود، تظهر حوسبة الحافة، مستخدمة موارد الشبكة المحيطة لتفريغ بعض الخدمات من تطبيقات IoT وتقديمها بالقرب من المستخدمين.

في هذا السياق، يصبح اختيار جهاز الحافة أمرًا حاسمًا في تلبية احتياجات جودة الخدمة (QoS) للتطبيقات، خاصة فيما يتعلق بوقت الإستجابة. يركز هذا البحث على المركبات الذاتية القيادة، التي تتطلب وقت إستجابة سريع جدا لاتخاذ قرارات سريعة ودقيقة. الهدف هو التنبؤ بتدهور جودة الخدمة لأجهزة الحافة لاختيار جهاز يلبي متطلبات المركبات الذاتية.

تتميز الانخفاضات بقيمة QoS بتجاوز النطاق المطلوب لتشغيل مثالي للمركبات الذاتية. لذلك، يتمحور العمل حول التنبؤ بدقة بقيم QoS المستقبلية مع مراعاة القيود الزمنية. وهذا يتضمن تطوير وتقييم نماذج التنبؤ بجودة الخدمة (QoS)، والتي تعتبر ضرورية جدا نظرًا لتعقيد البيانات، خاصة في سياق المركبات الذاتية.

لتحقيق ذلك، يتم تطوير نماذج التنبؤ بالسلاسل الزمنية بناءً على التعلم الآلي، مما يتيح تقدير قيم جودة الخدمة (QoS) المتعلقة بالاتصالات الشبكية بين المركبة وأجهزة الحافة، بالإضافة إلى العبء العملي لكل جهاز حافة محتمل.

الكلمات المفتاحية: إنترنت الأشياء، شبكات 5G ،جودة الخدمة، المدن الذكية، المركبات الذاتية القيادة، حوسبة الحافة، التنبؤ بالسلاسل الزمنية، التعلم الآلي.

Table des matières

Dédicace i

Remerciements ii

Résumé iii

Abstract iv

ملخص v

Table des matières vi

Table des figures x

Liste des tableaux xii

Liste des sigles et acronymes xiv

Introduction 1

Villes intelligentes et véhicules autonomes . . . 1
Enjeux de qualité de service pour les véhicules autonomes . . . 3
Formalisation du problème . . . 4
Organisation du mémoire . . . 5

I État de l'art 6

1 Background 7

1.1 Introduction . . . 7
1.2 Véhicules autonomes . . . 7
1.2.1 Impacts et défis des véhicules autonomes . . . 9
1.3 Internet des Objets (IoT) . . . 10
1.4 Métriques de QoS pour les véhicules autonomes . . . 11
1.4.1 Concept de Qualité de service . . . 12
1.4.2 Besoin en QoS des véhicules autonomes . . . 12
1.5 Edge computing pour les véhicules autonomes . . . 13
1.5.1 Multi-access Edge Computing et 5G . . . 14
1.5.2 Rôle clé de l'edge computing pour les véhicules autonomes . . . 15
1.5.2.1 Scénario1 : Connaissance en temps réel de la situation routière locale 16
1.5.2.2 Scénario2 : Transparence (pour le dépassement) . . . 16

1.5.2.3 Scénario3 : Détection des usagers vulnérables de la route 17
1.5.3 Défis de l'edge computing pour les véhicules autonomes 18
1.5.3.1 Disponibilité des données 18
1.5.3.2 Gestion de la confidentialité des données 18
1.5.3.3 Hétérogénéité et diversité 18
1.6 Séries chronologiques 19
1.6.1 Prédiction des séries chronologiques 19
1.6.1.1 Fenêtre temporelle (Window Length) 20
1.6.1.2 Horizon de prédiction (Forecasting Horizon) 20
1.6.1.3 Types de séries chronologiques 20
1.6.2 Modèles de prédiction des séries chronologiques 21
1.6.2.1 Stationnarité des séries chronologiques 22
1.6.2.2 Modèles structurés 23
1.6.2.3 Modèles non-structurés 24
1.6.2.4 Comparaison entre les modèles structurés et non-structurés 28
1.7 Conclusion 30

2 Modélisation et prédiction des séries chronologiques de QoS 31
2.1 Introduction 31
2.2 Informations générales sur les travaux examinés 32
2.3 Modélisation 32
2.3.1 Types de données historiques de QoS 33
2.3.2 Méthodes et modèles utilisés 34
2.3.2.1 Modèles proposés 34
2.3.2.2 Modèles comparatifs 36
2.3.3 Préparation et pré-traitement des données 36
2.4 Prédiction 38
2.4.1 Horizon de prédiction 38
2.4.1.1 Stratégie Récursive 39
2.4.1.2 Stratégie Direct 40
2.4.1.3 Stratégie DirRec 40
2.4.1.4 Stratégie MIMO (Multi-Input Multi-Output) 41
2.4.1.5 Stratégie DIRMO 41
2.4.2 Mesure de performance 45
2.4.2.1 Précision 45
2.4.2.2 Consommation de temps 46
2.5 Conclusion 47

II Contribution 48

3 Étude de cas 49
3.1 Introduction 49
3.2 Cadre du projet 49
3.2.1 Présentation de l'organisme d'accueil : LAAS-CNRS 49
3.2.2 Département RISC 50

3.2.2.1 L'équipe SARA . . . 50
3.3 Présentation du projet de recherche . . . 51
3.4 Solution et Architecture Proposées . . . 51
3.5 Analyse des Avantages et des Défis de l'Architecture Actuelle . . . 53
3.5.1 Avantages . . . 53
3.5.1.1 Tolérance aux Pannes . . . 53
3.5.1.2 Modularité . . . 54
3.5.1.3 Réduction du Temps de Traitement . . . 54
3.5.2 Défis . . . 54
3.5.2.1 Latence due aux Flux TCP/HTTP . . . 54
3.5.2.2 Problèmes de Sécurité . . . 55
3.5.2.3 Mécanisme de sélection des Nœuds Edges . . . 55
3.6 Positionnement du PFE dans le contexte de recherche . . . 55
3.6.1 Collecte du jeu de données . . . 56
3.6.2 Importance de la prédiction de QoS : Scénario Illustrant . . . 59
3.7 Conclusion . . . 60

4 Construction des modèles de prédiction de la QoS 61
4.1 Introduction . . . 61
4.2 Cadre d'application . . . 61
4.2.1 Vue d'ensemble de l'architecture du système . . . 61
4.2.2 Service de monitoring . . . 62
4.3 Processus de construction des modèles de prédiction . . . 63
4.3.1 Collecte de données . . . 64
4.3.2 Prétraitement de données . . . 64
4.3.2.1 Débruitage par filtre à moyenne mobile. . . . 65
4.3.2.2 Détection et Imputation des valeurs manquantes. . . . 68
4.3.2.3 Séparation des données d'entraînement et de test. . . . 69
4.3.2.4 Normalisation des données. . . . 70
4.3.3 Construction et entraînement des modèles . . . 70
4.3.3.1 Séparation des données d'entraînement et de validation. . . . 70
4.3.3.2 Modèles à évaluer. . . . 71
4.3.3.3 Réglage des hyperparamètres basé sur les données de validation. . . . 71
4.3.3.4 Entraînement et génération de modèles formés. . . . 72
4.3.4 Évaluation des modèles . . . 73
4.3.4.1 RMSE et MAE. . . . 73
4.3.4.2 Temps de Génération des Modèles. . . . 74
4.3.4.3 Temps de Génération de Prédictions. . . . 74
4.3.5 Génération de l'API . . . 75
4.3.5.1 Création de l'API . . . 75
4.3.5.2 Définition des points d'entrée API . . . 75
4.4 Conclusion . . . 78

5 Implémentation et tests 79
5.1 Introduction . . . 79

5.2 Configuration des expérimentations . . . 79
5.3 Jeu de données . . . 80
5.4 Hyperparamétrisation . . . 80
5.5 Prédictions univariées à un pas en avant . . . 83
5.5.1 Précision des prédictions . . . 83
5.5.2 Temps d'entraînement des modèles . . . 87
5.5.3 Temps de génération des prédictions . . . 88
5.5.4 Synthèse . . . 89
5.6 Prédictions univariées à N pas en avant . . . 89
5.6.1 Précision des prédictions . . . 96
5.6.1.1 Stratégie Recursive . . . 96
5.6.1.2 Stratégie Directe . . . 97
5.6.1.3 Stratégie hybride DirRec . . . 98
5.6.1.4 Stratégie MIMO . . . 100
5.6.1.5 Stratégie hybride DIRMO . . . 101
5.6.2 Temps d'entraînement des modèles . . . 102
5.6.3 Temps de génération des prédictions . . . 104
5.6.4 Synthèse . . . 104
5.7 Prédictions multivariées . . . 106
5.7.1 Prédiction multivariée par nœud . . . 106
Commentaires . . . 107
5.7.2 Prédiction multivariée par caractéristique . . . 108
Commentaires . . . 108
5.7.3 Synthèse . . . 109
5.8 Réalisation de l'API . . . 109
5.9 Conclusion . . . 110

Conclusion Générale **111**

Bibliographie **113**

Webographie **118**

Table des figures

1 Infrastructure technologique des villes intelligentes . . . 3

1.1 Capteurs importants des véhicules autonomes [69] . . . 8
1.2 Niveaux d'autonomie des véhicules [68] . . . 9
1.3 Étapes de l'IoT [77] . . . 10
1.4 Architecture IoT [1] . . . 11
1.5 Architecture MEC dans la 5G [86] . . . 15
1.6 Transparence (pour le dépassement) . . . 16
1.7 Détection des usagers vulnérables de la route . . . 17
1.8 Fenêtre temporelle et horizon de prédiction . . . 19
1.9 Distinction entre les séries chronologiques multivariées et univariées . . . 21
1.10 Séries non stationnaires (en haut) et stationnaires (en bas) [41] . . . 23
1.11 Architecture du modèle RNN : pliée (à gauche) et dépliée (à droite) [87] . . . 26
1.12 Architectures de cellules LSTM et GRU [83] . . . 27
1.13 Architectures du modèle ESN [20] . . . 28

2.1 Répartition des travaux examinés selon le contexte d'application . . . 32
2.2 Transformation des données de séries chronologiques vers des données tabulaires . . . 37
2.3 Stratégie récursive de prédiction à N pas en avant . . . 40
2.4 Stratégie direct de prédiction à N pas en avant . . . 40
2.5 Stratégie DirRec de prédiction à N pas en avant . . . 41
2.6 Stratégie MIMO de prédiction à N pas en avant . . . 41
2.7 Stratégie DIRMO de prédiction à N pas en avant . . . 42
2.8 Taxonomie des stratégies de prédiction à N pas en avant . . . 42
2.9 Comparaison des temps d'entraînement des modèles des différentes stratégies . . . 43

3.1 Organigramme du LAAS-CNRS [74] . . . 50
3.2 Architecture proposée pour l'intégration des véhicules autonomes dans la ville intelligente [39] . . . 52
3.3 Diagramme de séquence du fonctionnement de l'architecture . . . 53
3.4 Comparaison des temps de traitement entre exécution locale et externalisation vers un nœud edge . . . 54
3.5 Setup du prototype avec un véhicule et trois nœuds . . . 57
3.6 Corrélation entre le délai de communication du premier nœud edge et ses coordonnées spatiales . . . 58

4.1 Vue d'ensemble de l'architecture du système . . . 62
4.2 Service de monitoring . . . 63
4.3 Processus adopté pour la construction des modèles de prédiction de la QoS . . . 64

4.4 Fonctionnement du filtre à moyenne mobile pour la réduction de bruit 65
4.5 Prédictions sur une série chronologique à forte teneur en bruit 65
4.6 Prédictions sur la série chronologique débruitée 67
4.7 Architecture globale de l'API 75
4.8 Diagramme de séquence des interactions entre le système et l'API 77

5.1 Taille de lot (Batch Size) lors de l'optimisation des hyperparamètres pour LSTM (en haut), GRU (au milieu) et RNN (en bas) 83
5.2 Schéma de l'API et des technologies associées 109
5.3 Capture de l'interface web 110

Liste des tableaux

1.1 Indicateurs clés de performance (KPI) de QoS. 12
1.2 Comparaison entre le MEC et le cloud computing 14
1.3 Avantages et Inconvénients des deux grandes familles de modèles structurés : ARIMA et Lissage Exponentiel. 25
1.4 Comparaison des modèles RNN, LSTM, GRU et ESN. 28
1.5 Comparaison entre modèles structurés et modèles non-structurés 29

2.1 Informations générales sur les travaux examinés 33
2.2 Approches de modélisation dans les travaux examinés 35
2.3 Approches de prédiction dans les travaux examinés 39
2.4 Stratégies de prédiction à N pas en avant de la QoS dans la littérature 43
2.5 Avantages et inconvénients des différentes stratégies de prédiction à N pas en avant . 44

3.1 Description des caractéristiques du premier ensemble de données. 58

4.1 Modèles sélectionnés et objectifs 72
4.2 Points d'entrée de l'API 76

5.1 Hyperparamètres pour chaque modèle avec leur explication. 81
5.2 Plage des valeurs pour chaque hyperparamètre. 82
5.3 Durée moyenne d'optimisation des hyperparamètres des modèles. 82
5.4 Prédictions univariées à un pas en avant de la série node1_delay du premier ensemble de données. 84
5.5 Prédictions univariées à un pas en avant de la série node2_delay du premier ensemble de données. 84
5.6 Prédictions univariées à un pas en avant de la série node3_delay du premier ensemble de données. 84
5.7 Prédictions univariées à un pas en avant de la série node1_ram du premier ensemble de données. 85
5.8 Prédictions univariées à un pas en avant de la série node2_ram du premier ensemble de données. 85
5.9 Prédictions univariées à un pas en avant de la série node3_ram du premier ensemble de données. 85
5.10 Prédictions univariées à un pas en avant de la série node1_cpu du premier ensemble de données. 86
5.11 Prédictions univariées à un pas en avant de la série node2_cpu du premier ensemble de données. 86

5.12 Prédictions univariées à un pas en avant de la série node3_cpu du premier ensemble de données. 86
5.13 Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie récursive. 91
5.14 Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie directe. 92
5.15 Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie hybride DirRec. 93
5.16 Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie MIMO. 94
5.17 Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie DIRMO (s=2). 95
5.18 Résultats de la prédiction multivariée des caractéristiques de chaque nœud. 107
5.19 Résultats de la prédiction multivariée de chaque caractéristique pour les 3 nœuds. . . 108

Liste des sigles et acronymes

QoS	Quality of Service
TIC	Technologies de l'Information et de la Communication
IA	Intelligence Artificielle
ML	Machine Learning
AV	Autonomous Vehicule
IoT	Internet of Objects
KPI	Key Performance Indicator
MEC	Multi-access Edge Computing
5G	Cinquième génération de réseaux mobiles
eMBB	Enhanced Mobile Broadband
URLLC	Ultra-Reliable Low-Latency Communications
mMTC	Massive Machine Type Communications
ES	Exponential Smoothing
AR	Autorégressive
MA	Moving Average
ARIMA	Auto-Regressive Integrated Moving Average
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Reccurent Unit
ESN	Echo State Network
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
PoP	Point de Présence
MIMO	Multi-Input Multi-Output
API	Application Programming Interface

Introduction

Villes intelligentes et véhicules autonomes

D'ici 2050, environ 75% de la population vivra dans des zones urbaines [84]. Cette urbanisation rapide présente un défi pour les villes qui doivent garantir une bonne qualité de vie en termes de confort et de sécurité. C'est une tâche très difficile, car l'infrastructure urbaine fonctionne déjà avec une charge extrême, et avec la croissance de la population des zones urbaines, il y a une menace d'effondrement. Le concept de "Smart City" est l'une des tentatives pour résoudre ce défi.

Les villes intelligentes, ou smart cities en anglais, sont des villes utilisant les Technologies de l'Information et de la Communication (TIC) pour améliorer la qualité de vie de leurs citoyens, tout en réduisant leur impact environnemental [54]. Les villes intelligentes offrent une gamme de services innovants appelés "Smart services urbains". Ces services font partie d'un large éventail d'applications technologiques au sein des villes intelligentes. Ils utilisent notamment des données en temps réel pour améliorer les services associés aux infrastructures de la ville. Parmi ces smart services, nous pouvons notamment citer ceux relatifs aux secteurs d'activité suivants [2] :

- ***Transport.*** Les smart services de transport peuvent optimiser les flux de circulation (e.g., synchronisation des feux de signalisation), améliorer les temps de trajet (e.g., optimisation des itinéraires) et réduire la pollution (e.g., réservation en temps réel de voies des transports en commun).

- ***Gestion des déchets.*** Les smart services peuvent aider à réduire la quantité de déchets produits (e.g., capteurs de niveau de remplissage des poubelles) et optimiser leur traitement (e.g., surveillance des sites d'enfouissement).

- ***Éclairage public.*** Un autre secteur où les smart services urbains opèrent et peuvent optimiser l'éclairage tout en réduisant la consommation d'énergie (e.g., capteurs de lumière et de présence).

- ***Gestion de l'énergie.*** Les smart services peuvent optimiser la production (e.g., gestion des panneaux solaires), la distribution (e.g., stockage d'énergie) et la consommation d'énergie (e.g., surveillance en temps réel).

- ***Sécurité publique.*** Les smart services peuvent améliorer la sécurité des citoyens en détectant en temps réel des incidents d'ordre criminel ou autre.

La latence, qui désigne le délai ou le temps de réponse entre l'émission d'une requête et la réception de la réponse, est un facteur crucial à considérer. La plupart de ces services sont sensibles à la latence, d'autant plus qu'ils utilisent des ressources énergétiques limitées. Cela implique l'utilisation d'un nombre important d'appareils connectés, la génération d'une grande quantité de données et le besoin d'une importante capacité de stockage. [15]. Malgré ses énormes capacités de stockage et de calcul, le cloud computing ne peut pas garantir les critères de faible latence et de réponse en temps réel car il est déployée au niveau du réseau central [24]. Cependant, l'edge computing avec la proximité de ses nœuds et serveurs (fournis au niveau du réseau périphérique) peut assurer les critères mentionnés précédemment. D'autre part, la Communication Ultra-Fiable à Faible Latence (URLLC) est un service clé majeur dans l'ère des réseaux 5G. Ces services, parmi d'autres, accessibles dans les villes intelligentes, facilitent la communication entre les appareils connectés. En particulier, ils rendent plus aisée l'interaction entre les applications connectées et les nœuds edge.

Les véhicules autonomes, également appelés "autonomous vehicles" en anglais, constituent des services urbains intelligents complets. En effet, ils sont capables de contribuer à la résolution des problèmes de mobilité en milieu urbain tout en améliorant l'efficacité environnementale des systèmes de transport. L'intégration de véhicules autonomes dans le tissu urbain aurait pour conséquence d'offrir aux citoyens une mobilité plus efficace et sécurisée, tout en entraînant une nette réduction des émissions de gaz d'échappement. De plus, ces véhicules peuvent être intégrés dans des services de transport intelligents.

Ces véhicules autonomes circulent dans la ville intelligente et échangent en permanence des informations avec leur environnement. Ces informations sont essentielles pour les applications embarquées sensibles à la latence des véhicules autonomes, notamment celles qui prennent des décisions. Elles permettent d'être au courant et de réagir de manière appropriée à chaque information reçue [37]. De plus, le véhicule autonome pourrait déléguer certaines tâches aux nœuds edge, plutôt qu'au cloud, afin d'optimiser la consommation d'énergie de son système et d'effectuer des calculs complexes tout en maintenant des communications à faible latence.

Pour déployer et mettre en œuvre leurs services intelligents, les villes intelligentes s'appuient sur une infrastructure technologique essentielle. Une approche pour intégrer les véhicules autonomes dans l'infrastructure de la ville intelligente est illustrée dans la figure 1. Les caractéristiques fondamentales de cette infrastructure comprennent notamment :

- ***Internet des Objets.*** Les villes intelligentes utilisent des milliers de capteurs et de dispositifs connectés pour collecter des données en temps réel sur des variables telles que la qualité de l'air, la température, la circulation, la consommation d'énergie, etc. Ces données servent à optimiser les smart services urbains [23].

- ***Technologie de communication de cinquième génération (5G).*** En offrant une meilleure connectivité (i.e., une plus grande quantité de données supportée) que les précédentes technologies de communication, la 5G permet aux capteurs/dispositifs connectés et aux smart services de communiquer et d'échanger des données avec une latence réduite. La 5G utilise des architectures cloud-native et virtualisées, avec des fonctions réseau déployées sous forme de logiciel plutôt que sur des équipements propriétaires [6].

- ***Cloud computing et Edge computing.*** En combinant le Cloud computing et l'Edge computing [56], les villes intelligentes peuvent traiter et/ou stocker des quantités massives de données tout en offrant une réponse rapide et en temps réel aux défis urbains.

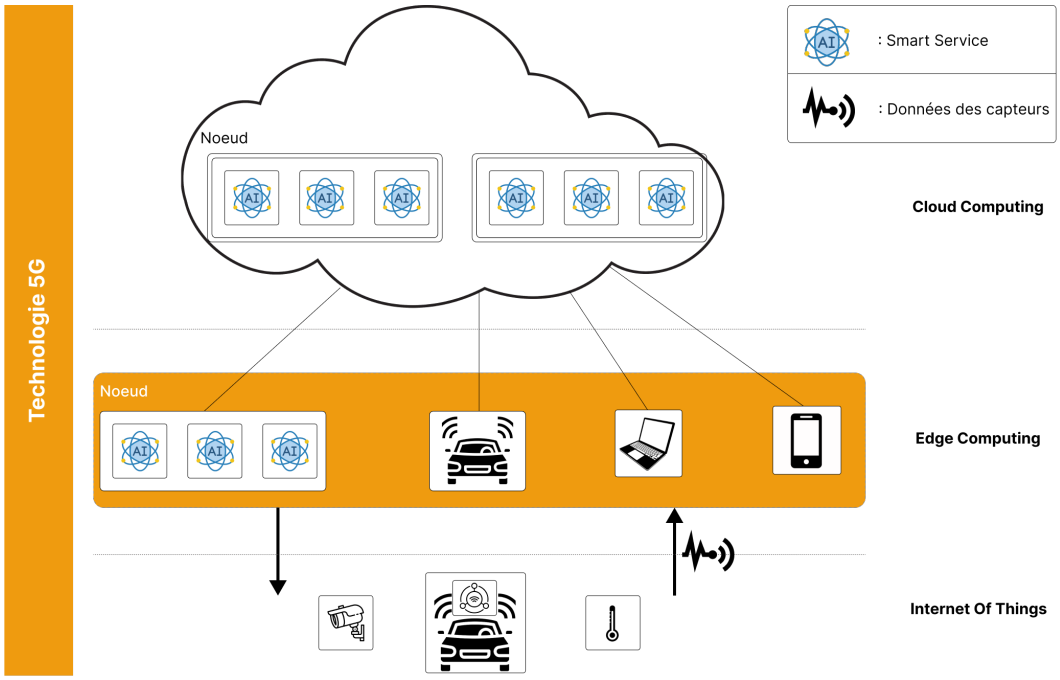


FIGURE 1 : Infrastructure technologique des villes intelligentes

- ***Intelligence Artificielle (IA).*** Les villes intelligentes utilisent également des technologies d'IA et notamment d'apprentissage automatique, ou Machine Learning (ML) en anglais, pour améliorer la prise de décision et automatiser les services urbains [35].

Enjeux de qualité de service pour les véhicules autonomes

Les véhicules autonomes doivent externaliser une partie de leurs calculs vers des ressources externes mais très proches dans la ville. Plus précisément, ils pourraient déployer des modules logiciels appropriés sur les nœuds edge disponibles dans leurs entourages. Ces modules devraient être capables de collecter, d'analyser et de transmettre à la voiture toutes les informations pertinentes pour sa navigation. De plus, les retards de communication entre la voiture et ces modules distants doivent être extrêmement courts pour ne pas affecter le bon fonctionnement de la voiture. Cela est assez difficile à réaliser compte tenu du fait que la voiture est en mouvement. En procédant ainsi, les nœuds edge pourraient être utilisés comme hôtes potentiels pour traiter des applications sensibles à la latence et réduire la quantité de données à acheminer vers le système central.

Cependant, parfois les nœuds edge pourraient ne pas être en mesure de satisfaire les exigences de faible latence, ou ils pourraient être surchargés de travail, ce qui pourrait retarder la communication, ou encore être en état de défaillance et ne pas pouvoir répondre au véhicule. Tous ces scénarios doivent être pris en compte au préalable avant de sélectionner un nœud edge pour déployer le module du véhicule.

Formalisation du problème

L'objectif principal de cette étude est de développer des modèles de prédiction visant à maintenir une QoS optimale pour les véhicules autonomes dans des environnements Edge Computing. Nous mettons particulièrement l'accent sur la prédiction des paramètres de la QoS, ce qui contribuera à la minimisation des perturbations et dégradations potentielles.

La tolérance zéro envers les dégradations de la QoS est essentielle, même pour de courtes périodes, car elles pourraient potentiellement entraîner des accidents graves ou des problèmes critiques. C'est précisément pourquoi nous accordons une importance primordiale aux interventions préventives.

Pour ce faire, il est nécessaire de prédire en continu les paramètres de la QoS pour l'ensemble des nœuds edge. Cela inclut la surveillance en temps réel des paramètres liés à la communication réseau entre les véhicules autonomes et les nœuds edge, ainsi que la charge de travail de ces nœuds, susceptible d'influencer les temps de traitement. Cette tâche exige à la fois une grande précision et, tout aussi important, une rapidité dans la génération des prédictions, afin de permettre au système de réagir efficacement.

Cela nous conduit à une question fondamentale : "**Comment pouvons-nous prédire avec précision les valeurs futures de la QoS pour les nœuds edge, tout en maintenant un coût en termes de temps raisonnable, afin de donner au système le temps nécessaire pour réagir ?**".

Pour répondre à cette question, deux sous-questions se posent :

1. **Quels sont les paramètres de la QoS les plus cruciaux à surveiller pour assurer le bon fonctionnement des véhicules autonomes ?**
2. **Quels modèles ou approches de prédiction peuvent fournir des résultats précis pour ces données, tout en garantissant un temps de génération de prédictions rapide pour permettre au système de réagir efficacement ?**

Approche et méthodologie

Répondre aux questions posées dans la section précédente nous amène à la nécessité de développer et de comparer divers modèles de prédiction de la QoS. Cette démarche contribuera non seulement à la sélection judicieuse des nœuds edge, mais aussi à garantir le bon fonctionnement des véhicules autonomes.

Dans ce travail, nous présentons un processus de développement et d'évaluation de modèles de prédiction de QoS au sein d'un réseau edge. Notre objectif est de trouver un modèle capable de prédire la QoS avec précision, le tout en respectant les contraintes de temps cruciales pour cette application.

Le processus proposé est composé de 3 phases : le prétraitement des données, la construction et l'évaluation des modèles. Il peut être résumé comme suit :

- La collecte des données relatives à la QoS, leur nettoyage et leur préparation pour l'entraînement des modèles.

- L'exploration et la sélection des modèles les plus prometteurs pour la prédiction de la QoS, basée sur notre état de l'art, la création de ces modèles, et leur adaptation au mieux.
- L'évaluation rigoureuse de la précision des modèles, ainsi que de leur temps de prédiction, afin de garantir qu'ils répondent aux exigences de temps nécessaires pour notre cas d'utilisation.

Organisation du mémoire

Ce mémoire est structuré en deux parties : "État de l'art" et "Contribution".

Dans la première partie, "État de l'art", nous explorons divers aspects en deux chapitres. Le premier chapitre, intitulée "Background", se penche sur la compréhension des véhicules autonomes et les défis qui les accompagnent, tout en mettant en lumière les métriques cruciales de la qualité de service (QoS) nécessaires à leur bon fonctionnement. De plus, nous introduisons le concept fondamental de l'Edge Computing, suivi d'une exploration approfondie des séries chronologiques, qui revêtent une importance majeure dans la prédiction des valeurs de la QoS. Cette exploration jette les bases nécessaires pour comprendre notre projet.

Le deuxième chapitre, "Modélisation et prédiction des séries chronologiques de QoS", se concentre sur la modélisation et la prédiction des séries chronologiques de QoS. Nous passons en revue les différentes approches et méthodes utilisées dans la littérature pour analyser et traiter les problèmes de prédiction de la QoS.

La deuxième partie, nommée "Contribution", est composée de trois chapitres. Le troisième chapitre, "Étude de cas", présente le cadre général du projet, qui s'inscrit dans le cadre d'une recherche plus vaste, ainsi que l'architecture et les défis du système actuel.

Le quatrième chapitre, "Construction des modèles de prédiction de QoS", détaille notre processus de développement et d'évaluation de modèles de prédiction de la QoS. Nous présentons les modèles que nous avons sélectionnés, en nous appuyant sur notre revue de la littérature, et exposons les raisons qui ont motivé nos choix.

Enfin, le cinquième et dernier chapitre, "Implémentation et tests", offre un aperçu des résultats de la mise en œuvre de notre processus. Il comprend également une analyse comparative approfondie des différents modèles et approches de prédiction, avant de synthétiser les conclusions tirées de nos tests.

Première partie

État de l'art

Chapitre 1

Background

1.1 Introduction

Afin de mieux appréhender le contexte de ce projet et en comprendre l'importance, il est essentiel de présenter plusieurs concepts fondamentaux. Dans ce chapitre, nous explorerons successivement les véhicules autonomes pour comprendre leur fonctionnement et les défis auxquels ils font face. Nous aborderons également les métriques de QoS essentielles et importantes pour le bon fonctionnement des véhicules autonomes. De plus, nous présenterons le concept clé de l'Edge Computing, qui a permis de répondre aux exigences de haute QoS des véhicules autonomes. Enfin, nous examinerons les séries chronologiques qui sont essentielles dans la modélisation et la prédiction des valeurs de QoS. Ces éléments jetteront les bases nécessaires pour mieux cerner le cadre de notre projet et comprendre notre contribution.

1.2 Véhicules autonomes

Un véhicule autonome, ou autonomous vehicule en anglais (AV), est un véhicule capable de percevoir son environnement et de se déplacer en toute sécurité avec peu ou pas d'intervention humaine. Les véhicules autonomes combinent une variété de capteurs pour percevoir les informations de leur environnement. Ces capteurs pemettent au véhicule de modéliser l'environnement pour identifier d'autres véhicules, les panneaux de signalisation, les piétons, etc. et d'effectuer les manœuvres nécessaires pour conduire les passagers à destination tout en respectant l'intégrité des autres usagers, les règles de circulation tout en évitant les obstacles. Quelques capteurs importants utilisés dans les véhicules autonomes sont illustrés dans la Figure 1.1.

À titre d'exemple, le capteur *RADAR* est utilisé pour mesurer la distance des objets, les *CAMÉRAS* peuvent détecter les panneaux de signalisation, les feux de circulation, les mouvements des piétons, les marquages de voie, etc. On peux les considérer comme les yeux des véhicules.

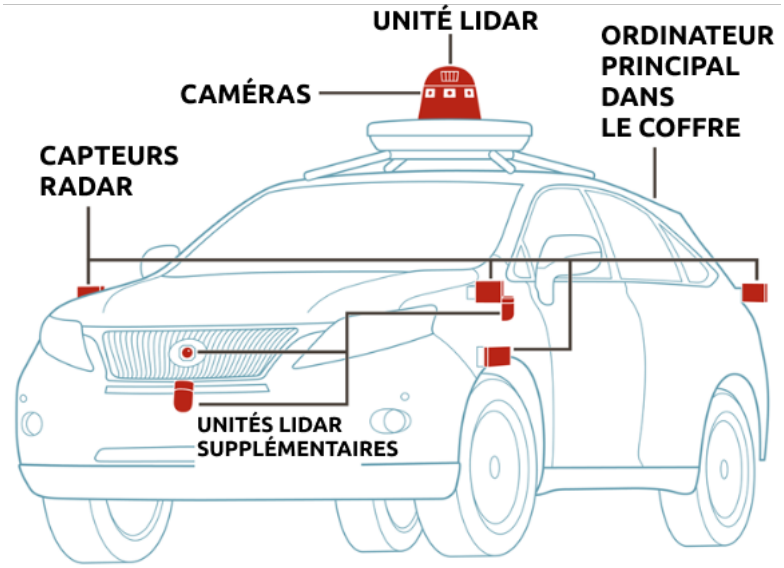


FIGURE 1.1 : Capteurs importants des véhicules autonomes [69]

Ces capteurs, entre autres, et parmi d'autres composants, doivent pouvoir communiquer entre eux et avec leur environnement, et c'est là que l'Internet des objets (IoT) joue un rôle essentiel dans le fonctionnement des véhicules autonomes. L'IoT permet de connecter tous types de dispositifs à Internet afin de partager des informations et d'utiliser des données à valeur ajoutée. Les véhicules autonomes sont ainsi connectés pour partager des informations provenant des capteurs embarqués, mais aussi des smartphones des piétons et des cyclistes, des capteurs de trafic, des détecteurs de stationnement, etc. Faisant ainsi des véhicules autonomes une application concrète de l'IoT.

Le niveau d'autonomie des véhicules varie considérablement d'un véhicule à l'autre. Un véhicule peut être entièrement contrôlé par son conducteur, partiellement autonome ou complètement autonome grâce à l'utilisation de méthodes d'intelligence artificielle pour interagir avec l'environnement et s'adapter aux événements imprévus tels que les accidents, les travaux routiers, les piétons, etc. [85]. Il existe 6 niveaux d'autonomie définis comme suit [85] [40] :

- ***Niveau 0 [Aucune Automatisation].*** À ce niveau, toutes les tâches sont confiées au conducteur, et seules quelques aides technologiques sous forme d'avertissements, comme la détection d'obstacles proches, sont disponibles.

- ***Niveau 1 [Assistance au Conducteur].*** Certaines tâches sont automatisées, telles que l'adaptation de la vitesse par un régulateur de vitesse adaptatif, mais le conducteur reste responsable des manœuvres et peut reprendre le contrôle total du véhicule si nécessaire.

- ***Niveau 2 [Automatisation Partielle].*** La plupart des manœuvres sont automatisées, mais le conducteur les surveille et peut intervenir à tout moment. Par exemple, l'assistance au stationnement est un exemple de ce niveau.

- ***Niveau 3 [Automatisation Conditionnelle].*** Le véhicule contrôle la navigation dans les deux dimensions (longitudinale et latérale), et le conducteur n'a besoin de conduire que dans des situations spécifiques en fonction des demandes du système.

- ***Niveau 4 [Automatisation Avancée].*** Sous des conditions météorologiques favorables, le véhicule prend des décisions opérationnelles et tactiques de manière autonome. Le conducteur n'intervient pas dans les tâches de conduite et de navigation, car toutes les tâches sont automatisées. Il est cependant responsable d'activer et de désactiver le mode automatisé.
- ***Niveau 5 [Automatisation Complète].*** Le véhicule peut effectuer toutes les tâches de conduite dans toutes les conditions météorologiques. La présence d'un être humain aux commandes n'est pas forcément nécessaire.

La Figure 1.2 illustre ces différents niveaux d'autonomie des véhicules, allant de la conduite entièrement contrôlée par le conducteur (Niveau 0) à une automatisation complète où la présence d'un être humain aux commandes n'est plus nécessaire (Niveau 5).

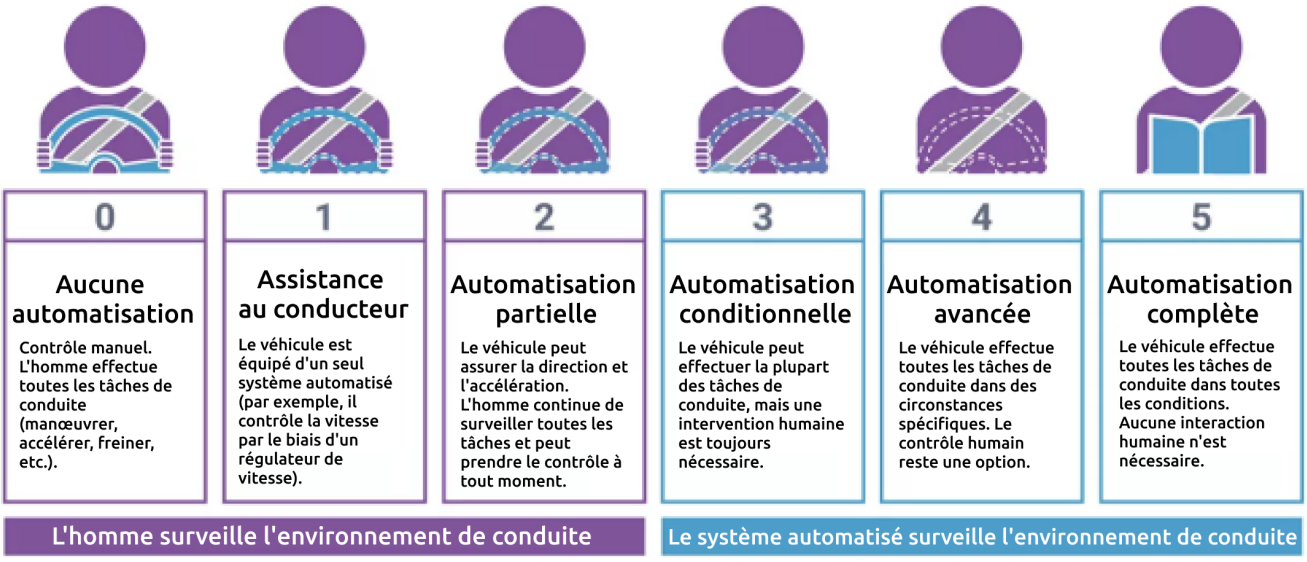


FIGURE 1.2 : Niveaux d'autonomie des véhicules [68]

1.2.1 Impacts et défis des véhicules autonomes

Les véhicules autonomes présentent des impacts positifs majeurs, tout d'abord ils améliorent considérablement l'expérience et la productivité des usagers, qui pourront se consacrer à d'autres tâches pendant leurs trajets. De plus, ils offrent une nette amélioration de la sécurité des déplacements, car les systèmes de pilotage autonomes sont plus rapides et fiables que l'homme (94% des accidents étant causés par des erreurs humaines [81]), et leur temps de réaction n'est pas sujet aux risques de dégradation (fatigue, prise de médicaments, etc.). Enfin, les véhicules autonomes contribuent à la fluidité du trafic, facilitant ainsi l'orchestration du réseau routier, et permettent une réduction de la consommation et des émissions de gaz à effet de serre grâce à l'optimisation des vitesses et des trajets, ainsi qu'à la disponibilité des transports publics en permanence.

Cependant, les véhicules autonomes font face à des limites techniques en raison de la complexité du trafic, exigeant un système robuste, fiable et performant en termes de perception de l'environnement. De plus, l'absence de réglementation concernant cette branche et les obstacles juridiques subsisteront tant que la question de la responsabilité en cas d'accident (conducteur, propriétaire, constructeur) ne sera pas résolue.

Par ailleurs, la question de la sécurité soulève des interrogations sur la sécurisation des systèmes, notamment en ce qui concerne la prise de contrôle du véhicule à distance.

Il est également important de noter que ces véhicules entraînent des coûts et des changements nombreux du point de vue humain et social, qui devront être pris en compte et supportables pour nos écosystèmes actuels.

1.3 Internet des Objets (IoT)

L'Internet des Objets (IoT) est un réseau d'objets physiques équipées de technologies de communication et d'information (TIC) qui se connectent à Internet [61]. L'IoT permet des services avancés pour la communication, le contrôle à distance, la gestion et la maintenance des objets physiques et des processus [21].

Les véhicules autonomes sont un exemple de service émergent de l'IoT, où ils ont besoin d'un traitement rapide et précis des données sensorielles pour éviter les collisions.

Le fonctionnement des système IoT peut être représenté en quatre étapes clés comme le présente Figure 1.3 [77] :

- ***La capture des données.*** Les capteurs et les terminaux IoT capturent les données de leur environnement (relevé de température, capture d'un évènement sous forme de flux vidéo, etc.).
- ***Le partage des données.*** Les données précédemment relevées seront mises à dispositions sur un cloud public ou privé via les connexions réseaux existantes.
- ***Le traitement des données.*** Les données recueillies seront analysées par logiciel pour déclencher des actions telles que la fermeture d'une porte ou le déclenchement d'une alarme.
- ***L'exploitation des données.*** Les données collectées de tous les terminaux d'un réseau IoT sont analysées. Il en ressort de solides insights qui aident à prendre des décisions et des mesures en toute confiance.

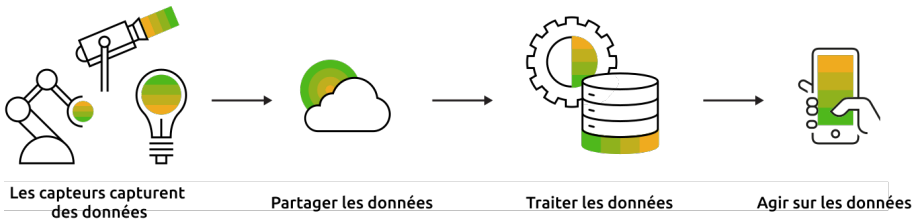


FIGURE 1.3 : Étapes de l'IoT [77]

Ces étapes sont divisées dans une architecture en 5 couches, chacune ayant un rôle spécifique dans le fonctionnement du système [1] :

- ***La couche perception.*** Cette couche rassemble divers dispositifs, tels que différents types de capteurs, conçus pour capturer des grandeurs physiques analogiques caractéristiques de leur environnement, comme l'humidité et le mouvement. Ces informations sont ensuite traitées et transmises à la couche réseau.

- ***La couche réseau.*** Aussi appelée couche de transmission, elle a pour objectif de router les informations captées vers la couche middleware de manière sécurisée, en utilisant des supports de communication tels que le WiFi ou le Bluetooth.

- ***La couche middleware.*** Cette couche sert d'interface entre la couche matérielle et la couche applicative. Elle reçoit les données de la couche réseau, les stocke dans une base de données et les soumet à des traitements et calculs pour créer de nouvelles informations.

- ***La couche applicative.*** Responsable de la distribution des diverses applications aux utilisateurs dans le cadre de l'IoT. Ces applications peuvent être liées à différents domaines tels que le transport, l'agriculture, etc.

- ***La couche affaires (Business).*** Cette couche vise à mettre en place des modèles d'entreprise, des graphiques et des organigrammes basés sur les données reçues de la couche applicative. Ces modules sont utilisés pour gérer le système IoT de manière efficace.

La Figure 1.4 représente visuellement cette architecture IoT.

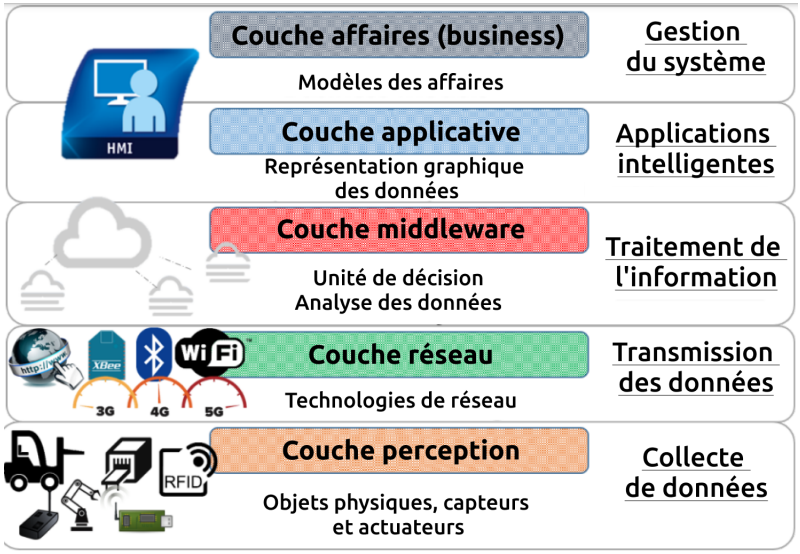


FIGURE 1.4 : Architecture IoT [1]

1.4 Métriques de QoS pour les véhicules autonomes

Dans cette section, nous abordons les métriques de qualité de service essentielles pour assurer le bon fonctionnement des véhicules autonomes. Ces véhicules, requièrent un traitement rapide et précis des données sensorielles pour éviter les collisions. Parallèlement, ils imposent des exigences strictes en matière de latence dans les communications avec d'autres dispositifs. La qualité des communications des véhicules autonomes revêt une importance capitale, étant donné leur intégration profonde dans la vie quotidienne. Nous explorons comment ces métriques influent sur la sécurité des individus et leur environnement.

1.4.1 Concept de Qualité de service

La qualité de service (QoS) se réfère à la capacité intrinsèque d'un système à répondre aux exigences spécifiques qui lui sont imposées [17]. Cela signifie que le système doit être en mesure de satisfaire ces exigences de manière efficace et fiable. Dans un réseau informatique, la QoS se réfère à la capacité du réseau à garantir une transmission fluide et sans interruption des données, en accordant une priorité aux applications qui nécessitent une performance élevée. La QoS évalue la capacité d'un système à fournir une performance conforme aux attentes et aux besoins spécifiques.

La QoS peut être évaluée et mesurée à l'aide de divers paramètres appelés indicateurs clés de performance, Key Performance Indicators (KPI) en anglais. Ces KPI peuvent être classés en deux catégories principales : KPI de Performance Système et KPI de Performance Réseau [31].

Les KPI de Performance Système englobent des facteurs tels que le CPU, la mémoire, la consommation d'énergie, le système d'exploitation, les caractéristiques de détection, etc. Ce sont des paramètres intrinsèques au système qui influencent directement son fonctionnement.

D'autre part, les KPI de Performance Réseau concernent des mesures liées à la transmission et à la communication des données. Ils incluent des paramètres tels que le délai, la gigue, la perte de paquets, la bande passante, la taille du tampon, le RSSI (Received Signal Strength Indicator), le RSRQ (Reference Signal Received Quality), le SNR (Signal-to-Noise Ratio), et bien d'autres. Ces KPI évaluent la performance du réseau dans la livraison efficace des données. Le tableau 1.1 résume certaines métriques d'évaluation de QoS.

	KPI	**Définition**
KPI de performance Réseau (liste non exhaustive)	Le délai	Le temps nécessaire pour que le premier octet (byte) arrive à la destination.
	La latence	Le temps entre l'envoi d'un signal ou d'une donnée et sa réception à destination.
	Le débit	La vitesse de transmission sur le support de communication.
	La gigue	Mesure la variation des délais de transmission.
	Le taux de perte	La proportion d'éléments de données non livrés avec succès dans un réseau.
KPI de performance Système (liste non exhaustive)	La charge du CPU	Le niveau d'utilisation du processeur du serveur.
	La charge du RAM	L'utilisation et la disponibilité de la mémoire système au serveur.
	La consommation d'énergie	Quantifie la quantité d'énergie électrique consommée par un système.

TABLE 1.1 : Indicateurs clés de performance (KPI) de QoS.

1.4.2 Besoin en QoS des véhicules autonomes

Pour les véhicule autonomes, une seule décision d'un véhicule dépend de diverses sources de données, collectées à partir de multiples canaux, créant des chaînes complexes interconnectées. Par exemple,

un véhicule autonome fusionne les sorties de ses différents modules internes, tels que les systèmes d'évitement de collision, de maintien de trajectoire, de détection d'objets en mouvement et de panneaux, afin d'assurer une conduite sécurisée et efficiente. Chacun de ces modules traite les informations provenant du matériel du véhicule, mais s'appuie également sur les données captées de son environnement. De plus, le traitement de ces informations peut être délégué à un serveur distant. Cependant, toute défaillance ou retard dans la prise de décision peut induire le véhicule autonome à prendre des mesures incorrectes, mettant potentiellement en péril des vies humaines.

Les véhicules autonomes sont soumis à des exigences strictes en matière de réactivité en temps réel. Par exemple, un être humain réagit en 100 à 150 millisecondes à un nouveau signal dans les meilleures conditions [34]. En conduite autonome, cela équivaut au temps entre la réception d'une entrée provenant d'un capteur ou de l'environnement, le traitement de cette information et la prise d'une décision de contrôle. Afin d'atteindre un niveau de sécurité comparable à celui des conducteurs humains, les véhicules autonomes devraient réagir en moins de 100 millisecondes [27], ce qui constitue une contrainte cruciale en matière de latence. Tout retard dans la prise de décision peut avoir des conséquences catastrophiques.

La latence de bout en bout, ou simplement latence, représente le délai qui s'écoule entre le moment où une demande est initiée et celui où la réponse correspondante est reçue. Dans le contexte des véhicules autonomes, la latence de bout en bout englobe le temps qu'il faut pour traiter, transmettre et recevoir les données nécessaires à la prise de décisions critiques. La latence s'impose comme la contrainte la plus critique au sein d'un système de conduite autonome [25]. Elle est influencée par deux facteurs principaux : le temps de calcul et le temps de transfert de données.

Le temps de calcul se réfère à la durée nécessaire pour traiter et analyser les données dans un système, tandis que le temps de transfert de données représente la durée pour déplacer des données entre des points dans un réseau ou canal de communication. Ces deux facteurs concernent respectivement la performance du système de traitement, tel que les nœuds Edge qui exécutent des tâches pour les véhicules, et la performance du réseau qui assure le transport de l'information. Ils sont évalués à travers les KPI du système et du réseau.

1.5 Edge computing pour les véhicules autonomes

Les systèmes actuels basés sur le cloud computing font face à plusieurs défis, notamment en matière de sécurité, de latence dans les communications et de performances médiocres, en raison de leur architecture centralisée de serveur. L'informatique en périphérie, edge computing en anglais, offre une solution à ces problèmes en déployant des serveurs, appelés serveurs edge ou nœuds Edge, à l'emplacement physique de l'utilisateur ou à proximité de celui-ci, permettant ainsi le traitement des données à proximité de leur source [12].

Le Multi-access Edge Computing (MEC), ou Informatique en Périphérie à Accès Multiples en français, est une mise en pratique concrète du concept d'edge computing. Tout comme l'edge computing, le MEC vise à résoudre les problèmes de latence en améliorant la fiabilité et l'efficacité globale du réseau et en éliminant le trajet réseau long entre les serveurs et les appareils des utilisateurs finaux.

Le nœud edge est un dispositif qui fournit des capacités de calcul et de traitement avancées à proximité de la source des données. Les exemples de nœuds Edge comprennent les routeurs, les commutateurs réseau, les points d'accès, et même les appareils mobiles à proximité tels que les télé-

phones portables, qui peuvent être utilisés comme des nœuds Edge.

Le Cloud Computing fonctionne de manière centralisée, où les applications à forte intensité de calcul sont déchargées dans le cloud central. Dans le contexte des véhicules autonomes, le Cloud Computing n'est pas efficace et ne peut pas être mis à l'échelle pour gérer d'énormes quantités de données véhiculaires. Certaines des limites du Cloud Computing sont les suivantes :

- Il fonctionne de manière centralisée, ce qui impose des coûts significatifs en termes d'énergie, de bande passante et de temps lors de la transmission des données entre les véhicules et le cloud, ou vice versa.
- Il est difficile de gérer et de mettre à jour en temps réel les informations de mobilité et de localisation.
- Les serveurs de Cloud Computing ne peuvent pas extraire les informations en temps réel des véhicules en raison des multiples sauts entre les véhicules et le serveur cloud.

Par conséquent, cela dégrade la qualité de service de l'utilisateur. Dans la communication entre véhicules, l'un des critères clés, sinon le plus important, est la faible latence. Le MEC peut devenir une infrastructure clé en rapprochant certaines fonctionnalités centrales des applications finales [45]. Une comparaison détaillée entre le Cloud Computing et le MEC est présentée dans le Tableau 1.2.

Éléments	**MEC**	**Cloud Computing**
Architecture	Distribuée	Centralisée
Opérateurs	Opérateurs mobiles	Fournisseurs de cloud
Utilisateurs cibles	Appareils mobiles	Utilisateurs d'Internet
Hiérarchie	Trois niveaux	Deux niveaux
Stockage de données	Courte durée	Longue durée
Noeuds de serveur	En grand nombre	En petit nombre
Exigences de bande passante	Basées sur les données envoyées aux clouds	Basées sur les données générées par les utilisateurs
Conscience de l'emplacement	Oui	Non
Surcharge de communication	Moyenne	Élevée
Accès à l'information	Informations localisées	Informations globalisées
Tolérance au délai	Inférieure à 10 ms	Quelques secondes ou plus

TABLE 1.2 : Comparaison entre le MEC et le cloud computing

1.5.1 Multi-access Edge Computing et 5G

La 5G, ou la cinquième génération de réseaux mobiles, représente une norme révolutionnaire de communication sans fil. Elle apporte des capacités avancées telles que des communications à très haut débit (eMBB, pour Enhanced Mobile Broadband), une latence très faible (dans le cadre d'URLLC, pour

Ultra-Reliable Low-Latency Communications) et la capacité de connecter simultanément un grand nombre d'appareils (dans le domaine du mMTC, pour Massive Machine Type Communications). Cependant, pour exploiter pleinement ces performances, il est nécessaire de repenser l'architecture globale du réseau [43].

La technologie MEC a été spécifiquement développée dans ce contexte. En intégrant les capacités de calcul, de stockage et de traitement des données au niveau de la périphérie du réseau, la MEC vise à réduire la charge sur le réseau central, tout en offrant des avantages tels qu'une latence réduite et une sécurité accrue.

La Figure 1.5 présente l'architecture MEC dans le contexte de la 5G. Plus précisément, le déploiement de MEC apporte un soutien solide aux trois principales catégories d'applications de la 5G : les applications nécessitant un grand nombre de capteurs IoT (mMTC), les applications nécessitant une latence ultra-faible (uRLLC) et les applications nécessitant des communications à très haut débit (eMBB).

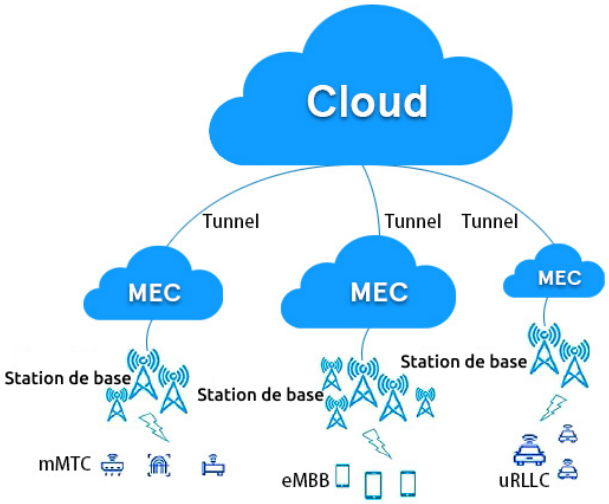


FIGURE 1.5 : Architecture MEC dans la 5G [86]

La 5G fournit la connectivité rapide et fiable nécessaire pour acheminer les données entre les dispositifs et les réseaux, tandis que la technologie MEC offre des capacités de calcul et de traitement à proximité de l'utilisateur. Cette synergie facilite la création d'expériences interactives en temps réel, particulièrement essentielles pour les applications demandant une latence ultra faible, telles que les véhicules autonomes. La technologie MEC permet ainsi d'exploiter pleinement les atouts de la 5G en déployant des capacités de calcul et de traitement en périphérie du réseau, c'est pourquoi ces deux technologies sont étroitement liées.

1.5.2 Rôle clé de l'edge computing pour les véhicules autonomes

En plus de garantir une faible latence, nécessaire au bon fonctionnement des véhicules autonomes, grâce au calcul à proximité, l'edge computing facilite l'exécution de nombreuses tâches autrement difficiles, voire impossibles. Dans ce qui suit, nous explorons l'importance de l'edge computing pour les véhicules autonomes à travers trois scénarios distincts, mettant en évidence les avantages concrets qu'il offre.

1.5.2.1 Scénario1 : Connaissance en temps réel de la situation routière locale

Pour les véhicules autonomes, savoir ce qui se passe en temps réel sur la route est crucial, surtout dans des zones routières critiques où les conditions de conduite peuvent changer rapidement (par exemple, la détection récente d'un nouveau cône de signalisation par un autre véhicule). De plus, il est essentiel de disposer de cartes locales haute définition qui peuvent être téléchargées depuis un serveur central.

Ce scénario ne se limite pas à la simple diffusion d'informations sur les conditions routières qui évoluent. Il s'étend également aux situations où il est nécessaire de partager et de regrouper instantanément des informations locales auprès de tous les participants à la circulation.

Dans ce contexte, l'edge computing joue un rôle crucial. Il permet de traiter *en temps réel* et *localement* les données nécessaires pour une compréhension précise de la situation routière. Cette approche offre une alternative à laisser les véhicules autonomes créer et gérer eux-mêmes leur propre connaissance en temps réel en utilisant les informations reçues de leurs pairs. L'edge computing optimise cette information en la fusionnant avec d'autres sources disponibles, puis en diffusant efficacement de grandes quantités de données à de nombreux utilisateurs localement.

1.5.2.2 Scénario2 : Transparence (pour le dépassement)

Dans les cas d'utilisation, des véhicules en convoi partagent les images des caméras des conditions routières devant eux avec les véhicules les suivant.

En particulier, l'application de Transparence (pour le dépassement) est un exemple d'application d'assistance à la Conduite. Elle avertit le véhicule hôte qui a l'intention de dépasser un véhicule distant des dangers potentiels, tels qu'un véhicule distant arrivant en sens inverse dans la voie de dépassement ou un manque d'espace devant le véhicule distant. L'objectif de ce cas d'utilisation est de prévenir les collisions frontales catastrophiques pendant une manœuvre de dépassement.

La Figure 1.6 illustre le scénario ainsi que les principaux états opérationnels identifiant ce cas d'utilisation :

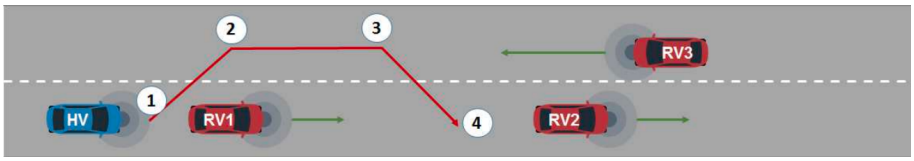


FIGURE 1.6 : Transparence (pour le dépassement)

- **État 1** = Le HV commence à recevoir la vidéo en continu du RV1.
- **État 2** = Le HV s'est entièrement déplacé dans la voie de dépassement, continue de recevoir la vidéo en continu du RV1.
- **État 3** = Le HV a atteint la position dans la voie de dépassement où il est prêt à commencer la manuvre pour revenir à la voie de départ.
- **État 4** = Le HV achève la manœuvre de dépassement et peut cesser de recevoir la vidéo en continu du RV1.

L'objectif est de fournir au véhicule HV une vision claire, fiable et en temps réel de la route devant lui lorsqu'il tente de dépasser un autre véhicule, tout en aidant à prévenir toute collision potentielle.

De plus, l'application devra réaliser plusieurs calculs, tels que l'estimation de la distance et du trajet nécessaires pour accomplir la manoeuvre de dépassement, la trajectoire du RV3 qui approche, l'écart estimé entre RV1 et RV2 à la fin de la manoeuvre, ainsi que le risque de collision entre HV et RV3 si leurs trajets se chevauchent. Par conséquent, la présence d'un serveur MEC à proximité peut être bénéfique non seulement pour abriter la puissance de calcul nécessaire au traitement des données, mais aussi (et surtout) pour diffuser la vidéo en temps réel et éventuellement effectuer le traitement vidéo en périphérie (ce qui correspond à un cas d'utilisation pertinent en MEC [12]).

L'Edge computing se révèle être une solution idéale pour des cas d'utilisation tels que celui-ci, où la communication à très faible latence et le contexte local jouent un rôle clé. La technologie MEC offre un canal de communication en temps réel avec les véhicules participants dans la zone, ainsi qu'une puissance de calcul pratiquement illimitée par rapport à l'environnement de processeur intégré dans le véhicule. Un serveur MEC peut également tirer parti d'informations supplémentaires qui ne sont pas directement accessibles pour les véhicules participants, provenant d'autres équipements dans la région.

1.5.2.3 Scénario3 : Détection des usagers vulnérables de la route

Ce cas d'utilisation concerne les usagers vulnérables de la route tels que les piétons et les cyclistes. Une exigence essentielle pour permettre une utilisation efficace des informations fournies par les usagers vulnérables est l'exactitude des informations de positionnement fournies par ces participants à la circulation. Des moyens supplémentaires pour utiliser les informations disponibles afin d'obtenir une meilleure précision fiable sont essentiels pour permettre une utilisation pratique des informations partagées par les usagers vulnérables. Les usagers vulnérables signalent leur présence/leur emplacement via leurs appareils mobiles (par exemple, smartphone, tablette), ainsi que l'utilisation de ces informations par les véhicules, constitue un élément clé important pour améliorer la sécurité routière et éviter les accidents.

Dans cet exemple, le véhicule HV qui prévoit de tourner à gauche est alerté de la présence d'un usager vulnérable (pouvant être un piéton, un cycliste, une moto ou une autre entité). La Figure 1.7 ci-dessous illustre le scénario impliquant un véhicule et un piéton.

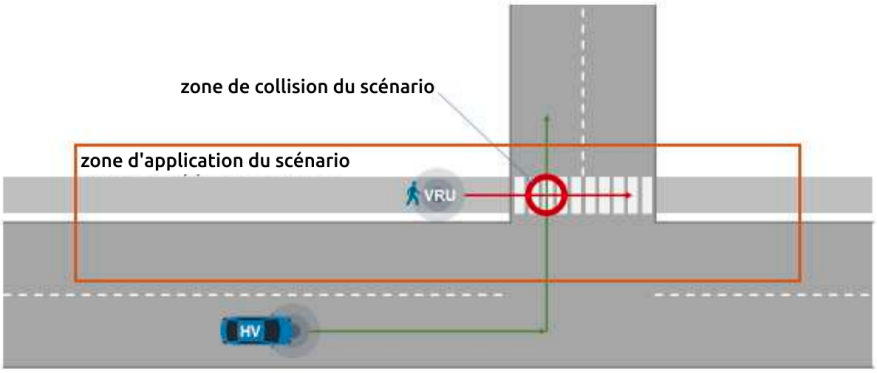


FIGURE 1.7 : Détection des usagers vulnérables de la route

Pour que l'application émette un avertissement valable, elle devra calculer les trajectoires du HV et de l'usager vulnérable, connaître la géométrie de la route et de l'intersection, et déterminer le risque de collision dans la période où leurs trajectoires se croisent.

L'Edge Computing s'avère être une solution idéale dans ce contexte, en particulier en exploitant le contexte local et les informations disponibles dans la zone. Cela permet d'améliorer la précision des informations de positionnement pour l'ensemble des participants à la circulation. En effet, un serveur MEC agit comme un serveur central qui a accès à toutes ces informations, tandis que le HV seul ne pourrait pas avoir une vue aussi étendue de la situation.

1.5.3 Défis de l'edge computing pour les véhicules autonomes

Les scénarios présentés dans la section précédente soulignent plusieurs défis d'ordre technique. Les solutions d'edge computing peuvent apporter une contribution significative à la réalisation efficace de nombreux scénarios intéressants pour les véhicules autonomes, en particulier lorsque les exigences en matière de performances et de systèmes sont complexes, notamment en termes de faible latence, de souplesse de déploiement et d'accès à des informations riches en contexte local. Cependant, ces solutions sont confrontées à divers défis. Nous abordons ci-dessous quelques-uns des problèmes et défis inhérents à l'edge computing dans le contexte des véhicules autonomes [47].

1.5.3.1 Disponibilité des données

Dans l'edge computing, les données peuvent être stockées, copiées et/ou mises en cache à divers endroits du réseau. À mesure que le nombre de véhicules connectés aux serveurs MEC augmente, la mise en cache dans le réseau, ainsi que la réplication des données, sont essentielles pour assurer la disponibilité des données. Le grand nombre de véhicules connectés entrave la disponibilité en temps réel des informations critiques. De plus, un service de disponibilité des données sans interruption n'est pas garanti lorsque plusieurs véhicules demandent les mêmes informations simultanément ou si les serveurs MEC sont perturbés par des cyberattaques.

1.5.3.2 Gestion de la confidentialité des données

La confidentialité revêt une importance critique pour la protection des informations privées et sensibles concernant les véhicules dans le cadre de l'edge computing. Les utilisateurs des véhicules autonomes font parfois confiance aux serveurs MEC pour gérer les applications sans savoir comment les informations circulent et qui utilise réellement leurs données. En cas d'attaque des serveurs MEC, un volume considérable d'informations peut être divulgué, mettant en danger la confidentialité des utilisateurs de véhicules.

1.5.3.3 Hétérogénéité et diversité

Le réseau MEC présente une grande diversité en termes de conception, incluant les systèmes d'exploitation, les structures, les appareils connectés et les services. Des problèmes surgissent lorsque différentes entreprises qui fournissent des services edge/cloud échangent des données et des applications [33]. Ce défi n'est pas simple à résoudre en raison de l'utilisation de différentes méthodes de communication sans fil, de diverses façons de virtualiser ou d'interfaces incompatibles. Il est crucial d'assurer une transition fluide des données entre les serveurs MEC et les véhicules.

1.6 Séries chronologiques

L'anticipation des valeurs futures de la qualité de service (QoS) avec une marge d'erreur réduite peut renforcer la sélection des nœuds edge offrant des meilleures performances pour les véhicules autonomes. Parmi les méthodes pertinentes pour effectuer de telles prédictions, la prédiction de séries chronologiques se distingue.

Une série chronologique peut être définie comme "un ensemble d'observations ordonnées x_t, chacune étant enregistrée à un moment spécifique t, collectées à intervalles de temps réguliers" [38].

Les paramètres de Qualité de Service (QoS) sont intrinsèquement liés au temps, car ils peuvent évoluer de manière significative au fil du temps en réponse à divers facteurs et conditions changeantes [58]. Par exemple, considérons la latence d'un réseau : elle peut varier en fonction de la congestion du réseau, de la charge du système et d'autres facteurs dynamiques.

La prédiction des séries chronologiques se révèle essentielle pour capturer cette dynamique temporelle complexe des paramètres de QoS. Contrairement aux approches qui supposent une distribution constante dans le temps, les méthodes de prédiction des séries chronologiques permettent de modéliser les tendances et variations temporelles inhérentes aux données de QoS. Ces modèles prennent en compte les relations temporelles entre les observations passées et actuelles pour extrapoler les tendances futures avec une meilleure précision [26]. Ainsi, dans le contexte de l'évolution dynamique des paramètres de QoS au fil du temps, la prédiction des séries chronologiques se positionne comme l'approche la plus apte à saisir et à anticiper ces variations et tendances, offrant ainsi des prévisions plus fiables et précises.

1.6.1 Prédiction des séries chronologiques

L'objectif de la prédiction des séries chronologiques est l'estimation des valeurs futures d'une séquence temporelle en analysant les comportements et les expériences antérieurs.

Pour toute prise de décision basée sur les prédictions des séries chronologiques, une anticipation précoce du futur est nécessaire. Les prédictions visent à parvenir à la meilleure évaluation des événements futurs.

D'un point de vue méthodologique, la prédiction des séries chronologiques implique de prendre une série existante de données $x_{t-w}, ..., x_{t-2}, x_{t-1}, x_t$ et d'estimer les valeurs de données $x_{t+1}, x_{t+2}, ..., x_{t+h}$ à venir.

Dans le contexte de la prédiction des séries chronologiques, deux éléments essentiels jouent un rôle crucial : la fenêtre temporelle (window length) et l'horizon de prédiction (forecasting horizon). Ces facteurs influencent la manière dont les prédictions sont formulées et la portée temporelle des prédiction futures. La Figure 1.8 illustre visuellement la fenêtre temporelle et l'horizon de prédiction, où w est la fenêtre temporelle et h est l'horizon de prédiction.

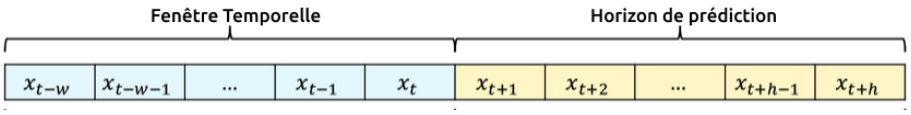


FIGURE 1.8 : Fenêtre temporelle et horizon de prédiction

1.6.1.1 Fenêtre temporelle (Window Length)

La fenêtre temporelle (Window Length) fait référence au nombre d'observations passées prises en compte lors de la création d'une prédiction. C'est un élément crucial car il détermine la quantité d'informations historiques utilisées pour estimer les valeurs futures. Par exemple, si une fenêtre temporelle de 7 jours est choisie pour prédire le trafic routier, cela signifie que les données des 7 derniers jours seront utilisées pour formuler la prédiction.

1.6.1.2 Horizon de prédiction (Forecasting Horizon)

L'horizon de prédiction (Forecasting Horizon) se rapporte à la période de temps pour laquelle les prédictions sont générées. Il définit combien d'étapes dans le futur les prédictions vont s'étendre.

Dans le contexte des véhicules autonomes, l'importance de l'horizon de prédiction est particulièrement cruciale. Plus précisément, la prédiction de la Qualité de Service (QoS) est fortement influencée par cet horizon. Pour illustrer cela, considérons une situation où une adaptation de vitesse est envisagée en réponse à une perte de communication. Un horizon de prédiction adéquat devient essentiel pour prendre des décisions réfléchies et sécurisées. Si une situation exigeant un arrêt complet se présente, anticiper et décélérer légèrement à l'avance pour maintenir une expérience utilisateur optimale est préférable à un arrêt brusque [25].

En fonction de la taille de l'horizon de prédiction, on peut classifier le problème de prédiction en deux catégories distinctes :

- ***Prédictions à un pas en avant*** (1-step ahead). Par exemple, pour une prédiction à un pas en avant, on tente de prédire la valeur à l'instant $t + 1$ en se basant sur les observations jusqu'à l'instant t.

- ***Prédictions à plusieurs pas en avant*** (multi-steps ahead, ou N-steps ahead, où N représente la taille de l'horizon). Par exemple, dans les prévisions à plusieurs pas en avant, on tente de prédire les valeurs pour les instants $t+1, t+2, t+3$, et ainsi de suite, en utilisant les observations jusqu'à l'instant t.

1.6.1.3 Types de séries chronologiques

Le nombre de variables observées dans le temps revêt une importance cruciale. Cela a conduit à deux types distincts des séries chronologiques : Séries Chronologiques Univariées et Séries Chronologiques Multivariées.

- ***Séries Chronologiques Univariées.*** Ce sont des ensembles de données où une seule variable est observée à chaque instant, comme la température chaque heure.

- ***Séries Chronologiques Multivariées.*** Ce sont des ensembles de données où deux variables ou plus sont observées à chaque instant.

Par exemple, on considère la prédiction des paramètres de QoS. Si on ne considère qu'une seule variable, comme le débit de données, alors il s'agit d'une série chronologique univariée. Cependant, si on prend en compte plusieurs variables telles que le débit de données, la latence de communication et la bande passante, alors la série chronologique est dite multivariée. La distinction entre ces deux types revêt une importance majeure dans la formulation et la résolution des problèmes de prédiction des séries chronologiques.

Dans une série univariée, une seule variable est ciblée pour la prédiction, bien que d'autres variables puissent être présentes [38]. Par exemple, si on considère un véhicule et on souhaite prédire le débit de données, la série chronologique est considérée comme univariée. D'autres variables telles que la vitesse du véhicule, la mobilité, etc., peuvent être présentes en tant que variables exogènes, mais l'accent est mis sur la prédiction du débit de données. Ainsi, cette série est qualifiée d'univariée et non de multivariée. Pour différencier plus clairement ces deux types, il suffit d'identifier le nombre de variables ciblées que l'on souhaite prédire. La Figure 1.9 illustre la distinction entre les séries chronologiques multivariées et univariées lorsqu'on considère un horizon de prédiction égal à 1 (1-step ahead).

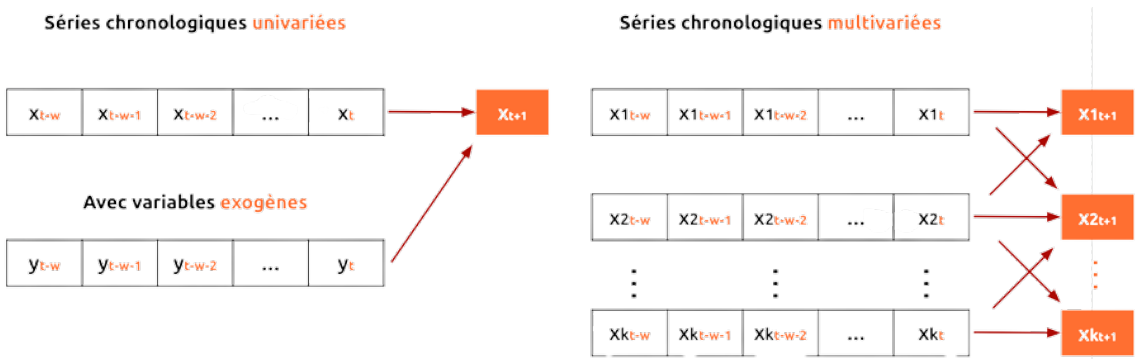


FIGURE 1.9 : Distinction entre les séries chronologiques multivariées et univariées

1.6.2 Modèles de prédiction des séries chronologiques

L'objectif central de la prédiction des séries chronologiques consiste à concevoir des modèles capables de capturer les tendances et les relations des données temporelles. Ceci permet ainsi la production de prédictions fiables pour les valeurs futures. Cette démarche aspire à élaborer un modèle qui assimile les comportements passés et qui peut extrapoler ces acquis afin d'anticiper avec précision les évolutions à venir.

Lorsqu'il s'agit d'explorer les modèles de prédiction des séries chronologiques, deux catégories principales se dessinent : 1) les modèles structurés, souvent appelés classiques ou traditionnels, et 2) les modèles non-structurés, également connus sous le nom de modèles basés sur l'apprentissage automatique (ML) [3]. Cette classification offre un aperçu éclairant des approches adoptées par ces modèles, mettant en lumière leurs dissemblances fondamentales dans le traitement des données temporelles.

D'un côté, les modèles structurés sont construits autour d'un cadre prédéfini visant à anticiper les évolutions d'une série chronologique. Dans ce type de modèle, les relations entre les données historiques et les valeurs futures sont spécifiées à l'avance [3]. Il suit une formule précise et cherche simplement à ajuster les paramètres pour relier les valeurs historiques de la série à ses valeurs futures, en se basant souvent sur des calculs linéaires.

De l'autre côté, les modèles non structurés, tels que les réseaux de neurones, possèdent la flexibilité d'apprendre automatiquement des relations plus complexes et non linéaires entre différentes données temporelles. Au lieu de suivre des relations prédéfinies par une formule, ils sont capables d'explorer diverses combinaisons non linéaires entre les données passées et futures afin d'identifier

des schémas complexes et subtils. Cette approche peut se révéler particulièrement utile lorsque les interactions entre les variables temporelles sont difficiles à exprimer de manière linéaire.

Dans ce qui suit, nous allons examiner quelques-uns des modèles les plus couramment employés dans chaque catégorie pour la prédiction des séries chronologiques.

1.6.2.1 Stationnarité des séries chronologiques

La stationnarité est un concept fondamental dans la prédiction de séries chronologiques. Une série chronologique stationnaire est celle dont les propriétés statistiques, telles que la moyenne et la variance, restent constantes au fil du temps. En d'autres termes, le comportement de la série temporelle n'affiche aucun changement systématique ni tendance [7].

Pourquoi la stationnarité est-elle importante ? De nombreux modèles de prédiction de séries chronologiques, notamment les modèles structurés, supposent la stationnarité des données. La stationnarité simplifie le processus de modélisation en garantissant que les relations entre les valeurs passées et futures restent cohérentes. Les modèles structurés s'appuient souvent sur l'hypothèse que les motifs sous-jacents des données se répéteront, et cette hypothèse est plus susceptible d'être vérifiée dans les séries stationnaires.

Dans un modèle structuré, l'hypothèse de stationnarité garantit que les coefficients estimés à partir des données historiques sont pertinents et applicables aux prédictions futures. Cependant, les séries chronologiques réelles ne sont souvent pas stationnaires par nature. Elles peuvent présenter des tendances, des saisons ou d'autres motifs non constants. Dans de tels cas, il devient nécessaire de transformer les données pour atteindre la stationnarité. La différenciation, qui consiste à calculer la différence entre les observations consécutives, est souvent utilisée pour rendre la série stationnaire. Les coefficients sont estimés en se basant sur les données différenciées, les prédictions sont générées, puis l'opération inverse est effectuée. Parfois, il est nécessaire d'appliquer la différenciation plusieurs fois pour obtenir une série stationnaire.

En revanche, les modèles non structurés sont capables d'apprendre la nature non linéaire et non stationnaire d'une série chronologique, de sorte que la différenciation n'est pas nécessaire dans ce cas.

La Figure 1.10 illustre la différence entre une série non stationnaire (en haut) où l'on observe clairement une tendance à la hausse, et une série stationnaire (en bas). En fait, la série en bas est la même que celle en haut après l'application de la différenciation pour la rendre stationnaire.

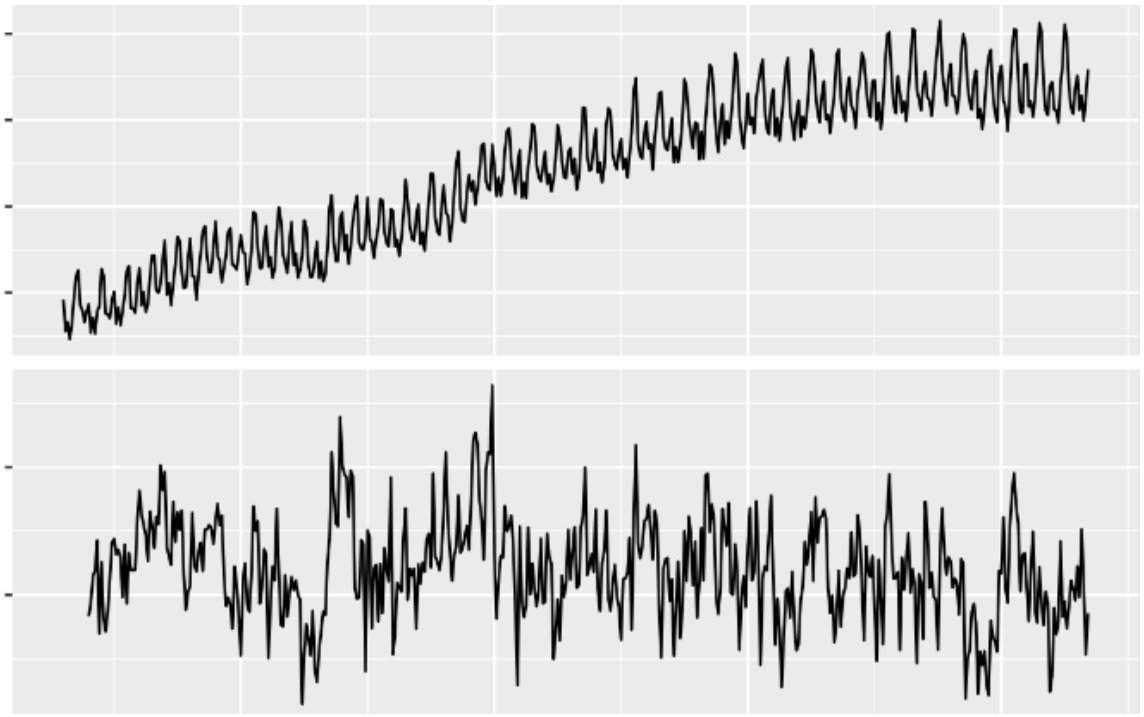


FIGURE 1.10 : Séries non stationnaires (en haut) et stationnaires (en bas) [41]

1.6.2.2 Modèles structurés

- ***Exponential Smoothing (ES).*** Ou lissage exponentiel en français. Ce modèle attribue des poids décroissants de manière exponentielle aux observations passées [19]. Il suppose que les données plus récentes affectent les données futures de manière plus significative que les données plus anciennes. Le Lissage Exponentiel Simple est représenté par :

$$\hat{y}_{t+1} = \alpha \cdot y_t + (1-\alpha) \cdot y_{t-1} + (1-\alpha)^2 \cdot y_{t-2} + (1-\alpha)^3 \cdot y_{t-3} + \dots \tag{1.1}$$

où $\hat{y}_{t+1}$ est la valeur prédite au temps $t+1$, y_t est la valeur réelle au temps t et α est le paramètre de lissage qui représente le coefficient à estimer. Ce modèle représente la formule de base au sein de la famille des modèles structurés ETS. L'acronyme "ETS" provient de "Error, Trend, and Seasonality", car d'autres variations au sein de cette famille de modèles prennent en compte les erreurs, les tendances et la saisonnalité lors de la prédiction des valeurs futures.

- ***Autorégressif (AR).*** Le modèle autorégressif prédit la prochaine valeur en fonction d'une combinaison linéaire de ses p valeurs précédentes [19]. L'$AR(p)$ est représenté par :

$$\hat{y}_{t+1} = \mu + \sum_{i=0}^{p} \phi_i \cdot y_{t-i} \tag{1.2}$$

où $\hat{y}_{t+1}$ est la valeur prédite au temps $t+1$, μ est la moyenne de la série chronologique, ϕ_i sont les coefficients autorégressifs et y_{t-i} sont les observations précédentes. L'ordre du modèle p et les coefficients autorégressifs ϕ_i sont les paramètres à estimer.

- ***Moving Average (MA).*** Ou moyenne mobile en français. Le modèle de Moving Average, généralement noté $MA(q)$, repose sur l'utilisation des erreurs passées pour prédire les valeurs

futures [19]. Contrairement à d'autres modèles, le $MA(q)$ n'utilise pas directement les valeurs précédentes de la série. Au lieu de cela, il calcule les erreurs résiduelles en comparant les prédictions antérieures avec les valeurs réelles observées. Ces erreurs, qui représentent les variations non expliquées par le modèle, sont ensuite utilisées pour ajuster les prédictions futures. Il prédit la prochaine valeur en fonction d'une combinaison linéaire des q erreurs résiduelles précédentes. Le $MA(q)$ est représenté par :

$$\hat{y}_{t+1} = \mu + \sum_{i=0}^{q} \theta_i \cdot \epsilon_{t-i} \tag{1.3}$$

où $\hat{y}_{t+1}$ est la valeur prédite au temps $t+1$, q est le nombre d'erreurs résiduelles passées considérées, ϵ_{t-i} est l'erreur résiduelle ($\epsilon_{t-i} = \hat{y}_{t-i} - y_{t-i}$) au temps $t-i$ et θ_i sont les coefficients de moyenne mobile. L'ordre du modèle q et les coefficients de moyenne mobile θ_i sont les paramètre à estimer.

- ***Auto-Regressive Integrated Moving Average (ARIMA).*** Le modèle ARIMA est l'un des modèles les plus largement utilisés pour la prédiction des séries chronologiques, grâce à ses performances exceptionnelles [29]. Il combine trois composantes essentielles : l'AutoRégressif (AR), l'Intégration (I) et la Moyenne Mobile (MA) [19]. Les modèles AR et MA sont conçus pour être appliqués sur des séries stationnaires, tandis que la composante I est utilisée pour rendre stationnaire une série si nécessaire en effectuant des différenciations.

  La notation ARIMA est définie par trois termes : p, d, et q, qui représentent respectivement l'ordre du modèle AutoRégressif (AR), le nombre de différenciations nécessaires pour rendre la série stationnaire, et l'ordre du modèle Moyenne Mobile (MA). Ces paramètres sont spécifiés dans le modèle pour mieux s'adapter aux caractéristiques de la série chronologique.

  De plus, le modèle ARIMA peut être étendu pour prendre en compte des séries présentant des saisons ou des cycles périodiques. Dans ce cas, il est appelé modèle SARIMA (Seasonal ARIMA), et des paramètres supplémentaires sont ajoutés pour tenir compte de la périodicité saisonnière. Cette extension permet de mieux capturer les motifs saisonniers et les variations cycliques, ce qui en fait un outil puissant pour modéliser des séries chronologiques plus complexes.

Le tableau 1.3 présente une synthèse des avantages et inconvénients des deux principales familles de modèles structurés : ARIMA et Lissage Exponentiel [41].

1.6.2.3 Modèles non-structurés

Dans les modèles non-structurés, une multitude d'approches reposant sur des algorithmes d'apprentissage automatique et des réseaux de neurones sont employées. Parmi celles-ci, un ensemble de modèles s'est particulièrement distingué en matière de prévision de séries chronologiques, affichant des performances exceptionnelles. Il s'agit de l'architecture de séquence RNN (Réseau de Neurones Récurrents) ainsi que de ses dérivés tels que les LSTM, les GRU, et même les ESN qui reposent également sur les RNN. Ces modèles ont connu un succès considérable dans la prédiction de séries chronologiques dans divers domaines. Dans la suite, nous allons présenter brièvement l'architecture fondamentale de chacun de ces modèles :

Modèle	Avantages	Inconvénients
Lissage Exponentiel	- Facilité d'apprentissage et d'application. - Prédictions précises en prenant en compte les différences entre les valeurs anticipées et les valeurs réelles. - Importance accordée aux données récentes pour des prédictions plus actuelles. - Peut négliger la variation aléatoire, créant des prédictions lisses.	- Difficulté à gérer les tendances avec variations cycliques ou saisonnières. - Moins adapté aux prédictions à long terme et aux séries saisonnières.
ARIMA	- Généralisation pratique pour les prédictions à court terme. - Capable de modéliser les séries non stationnaires avec tendances ou cycles. - Peut modéliser les tendances avec variations saisonnières.	- Peut être difficile de prédire les points de retournement précisément (les moments précis où une série change de direction). - Détermination subjective de l'ordre du modèle (p, d, q). - Moins adapté aux prédictions à long terme. - Moins pratique pour les grandes séries (Lorsque la série est grande, la détermination de ces ordres peut devenir plus complexe).

TABLE 1.3 : Avantages et Inconvénients des deux grandes familles de modèles structurés : ARIMA et Lissage Exponentiel.

- ***RNN (Réseau de Neurones Récurrents).*** Un RNN est un modèle qui fonctionne comme une boucle de rétroaction, en utilisant les informations précédentes pour traiter de nouvelles données dans un ordre séquentiel [28]. En d'autres termes, les sorties précédentes (ou l'état précédent) sont utilisées comme entrées pour le prochain calcul, créant ainsi une progression séquentielle dans le temps. Cependant, il peut avoir du mal à saisir les dépendances à long terme dans les séries chronologiques ou des séquences en générale, c'est le problème de la disparition du gradient. Pour illustrer ce modèle, on imagine de lire une histoire mot par mot et essayer de deviner la suite en fonction de ce qu'on a lu précédemment. On peut bien deviner la suite si ce que l'on a lu est court. Néanmoins, cela peut devenir difficile si l'histoire est très longue.

  La Figure 1.11 illustre l'architecture d'un RNN, démontrant clairement comment l'état à l'instant $t-2$ (noté $h(t-2)$) est employé pour déterminer l'état à l'instant $t-1$. Ce dernier est ensuite utilisé pour calculer l'état à l'instant t et ainsi de suite. Ainsi, $h(t)$, l'état à t, est une fonction de l'état à $t-1$ et de la valeur à de la série à t : $h(t) = f(h(t-1), x(t))$. Ce mécanisme crée une séquence qui relie les états successifs entre eux, établissant une cohérence temporelle dans la modélisation des données.

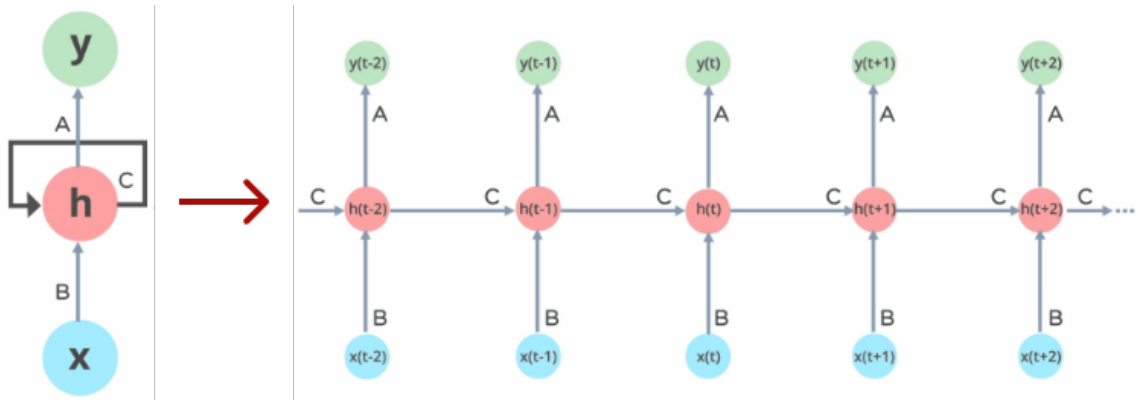


FIGURE 1.11 : Architecture du modèle RNN : pliée (à gauche) et dépliée (à droite) [87]

Cette architecture a permis aux RNN de surmonter les limitations d'autres architectures de réseaux de neurones. Contrairement à ces dernières, les RNN sont capables de traiter les données séquentielles, de prendre en compte non seulement l'entrée actuelle mais également les entrées précédentes, et de mémoriser les informations des entrées antérieures.

- ***LSTM (Long Short-Term Memory).*** Les LSTM sont une variante des RNN qui résolvent le problème de la disparition du gradient en utilisant des portes de mémoire [64]. Ces portes permettent au modèle de choisir quand oublier ou conserver des informations. Les LSTM sont capables de capturer des dépendances à long terme, ce qui en fait un choix populaire pour la prédiction de séries chronologiques. Prenons plutôt l'exemple d'un lecteur de livre qui prend des notes pour se rappeler les détails importants. Lors de la lecture, ce lecteur décide ce qu'il doit noter et ce qu'il doit oublier, ce qui l'aide à comprendre l'histoire même si celle-ci est longue et complexe.

- ***GRU (Gated Recurrent Unit).*** Les GRU sont une autre variante des RNN qui simplifie l'architecture des LSTM tout en conservant une performance similaire. Ils intègrent des mécanismes de portes pour gérer les informations à court terme et à long terme, mais avec une structure plus légère [64].

  Ils utilisent deux types principaux de portes : la "porte de réinitialisation" (reset gate) et la "porte d'update" (update gate).

  La porte de réinitialisation décide quelles parties des informations passées doivent être oubliées, en conservant ce qui est pertinent pour la prédiction. La porte d'update détermine comment combiner les informations passées avec les nouvelles pour la prédiction.

  La Figure 1.12 illustre la différence entre une cellule LSTM et une cellule GRU. Les GRU utilisent moins de portes que les LSTM. Ils utilisent 3 portes : (1) Une porte d'oubli (Forget Gate) qui permet de décider quelles informations doivent être oubliées de la cellule de mémoire (cell state), cela permet au modèle de filtrer les informations passés moins importantes ou obsolètes, tout en gardant en mémoire celles qui sont pertinentes. (2) Une Porte d'entrée (Input Gate) qui détermine quelles nouvelles informations doivent être stocké dans la cellule de mémoire. Et (3) une porte de sortie (Output Gate) qui contrôle la sortie des informations stockées dans la cellule de mémoire. Elle décide quelles parties de la mémoire doivent être utilisées pour calculer la sortie actuelle.

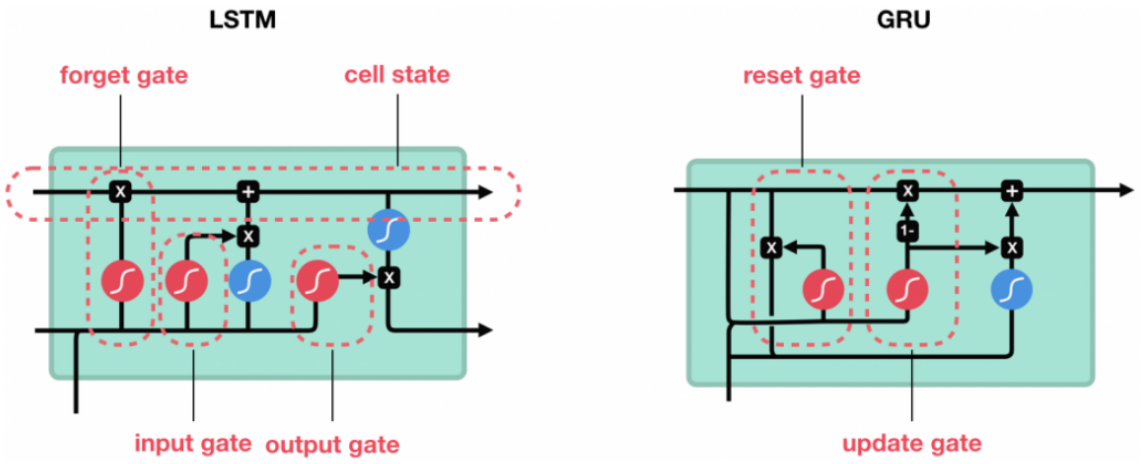


FIGURE 1.12 : Architectures de cellules LSTM et GRU [83]

Moins de portes signifie généralement moins de paramètres à entraîner, rendant les GRU plus simples et rapides que les LSTM. Cependant, la simplicité n'est pas le seul facteur. Les modèles plus complexes peuvent mieux capturer les dépendances temporelles complexes, ce qui peut être crucial pour certaines tâches de prédiction.

- ***ESN (Echo State Network).*** L'ESN est un type spécial des RNN qui est basé sur le concept d'apprentissage par réservoir (Reservoir Learning). Comparé aux RNN, LSTM et GRU, il nécessite beaucoup moins de temps pour l'entraînement. Sa simplicité est son principal atout, avec une structure constituée d'une couche d'entrée, d'une couche réservoir et d'une couche de sortie [16].

  Le "Reservoir Learning" est une approche d'apprentissage qui repose sur un réservoir aléatoire pour capturer et traiter les informations des données, principalement utilisée dans les ESN. Ce réservoir est composé de neurones interconnectés aléatoirement, appelés "neurones réservoir". Ils forment un sous-système complexe qui capture les motifs et les dépendances temporelles dans les données séquentielles, comme les séries chronologiques.

  Dans les ESN, le réservoir effectue une transformation initiale des données d'entrée à l'aide de sa structure aléatoire. Les poids des connexions entre le réservoir et la couche de sortie sont ajustés lors de l'entraînement pour exploiter les représentations internes du réservoir. Cette approche cible à capturer les dépendances temporelles et à améliorer les prédictions.

  L'architecture des ESN comprend trois couches principales : la couche d'entrée, la couche réservoir et la couche de sortie. L'idée clé est d'utiliser le réservoir pour transformer les données d'entrée en une forme plus adaptée à la capture des motifs temporels complexes, en exploitant les interactions aléatoires entre les neurones du réservoir.

  Cependant, l'ESN peut être sensible aux données sur lesquelles il est appliqué. Sa performance peut varier en fonction de la qualité et de la nature des données. Des données bruitées, mal préparées ou manquant de modèles clairs peuvent entraîner une performance variable de l'ESN [82].

  La Figure 1.13 présente l'architecture du modèle ESN, composée de trois principales couches : la couche d'entrée, la couche réservoir et la couche de sortie.

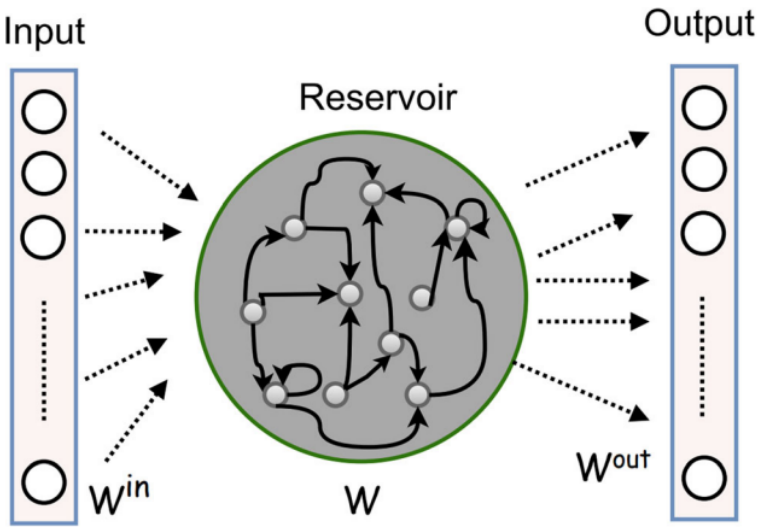


FIGURE 1.13 : Architectures du modèle ESN [20]

Le tableau 1.4 présente une comparaison entre les modèles RNN, LSTM, GRU et ESN en fonction de plusieurs critères [66].

Indicateur	RNN	LSTM	GRU	ESN
Structure	Simple	Plus complexe	Plus simple que LSTM	Simple
Entraînement	Peut être difficile	Peut être plus difficile	Plus facile que LSTM	Plus rapide
Performance	Bonne pour les tâches simples	Bonne pour les tâches complexes	Peut être intermédiaire	Variable
État caché	Unique	Multiple (cellule de mémoire)	Unique	Unique
Portes	Aucune	Entrée, sortie, oubli	Mise à jour, réinitialisation	Aucune
Capacité à conserver les dépendances à long terme	Limitée	Forte	Intermédiaire	Variable

TABLE 1.4 : Comparaison des modèles RNN, LSTM, GRU et ESN.

1.6.2.4 Comparaison entre les modèles structurés et non-structurés

Après avoir étudié les modèles les plus couramment utilisés de chaque catégorie, nous procédons à une comparaison dans le tableau 1.5 [8, 48] [67]. Cette comparaison met en évidence divers aspects des deux types de modèles.

Un point clé à noter est la capacité des modèles non-structurés, notamment les RNN et leurs dérivés, à effectuer des prédictions en ligne en maintenant leur état interne qui s'adapte et se met à jour à

mesure que de nouvelles données arrivent. Cela leur permet d'évaluer les informations entrantes en temps réel et d'ajuster leurs prédictions en conséquence. En revanche, les modèles structurés conservent les mêmes paramètres ajustés lors de l'entraînement et nécessitent donc une réadaptation ou un ré-entraînement du modèle pour inclure les nouvelles informations issues des nouvelles données arrivées [5].

L'analyse de cette comparaison révèle que chaque type de modèles présente ses propres avantages et inconvénients en termes d'interprétabilité, de performance, de facilité d'entraînement, de quantité de données requises, etc. Il est donc crucial de choisir la méthode en fonction des caractéristiques spécifiques de chaque projet et de ses priorités. Les modèles non-structurés tendent à offrir une précision de prédiction supérieure, mais ils peuvent nécessiter des volumes de données d'entraînement plus importants et être moins interprétables que les modèles structurés. Cependant, avec les progrès continus de l'apprentissage automatique, ces méthodes deviennent de plus en plus fiables et interprétables.

En définitive, le choix entre les deux méthodes dépend des objectifs et contraintes particuliers de chaque projet ainsi que des ressources disponibles.

Critères de Comparaison	**Modèles Structurés**	**Modèles Non-Structurés**
Flexibilité	Moins flexible	Plus flexible
Complexité des Motifs	Peut être limité pour les motifs complexes	Peut capturer des motifs complexes
Dépendances Temporelles à Long Terme	Peut avoir du mal à capturer des dépendances à très long terme	Capable de capturer des dépendances à long terme
Complexité de Mise en œuvre	Relativement plus simple	Peut être complexe à mettre en œuvre
Exigences de Données	Peut nécessiter des données bien structurées	Peut gérer des données moins structurées
Performances	Convient pour des séries avec des motifs clairs	Peut convenir à une variété de séries avec des structures différentes
Taille des Données Nécessaire	Relativement plus petite	Peut nécessiter une taille de données plus importants pour l'entraînement
Complexité en Termes de Temps	Plus rapide à mettre en œuvre	Peut nécessiter plus de temps de calcul pour l'entraînement
Interprétabilité	Plus interprétable	Moins interprétable
Possibilité de Mise à Jour	re-entraînement nécessaire	mise à jour en continu possible

TABLE 1.5 : Comparaison entre modèles structurés et modèles non-structurés

1.7 Conclusion

Ce chapitre a fourni une base solide pour appréhender les fondements essentiels qui encadrent notre projet. En explorant les véhicules autonomes, leurs défis, ainsi que les métriques cruciales de la QoS, nous avons saisi l'ampleur de l'environnement dans lequel se situe notre étude. La définition du concept de l'Edge Computing nous a permis de comprendre comment répondre aux exigences de haute QoS dans le contexte des véhicules autonomes. De plus, la mise en lumière des séries chronologiques comme outil vital pour la prédiction des paramètres de QoS a ouvert la voie à notre prochain chapitre. Ce dernier réalise une revue de littérature sur la modélisation et la prédiction des séries chronologiques de QoS, ce qui renforcera davantage notre compréhension et notre capacité à faire face aux défis de notre projet.

Chapitre 2

Modélisation et prédiction des séries chronologiques de QoS

2.1 Introduction

La prédiction et l'estimation des futures valeurs de la QoS (Qualité de Service) des nœuds edge constituent le premier et principal point qui permet d'anticiper les futures dégradations.

La QoS est impactée par divers facteurs hautement dynamiques au fil du temps, tels que la latence, la mobilité, la congestion du réseau, la charge CPU et RAM, entre autres [57]. Les séries chronologiques permettent de saisir ces variations temporelles en se basant sur les valeurs historiques et de prédire les valeurs futures. Par exemple, la variation de la latence peut suivre une tendance ascendante progressive sur une période de temps, ce qui signifierait que la latence augmente de manière constante au fil du temps. Un modèle de prédiction basé sur les séries chronologiques peut capturer cette tendance. Le modèle peut également prendre en compte des facteurs contextuels tels que la congestion du réseau ou la charge du serveur pendant cette période de latence accrue. Ainsi, en détectant et en intégrant ces différentes informations, le modèle de prédiction peut fournir des prédictions plus précises pour la latence future.

La prédiction des valeurs de la QoS est un problème de recherche largement étudié, avec de nombreuses approches disponibles pour aborder cette problématique. Dans ce chapitre, nous nous focalisons sur une revue de la littérature consacrée à l'application des séries chronologiques à la prédiction de la QoS. Nous examinons les différentes approches et méthodes utilisées dans la littérature pour l'analyse et le traitement de ces données.

Nous diviserons notre analyse en deux phases : la modélisation et la prédiction. La modélisation implique la création d'un modèle à partir de données historiques de QoS, tandis que la prédiction utilise ce modèle pour générer des prédictions [51]. Dans la phase de modélisation, notre concentration se porte sur trois volets principaux : (1) les types de données historiques de QoS (univariées, multivariées), (2) les méthodes et modèles utilisés, ainsi que (3) la préparation et le prétraitement des données. Dans la deuxième phase, nous focalisons sur deux aspects : (1) l'horizon de prédiction (1 pas en avant ou N pas en avant), (2) les mesures d'évaluation utilisées ainsi que la consommation de temps des modèles, le cas échéant.

2.2 Informations générales sur les travaux examinés

Le Tableau 2.1 présente les travaux de recherche examinés, fournissant une vue d'ensemble de leur année de publication, du contexte d'application et des attributs QoS abordés.

Au cours de notre recherche, nous avons passé en revue un total de 14 articles. En ce qui concerne le contexte d'application, nous avons donné la priorité aux travaux portant sur la prédiction des attributs QoS dans des situations similaires à celle de notre étude, c'est-à-dire les véhicules autonomes dans un environnement d'edge computing. La deuxième priorité était de rechercher des travaux situés dans le contexte plus large de l'edge computing. Nous avons ensuite examiné les travaux traitant de la prédiction des attributs QoS dans le domaine du cloud computing. En dernier recours, nous avons considéré le contexte plus général du service computing.

Chaque travail se démarque par les attributs QoS spécifiques qu'il cible, comme indiqué dans le tableau. Le temps de réponse ressort comme l'attribut le plus largement pris en compte et expérimenté. Il joue un rôle clé dans l'évaluation des performances d'un service, ayant un impact direct sur l'expérience des utilisateurs. Une réponse rapide améliore l'expérience, tandis qu'un temps de réponse lent peut causer de la frustration et réduire la satisfaction. Par conséquent, la mesure et l'amélioration du temps de réponse restent une priorité majeure pour les fournisseurs de services qui visent à garantir des performances optimales et à préserver leur réputation.

La figure 2.1 montre la distribution des travaux examinés selon le contexte d'application.

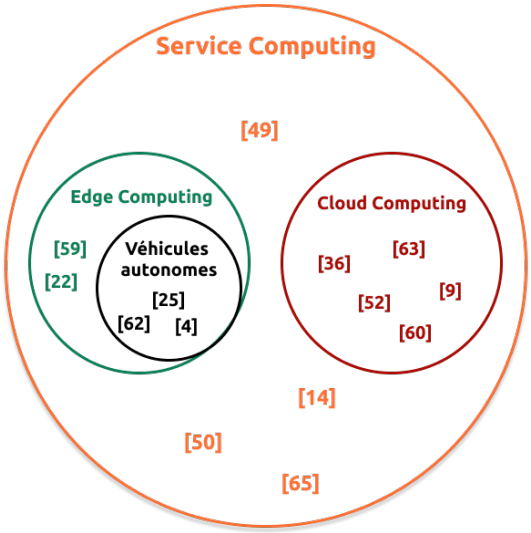


FIGURE 2.1 : Répartition des travaux examinés selon le contexte d'application

2.3 Modélisation

La phase de modélisation est un maillon essentiel pour anticiper l'avenir et comprendre les données passées. Elle permet de créer des modèles solides à partir de données historiques. Dans cette section, nous cherchons à explorer en détail cette phase en mettant en lumière les types de données,

Auteurs	Année de publication	Contexte d'application	Attributs QoS cibles
Xu et al. [62]	2023	Véhicules Autonomes en Edge Computing	Délai et Taux de Perte de Paquets (PLR)
Zhang et al. [65]	2022	Service Computing	Temps de réponse, Débit, Fiabilité et Disponibilité
White et al. [59]	2022	Edge Computing	Temps de réponse
Kousaridas et al. [25]	2021	Véhicules Autonomes en Edge Computing	Débit ascendant (UL throughput)
Barmpounakis et al. [4]	2021	Véhicules Autonomes en Edge Computing	Débit ascendant (UL throughput)
Jin et al. [22]	2021	Edge Computing	Temps de réponse et Débit
Syu et al. [52]	2019	Cloud Computing	Temps de réponse
Sun et al. [49]	2019	Service Computing	Temps de réponse et Débit
Ding et al. [9]	2018	Cloud Computing	Temps de réponse et Débit
White et al. [60]	2018	Cloud Computing	Temps de réponse
Syu et al. [50]	2017	Service Computing	Temps de réponse
Ye et al. [63]	2016	Cloud Computing	Temps de réponse et Débit
Fanjiang et al. [14]	2016	Service Computing	Temps de réponse
Nourikhah et al. [36]	2015	Cloud Computing	Temps de réponse

TABLE 2.1 : Informations générales sur les travaux examinés

les méthodes de modélisation et les étapes de développement pour former un modèle en vue de la prédiction dans l'ensemble des travaux examinés.

2.3.1 Types de données historiques de QoS

Comme illustré dans le Tableau 2.2, la plupart des travaux examinées se sont focalisées sur les séries chronologiques univariées. Dans cette approche, le modèle est construit en utilisant uniquement les données passées de l'attribut QoS cible. Cette préférence découle principalement de l'idée que prédire des séries univariées est la méthode la plus basique. De plus, de nombreuses méthodes structurées populaires, comme les modèles ARIMA, se basent sur des données univariées. Il est important de noter que la plupart de ces méthodes structurées ne s'appliquent pas aux données multivariées, renforçant ainsi la préférence pour les approches univariées. Certains travaux [4], notamment ceux portant sur les véhicules autonomes, considèrent l'utilisation de variables exogènes conjointement avec les données historiques de l'attribut cible, telles que la vitesse du véhicule et ses coordonnées, car ces paramètres ont un impact direct sur la QoS et peuvent améliorer la précision des prédictions pour l'attribut cible.

D'autres travaux, comme [62, 49], visent à prédire plusieurs attributs de QoS, mais ils adoptent néanmoins une approche univariée. Ils prédisent chaque attribut ou variable individuellement, ce

choix pouvant découler non seulement de la complexité accrue lors de la prédiction simultanée de plusieurs attributs, mais aussi de la possibilité d'appliquer d'autres modèles structurés. Cela est d'autant plus pertinent car ces modèles ne sont généralement pas adaptés à la gestion des données multivariées.

Quelques travaux se sont fondés sur des données multivariées [62, 49, 63]. Les auteurs considèrent et partent du postulat que les séries historiques distinctes des divers attributs ciblés sont bénéfiques pour les prédictions. Cette supposition repose sur l'existence de corrélations positives entre ces différentes attributs pour un même service. Les informations provenant d'autres séries chronologiques de QoS, présentant une corrélation positive, ont le potentiel d'améliorer la précision des prédictions.

Enfin, il est intéressant de se demander si davantage d'informations historiques conduisent à des prédictions plus précises. Plus spécifiquement, il s'agit de savoir si l'utilisation de l'historique de QoS basée sur une approche multivariée est plus efficace que les données d'une approche univariée pour prédire les valeurs futures de QoS. Certaines des études examinées apportent des réponses empiriques à cette question. Par exemple, les auteurs de l'étude [63] ont modifié les modèles ARIMA et ES pour prendre en compte des données multivariées. Leurs conclusions indiquent que cette approche modifiée est plus précise que les modèles ARIMA et ES univariés d'origine. Cependant, cette amélioration potentielle en termes de précision doit être balancée avec une complexité accrue et une consommation de temps plus importante, en raison de la gestion plus élaborée des données multivariées.

2.3.2 Méthodes et modèles utilisés

Les modèles et méthodes utilisés peuvent être regroupés en deux catégories : (1) les modèles proposés et (2) les modèles comparatifs. Les modèles comparatifs servent de références pour évaluer les performances ou comme bases pour les approches proposées. Ces modèles incluent généralement des solutions de référence.

Les solutions de référence englobent des méthodes naïves telles que la persistance (qui prédit la prochaine valeur comme la dernière valeur observée, soit $\hat{y}_{t+1} = y_t$), la moyenne (qui prédit la prochaine valeur comme la moyenne des valeurs passées, soit $\hat{y}_{t+1} = \frac{1}{N}\sum_{i=1}^{N} y_i$), etc. Les solutions de référence sont principalement employées pour la comparaison avec les approches proposées. Ces dernières doivent surpasser les solutions de référence en termes de précision de prédiction, faute de quoi elles perdent leur signification et leur utilité [41]. Toutefois, en termes de temps, les solutions de référence sont généralement plus rapides en raison de leur simplicité.

2.3.2.1 Modèles proposés

Une accord dans le domaine des séries chronologiques est qu'il n'existe pas d'approche ou un modèle universelle qui puisse convenir et prédire correctement différents types d'instances de séries chronologiques, car différentes approches ont leur propre spécialité et propriétés statistiques. Ainsi, en pratique, une étape initiale commune dans une tâche de modélisation et de prédiction consiste à analyser les données ciblées afin de comprendre leurs propriétés statistiques pour sélectionner l'approche le plus approprié.

Les méthodes structurées de séries chronologiques qui ont été considérées comprennent la famille ARIMA [25, 9, 63, 49], le lissage exponentiel (ES) [63] et ARFIMA [36]. Parmi ces méthodes, les modèles ARIMA et les méthodes ES sont considérés comme la norme standard dans les séries chronologiques et les solutions les plus largement utilisées dans de nombreux domaines d'applica-

Auteurs	Type des données	Modèles proposés	Modèles comparatifs
Xu et al. [62]	Univariées	Informer	CNN et Programmation génétique
Zhang et al. [65]	Multivariées	RNN	ARIMA et ANN
White et al. [59]	Univariées	ESN	Naïve(persistance), ARIMA, ES, GRU et LSTM
Kousaridas et al. [25]	Univariées	ARIMA	-
Barmpounakis et al. [4]	Univariées	LSTM	-
Jin et al. [22]	Univariées	basé sur LSTM	LSTM
Syu et al. [52]	Univariées	-	ARIMA, ES et ANN
Sun et al. [49]	Univariées	ARIMA	ESN, RNN et ANN
Ding et al. [9]	Multivariées	ARIMA	Méthode Kalman
White et al. [60]	Univariées	LSTM	Naïve(persistance), ARIMA et GRU
Syu et al. [50]	Univariées	-	ARIMA, ES, ANN, Garch et Programmation génétique
Ye et al. [63]	Multivariées	ARIMA et ES modifiés	ARIMA et ES (originaux) et VAR
Fanjiang et al. [14]	Univariées	Programmation génétique	Naïve, ARIMA, ES et ANN
Nourikhah et al. [36]	Univariées	ARFIMA (variation d'ARIMA)	Naïve(moyenne) et ARIMA

TABLE 2.2 : Approches de modélisation dans les travaux examinés

tion (comme le montre le Tableau 2.2, les modèles ARIMA sont la méthode la plus courante dans les études examinées, soit comme référence pour la comparaison, soit comme base pour des améliorations).

Les méthodes structurées peuvent être catégorisées davantage selon plusieurs critères (propriétés statistiques), comprenant linéaire et non linéaire, mémoire courte et mémoire longue (modèles ARIMA sans mémoire et modèles ARFIMA avec mémoire longue) [63], et univariée et multivariée [63]. Il a été indépendamment examiné et confirmé que les méthodes statistiques à mémoire longue [63] et basées sur des séries multivariés [36] sont plus adaptées à la prédiction des séries chronologiques de QoS car de nombreux exemples du monde réel sont non linéaires, présentent une dépendance à long terme (mémoire longue) et contiennent des corrélations positives entre les différentes attributs de QoS d'un service (ce qui ne peut pas être traité par des méthodes univariées).

Les résultats de [36] et [63] indiquent que les modèles ARIMA univariés (originaux), largement considérés et utilisés, pourraient ne pas être une option idéale pour le problème de QoS, et avant d'effectuer des tâches de prédiction, il est essentiel d'examiner les séries chronologiques de QoS ciblées et traitées pour comprendre leurs propriétés statistiques afin de sélectionner la méthode la plus appropriée parmi un ensemble de candidats.

Un ensemble de techniques novatrices initialement proposées et développées dans le domaine

d'apprentissage automatique ont également été considérées et utilisées pour le problème de QoS, notamment Informes (modèles basés sur les transformateurs) [62], ESN [59], la programmation génétique [14], LSTM [4, 22, 60] et RNN [65].

Dans le contexte des véhicules autonomes, selon les résultats de [62, 4], les données de QoS dans de tels cas présentent des caractéristiques très complexes. Les méthodes traditionnelles structurées fournissent un cadre théorique pour la prédiction de la QoS, mais elles reposent souvent sur des hypothèses correspondantes et peuvent ne pas capturer les caractéristiques complexes des systèmes de communication réels des véhicules autonomes. En revanche, les méthodes d'apprentissage profond offrent une alternative prometteuse, car elles peuvent apprendre à partir de vastes données et fournir des résultats de prédiction plus précis. Cependant, les algorithmes d'apprentissage profond actuels présentent des limites dans la prédiction de la QoS qui pourraient ne pas convenir aux exigences des applications de véhicules autonomes nécessitant des algorithmes simples et efficaces. C'est pourquoi nous devrions également envisager des modèles ayant une complexité de calcul moindre.

De plus, en observant le Tableau 2.2, on constate que les RNN et leurs dérivés, notamment les LSTM, sont les techniques non-structurées les plus couramment utilisées pour ce problème, ce qui renforce l'importance de la mémoire longue dans la résolution des problèmes de QoS. Par ailleurs, plusieurs études empiriques rapportent que la programmation génétique offre une meilleure précision de prédiction [50, 52], surpassant les autres méthodes. Néanmoins, la programmation génétique est également la solution qui demande le plus de temps parmi toutes les approches examinées, même plus que les modèles basés sur les réseaux de neurones.

2.3.2.2 Modèles comparatifs

Les études empiriques basées sur la comparaison, telles que [50] et [52], comparent différents candidats pour observer leurs performances, sans proposer d'approche spécifique, mais se contentent de faire des comparaisons. En revanche, dans les études proposant des modèles, des solutions sont incluses pour être comparées à leurs approches proposées. D'autres études, comme [25] et [4], n'ont pas pris en compte de référence de base pour la comparaison expérimentale, ce qui rend la supériorité de leurs solutions proposées peu claire et non vérifiée.

Comme nous pouvons l'observer dans le tableau 2.2, les modèles ARIMA et ES sont utilisés dans la plupart des études consultées à des fins de comparaison. Cela indique qu'ARIMA et ES servent de bonnes références pour comparer les performances de différentes méthodes de prédiction de séries chronologiques. Cela explique également pourquoi la plupart des études choisissent des données univariées, car ces modèles de référence ne prend pas en charge les données multivariées.

D'autre part, le modèle RNN et ces dérivés : LSTM, GRU et ESN sont les modèles non-structurés les plus largement utilisés pour la comparaison, ce qui indique leur efficacité dans la prédiction de séries chronologiques.

Certains travaux [59, 60, 36, 14] ont également envisagé d'utiliser des modèles Naïves avec d'autres modèles à des fins de comparaison. Les modèles Naïve les plus largement utilisés sont la persistance et la moyenne, respectivement. Ces deux méthodes sont souvent utilisés comme points de référence pour évaluer la performance des modèles plus complexes.

2.3.3 Préparation et pré-traitement des données

L'objectif du prétraitement des données vise à améliorer les performances du processus d'entraînement et à augmenter la précision des modèles résultants. Le prétraitement des données commence

généralement par deux étapes, suivant une approche similaire à celle décrite dans [65, 4] à savoir le débruitage et la normalisation des échelles. La première étape élimine les bruits contenus dans les observations historiques de la QoS, tandis que la deuxième étape réajuste les valeurs de la QoS pour qu'elles se situent dans la plage entre 0 et 1.

Pour les modèles non structurés, une étape supplémentaire est nécessaire pour convertir les données de séries chronologiques en un format tabulaire [59], rendant ainsi le problème compatible avec une approche supervisée. Les modèles non structurés, tels que les réseaux de neurones, les RNN et leurs variantes, requièrent que les données soient organisées en vecteurs d'entrée (X) et vecteurs de sortie (y).

Dans le cas des séries chronologiques, cette transformation peut être réalisée en utilisant les observations des pas de temps précédents ($t-1, t-2, ..., t-n$) comme entrées et l'observation du pas de temps actuel (t) comme sortie. Ces séries chronologiques peuvent ensuite être concaténées pour former un ensemble de données prêt à être utilisé dans un processus d'apprentissage supervisé.

La Figure 2.2 illustre un exemple de cette transformation, montrant comment un ensemble de données de séries chronologiques est converti en un ensemble de données adapté à un problème d'apprentissage supervisé. Dans ce contexte, le "window size" est utilisé avec une valeur de 3, indiquant la taille de la fenêtre temporelle utilisée. Par ailleurs, étant donné que la dimension du vecteur de sortie y est 1, cela reflète la prédiction d'un pas en avant (1-step ahead).

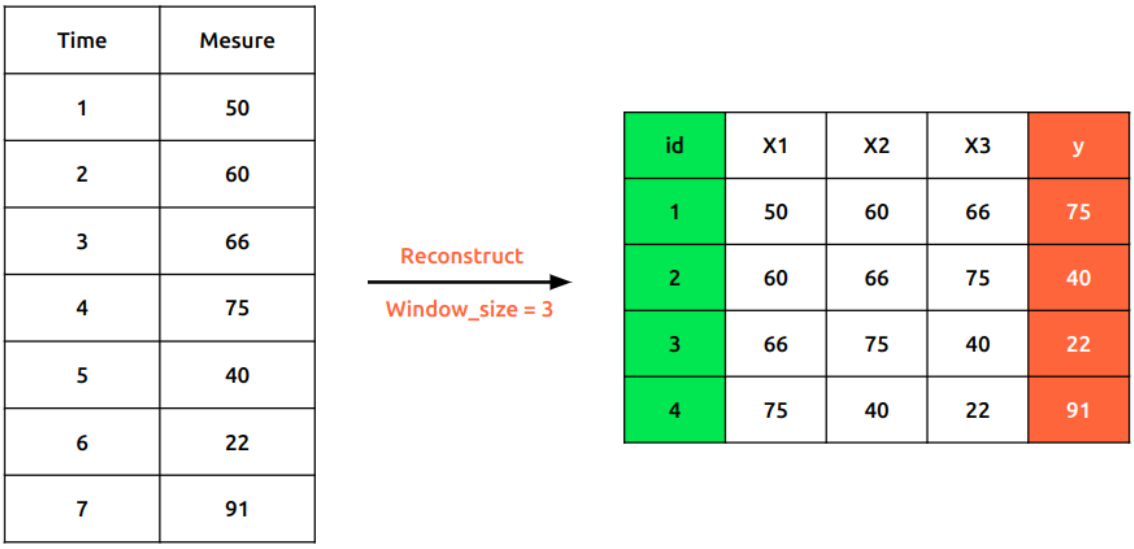


FIGURE 2.2 : Transformation des données de séries chronologiques vers des données tabulaires

De plus, le pré-traitement des données effectué dans l'étude comparative [60] a choisi de ne pas élaguer la série chronologique pour éliminer les valeurs aberrantes (outliers). Cela vise à évaluer la robustesse des approches de prédiction vis-à-vis des valeurs aberrantes. Nous estimons qu'il est très important de conserver les valeurs aberrantes dans l'ensemble de données. En effet, supprimer les valeurs aberrantes expérimentées dans son ensemble aurait un impact négatif sur la justesse et la crédibilité des résultats expérimentaux (il serait pas possible d'observer et de connaître la capacité de chaque méthode comparée à traiter les valeurs aberrantes de la QoS en pratique).

Certains des travaux examinés ont omis de détailler leurs approches en matière de préparation et de prétraitement des données [25, 62]. Cette lacune rend leurs propositions incomplètes et entrave

la reproductibilité de leurs résultats. Il est impératif de mettre en évidence les étapes préalables à l'expérimentation afin que les autres puissent comprendre les processus mis en œuvre.

La plupart des études examinées n'ont pas traité le problème des observations manquantes dans les séries chronologiques de QoS historiques. En pratique, cependant, un service peut ne pas être invoqué périodiquement, et ainsi, ses observations de QoS ne sont pas enregistrées complètement à chaque instant, entraînant des éléments manquants dans l'historique de la QoS.

2.4 Prédiction

Ayant achevé une analyse approfondie de la phase de modélisation qui consiste à mettre en place le modèle, nous franchissons désormais l'étape de la prédiction. Au cours de cette section, nous analysons des aspects spécifiques liés à la prédiction à l'aide du modèle construit. Ces aspects incluent l'horizon de prédiction, qu'il soit d'un pas en avant ou de N pas en avant, les critères d'évaluation choisis, ainsi que, le cas échéant, la prise en compte de la consommation temporelle des modèles. A l'issue de cette analyse, nous aurons une vision globale de la littérature existante dans la prédiction des séries chronologiques de QoS.

2.4.1 Horizon de prédiction

Dans les travaux examinés, deux types de prédictions existent : les prédictions à un pas en avant (1-step-ahead) et les prédictions à N pas en avant (N-step-ahead) (c'est-à-dire une longueur d'horizon de prédiction de 1 et N, respectivement). Dans le premier type, l'objectif est de produire la valeur de l'attribut de QoS à l'instant $t + 1$. En revanche, dans le cas d'une prédiction à N pas en avant, le Résultat de la prédiction inclut les valeurs à l'instants $t + 1, t + 2, ..., t + N$.

Comme indiqué dans le Tableau 2.3, la plupart des études actuelles supposent une prédiction à un pas en avant pour les séries chronologiques de QoS, dans laquelle seule la valeur future de l'attribut de QoS ciblé au prochain instant $(t + 1)$ est prédite et produite à chaque fois. Cependant, dans certains contextes d'application, une séquence de valeurs de QoS futures à long terme (à savoir, les valeurs de QoS prédites aux instants futurs $t + 1, t + 2, ..., t + N$) est demandée. Dans de tels contextes d'application, un modèle capable d'effectuer une prédiction à N pas en avant est nécessaire.

Lors de l'analyse des travaux dans le contexte des voitures autonomes [25, 62, 4], nous avons constaté que tous les travaux examinés ont privilégié les prédictions à N pas en avant (les 3 premières lignes dans le tableau 2.3). Cette tendance souligne l'importance cruciale de la prédiction à long terme dans ce contexte d'application critique. Cette approche permet d'accorder au système le temps nécessaire pour anticiper les éventuels changements, ce qui est d'une importance capitale étant donné la mobilité du véhicule et sa capacité à se déplacer à des vitesses élevées. L'orientation vers les prédictions à N pas en avant témoigne de la préoccupation pour la sécurité et la réactivité du système dans des conditions de fonctionnement complexes et dynamiques. En effet, dans le domaine des véhicules autonomes, la prise en compte d'un horizon de prédiction étendu s'avère essentielle pour garantir la sécurité des passagers et des autres usagers de la route, tout en assurant des performances optimales du véhicule dans des situations variées.

Contrairement aux prédictions à 1 pas en avant, les tâches de prédiction à N pas en avant sont plus complexes. Ces tâches impliquent de faire face à diverses complications supplémentaires, telles que l'accumulation d'erreurs, une précision réduite et une incertitude accrue.

Auteurs	Horizon de prédiction	Métrique d'évaluation	Consommation de temps
Xu et al. [62]	7 pas en avant	MAE et RMSE	-
Zhang et al. [65]	N pas en avant	MAE et RMSE	-
White et al. [59]	1 pas en avant	RMSE	Temps de génération du modèle et Temps de production des prédictions
Kousaridas et al. [25]	Jusqu'à 7 pas en avant	MAE et MAPE	-
Barmpounakis et al. [4]	7 pas en avant	RMSE et NMSE	-
Jin et al. [22]	1 pas en avant	RMSE	Temps de production des prédictions
Syu et al. [52]	1 pas en avant	MAE, RMSE et MAPE	Temps de génération du modèle
Sun et al. [49]	1 pas en avant	RMSE	-
Ding et al. [9]	1 pas en avant	MAE et RMSE	-
White et al. [60]	1 pas en avant	RMSE	-
Syu et al. [50]	1 pas en avant	MAE et MAPE	Temps de génération du modèle et Temps de production des prédictions
Ye et al. [63]	N pas en avant	RMSE	-
Fanjiang et al. [14]	1 pas en avant	MAE, MAPE et un test statistique	-
Nourikhah et al. [36]	1 et N pas en avant (de 1 jusqu'à 48)	MAE et MAPE	-

TABLE 2.3 : Approches de prédiction dans les travaux examinés

Dans la littérature, 5 stratégies ont été proposées pour aborder une tâche de prédiction à N pas en avant [53]. Ces stratégies sont résumées dans ce qui suit.

2.4.1.1 Stratégie Récursive

Il s'agit de la stratégie la plus couramment employée. Elle applique de manière répétée un modèle de prédiction à un pas en avant N fois pour générer N prédictions. Chaque fois, la valeur estimée précédemment est utilisée comme entrée pour la prédiction suivante. La Figure 2.3 illustre graphiquement la stratégie récursive, où f représente le modèle de prédiction, y est une valeur réelle et y' est la valeur prédite à chaque itération.

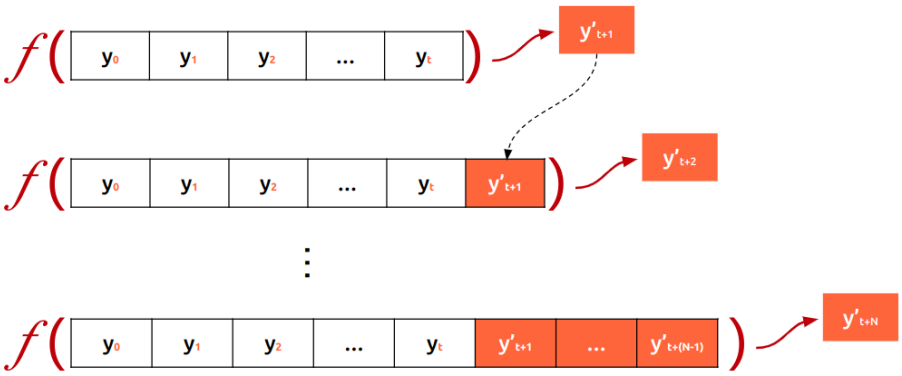


FIGURE 2.3 : Stratégie récursive de prédiction à N pas en avant

2.4.1.2 Stratégie Direct

Contrairement à la stratégie précédente qui repose sur un seul modèle, la stratégie Direct implique l'entraînement d'un ensemble de N modèles de prédiction. Chaque modèle retourne une prédiction pour la valeur i_{eme} ($i \in 1, ..., N$), comme illustré dans la Figure 2.4.

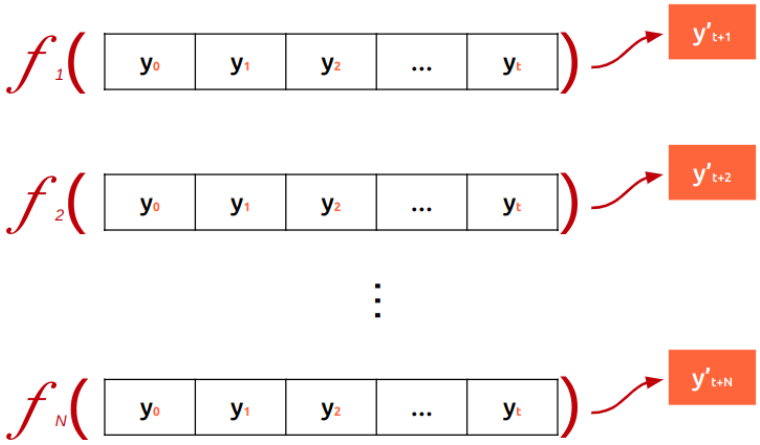


FIGURE 2.4 : Stratégie direct de prédiction à N pas en avant

2.4.1.3 Stratégie DirRec

Elle combine des les stratégies Direct et Récursive. Elle implique l'utilisation d'un modèle différent à chaque étape, tout en incorporant les prédictions des étapes précédentes dans l'ensemble d'entrée, comme illustré dans la Figure 2.5.

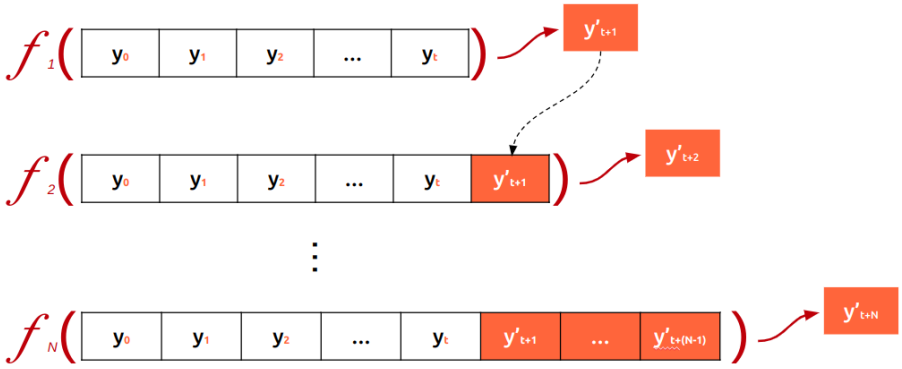


FIGURE 2.5 : Stratégie DirRec de prédiction à N pas en avant

2.4.1.4 Stratégie MIMO (Multi-Input Multi-Output)

Contrairement aux stratégies précédentes qui génèrent une seule valeur de prédiction, la stratégie MIMO produit un vecteur de valeurs futures en une seule étape. Toutefois, il est important de noter que cette stratégie nécessite des modèles capables de générer des vecteurs en sortie, tels que les modèles de réseaux de neurones. La Figure 2.6 illustre visuellement cette stratégie.



FIGURE 2.6 : Stratégie MIMO de prédiction à N pas en avant

2.4.1.5 Stratégie DIRMO

La stratégie DIRMO est une approche hybride qui combine les forces des stratégies Direct et MIMO. En décomposant la tâche de prédiction de N valeurs en blocs, chaque bloc est ensuite prédit de manière MIMO.

D'une autre manière, la tâche de prédiction à horizon N est décomposée en m tâches de prédiction à sortie multiple (m = N / s), chacune ayant une taille de sortie s ($s \in 1, , N$).

Lorsque $s = 1$, le nombre de tâches de prédiction m est égal à N, correspondant ainsi à la stratégie Direct. Lorsque $s = N$, le nombre de tâches de prédiction m est égal à 1, correspondant alors à la stratégie MIMO. La Figure 2.7 illustre visuellement cette stratégie dans le cas où la taille de sortie $s = 2$.

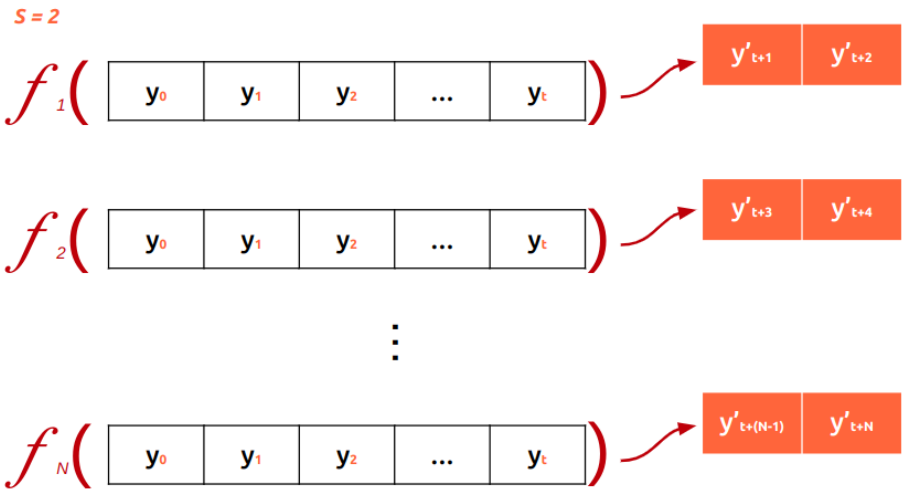


FIGURE 2.7 : Stratégie DIRMO de prédiction à N pas en avant

Le schéma présenté dans la Figure 2.8 représente la taxonomie de ces stratégies, distinguant les stratégies dites à sortie unique (single-output), qui produisent et prédisent une seule valeur à la fois, des stratégies à sortie multiple (multiple-output), qui ont la capacité de prédire un vecteur de valeurs en sortie, donc plusieurs valeurs en même temps.

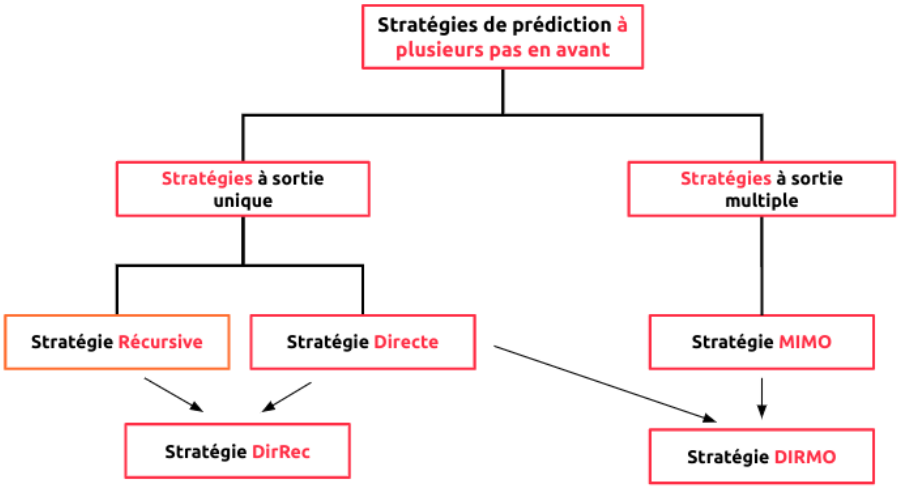


FIGURE 2.8 : Taxonomie des stratégies de prédiction à N pas en avant

Parmi les études examinées, 6 d'entre elles envisagent d'utiliser une prédiction à N pas en avant. Cependant, les travaux [25, 62, 4] n'ont pas mentionné la stratégie utilisée, tandis que les travaux [63, 36, 65] ont adopté une approche récursive. Afin d'analyser et d'étudier plus en détail les stratégies de prédiction à N pas adoptées dans la littérature, nous avons sélectionné un ensemble de 5 articles qui ont adapté la prédiction à N pas en avant pour la QoS. Cela représente l'ensemble de travaux supplémentaires que nous avons pu trouver (en plus de ceux mentionnés dans notre revue de la littérature initiale). Il est à noter que le nombre de travaux qui se penchent sur les prédictions à N pas (en avant) est limité. Le Tableau 2.4 résume ces travaux.

Auteurs	Année	Titre	Stratégie
Mistry et al. [32]	2018	Service Providers' Long-Term QoS Prediction Model	Récursive (single output strategy)
Haytamy et al. [18]	2020	A deep learning based framework for optimizing cloud consumer QoS-based service composition	MIMO (multiple output strategy) [4-steps ahead]
Rehman et al. [55]	2015	A User-side QoS forecasting and management of cloud services	Récursive (single output strategy) [8-steps ahead]
Fanjiang et al. [13]	2022	Time Series QoS Forecasting for Web Services Using Multi-Predictor-based Genetic Programming	Récursive, Direct and DirRec (DirRec atteint la plus haute précision)
Malak et al. [30]	2009	Web Service QoS Prediction Based on Multi Agents	Récursive (single output strategy)

TABLE 2.4 : Stratégies de prédiction à N pas en avant de la QoS dans la littérature

Comme illustré dans le tableau 2.4, la plupart des travaux de prédiction de la QoS s'appuient sur une stratégie récursive, qui est également l'approche la plus couramment utilisée (en incluant les 3 travaux de notre revue de littérature initiale). En réalité, la stratégie récursive est la moins complexe parmi les 5 stratégies. Une analyse comparative du temps d'entraînement des modèles a été réalisée dans [53] comme suit :

- Soit T_{SO} et T_{MO} le temps de calcul nécessaire pour former respectivement un modèle à sortie unique et un modèle à sortie multiple.
- Supposons $T_{MO} = T_{SO} + \delta$, ce qui est une hypothèse raisonnable car former un modèle avec une sortie sous forme de vecteur nécessite plus de temps que l'apprentissage d'un modèle avec une sortie unique.
- Ainsi, nous pouvons classer les stratégies de prédiction en fonction de leur temps de calcul pour la formation d'un modèle comme illustré dans la Figure 2.9 (Le temps nécessaire pour apprendre un modèle de la stratégie DirRec est égal à $(T_{SO}+u)$ car la taille de l'entrée augmente à chaque étape).



FIGURE 2.9 : Comparaison des temps d'entraînement des modèles des différentes stratégies

En ce qui concerne les performances de prédiction de ces stratégies, nous croyons que cela dépend fortement de l'ensemble de données. Par exemple, dans l'étude menée par [53], les différentes stratégies ont été comparées sur des données provenant d'un domaine différent de la QoS. Les conclusions tirées de cette comparaison sont les suivantes :

- Les stratégies à Sortie Multiple (MIMO et DIRMO) surpassent généralement les stratégies à Sortie Unique (Recursive, Direct, DirRec). MIMO et DIRMO montrant des performances similaires.

- La sélection du paramètre "s" dans la stratégie DIRMO joue un rôle crucial dans ses performances, et des améliorations supplémentaires dans les méthodes de sélection pourraient renforcer son potentiel.

- En ce qui concerne les stratégies à Sortie Unique, la stratégie Récursive présente systématiquement de meilleures performances que la stratégie Directe, tandis que DirRec est la stratégie globalement la moins efficace.

Une analyse comparative menée par [44] a confronté plusieurs modèles en utilisant les 5 stratégies avec des données de QoS, pour des horizons de prédiction de 4, 8 et 12. Les résultats ont montré que pour une prédiction à 4 pas en avant, la stratégie Direct a donné les meilleurs résultats tandis que pour des prédictions à 8 et 12 pas en avant, la stratégie MIMO est la meilleure.

Dans [13], une comparaison a été effectuée entre les 3 stratégies à sortie unique (Récursive, Direct et DirRec), où il a été constaté que DirRec offrait de meilleures précisions que les autres. L'article [13], qui a proposé une stratégie MIMO, l'a comparée avec une stratégie récursive et a conclu que la stratégie MIMO donne de meilleurs résultats. Cependant, cette comparaison ne peut pas être entièrement généralisée, car différents modèles ont été utilisés : ARIMA pour la stratégie récursive et LSTM pour la stratégie MIMO.

Il est donc évident que la performance des stratégies varie en fonction des données et des contextes. Par conséquent, le choix de la meilleure stratégie repose sur des tests approfondis et des évaluations sur les données spécifiques.

Le tableau 2.5 récapitule certains avantages et inconvénients des différentes stratégies.

Strategie	**Avantages**	**Inconvénients**
Récursive	Convient aux séries chronologiques sans bruit	Accumulation d'erreurs
Direct	Pas d'accumulation d'erreurs	Hypothèse d'indépendance conditionnelle (les valeurs futures prédites ne sont pas influencées par les prédictions antérieures)
DirRec	Équilibre entre Direct et Récursive	La taille de l'ensemble d'entrée augmente linéairement
MIMO	Pas d'hypothèse d'indépendance conditionnelle	Flexibilité réduite : même structure de modèle pour tous les horizons
DIRMO	Équilibre entre Direct et MIMO	Un paramètre additionnel à estimer

TABLE 2.5 : Avantages et inconvénients des différentes stratégies de prédiction à N pas en avant

À l'étape de prédiction, une question pertinente concerne la durée pendant laquelle un modèle peut être utilisé avant de devenir obsolète et moins précis, nécessitant d'être remplacé par un nouveau modèle formé avec des données actualisées en temps réel. Cette durée de vie du modèle dépend également du type de modèle utilisé. Par exemple, les modèles LSTM ont une mémoire à long terme et peuvent ajuster automatiquement leurs paramètres, ce qui pourrait prolonger leur durée de vie par rapport à des modèles tels qu'ARIMA, qui conservent les mêmes paramètres ajustés lors de la

première phase d'entraînement. Comme le montrent les expériences empiriques menées dans [52], en général, des prédictions à plus long terme entraînent une moindre précision si le même modèle est utilisé de manière répétée. Cependant, la plupart des études examinées n'ont pas pris en compte cette préoccupation.

2.4.2 Mesure de performance

Dans le domaine des études basées sur les séries chronologiques, l'évaluation des performances repose en premier lieu sur la précision des prédictions. Cependant, il est également essentiel de prendre en compte un autre facteur clé lors de cette évaluation, à savoir le coût, particulièrement en termes de temps. Cette considération devient d'autant plus cruciale dans des contextes tels que les véhicules autonomes, où chaque milliseconde compte.

En somme, la précision des prédictions et la consommation de temps sont deux aspects majeurs en matière de performances. Chacun de ces aspects a été abordé au moyen de diverses méthodes et mesures, comme indiqué dans le Tableau 2.3.

2.4.2.1 Précision

Pour calculer la précision d'un modèle, diverses mesures numériques ont été proposées et utilisées dans la recherche sur les séries chronologiques. Le calcul de ces mesures implique deux phases. Tout d'abord, il est nécessaire d'obtenir une séquence d'erreurs ponctuelles d'un modèle. Il existe plusieurs types d'erreurs ponctuelles différents, tels que les erreurs absolues ($e_i = |\hat{y}_i - y_i|$, où e_i est l'erreur au point i, $\hat{y}_i$ représente la valeur prédite et y_i représente la valeur réelle observée), les erreurs absolues relatives ($e_i = |\frac{y_i - \hat{y}_i}{y_i}|$) et les erreurs quadratiques ($e_i = (\hat{y}_i - y_i)^2$).

Une fois la séquence d'erreurs ponctuelles calculée, une méthode d'intégration est appliquée pour les agréger en une seule valeur, qui représente la précision globale du modèle. L'intégration la plus couramment utilisée est la moyenne des erreurs ponctuelles. Par exemple, le calcul de l'erreur absolue moyenne (Mean Absolute Error en anglais, MAE) se fait en prenant la moyenne de la séquence d'erreurs absolues (soit MAE = $\frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$). De même, l'erreur relative absolue moyenne (Mean Absolute Percentage Error en anglais, MAPE) est calculée en prenant la moyenne de la séquence d'erreurs absolues relatives (soit MAPE = $\frac{1}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$). L'erreur quadratique moyenne (Mean Squared Error en anglais, MSE) est calculée en prenant la moyenne de la séquence d'erreurs quadratiques (soit MSE = $\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$), tandis que l'erreur quadratique moyenne racine (Root Mean Squared Error en anglais, RMSE) est obtenue en prenant la racine carrée du MSE.

Pour comparer la précision de deux modèles concurrents, telles qu'une nouvelle approche proposée comparée à des modèles ARIMA ou un ensemble d'approches existantes comparées empiriquement deux à deux, plusieurs méthodes ont été utilisées dans les études examinées. Tout d'abord, la manière la plus simple est la comparaison directe des valeurs de précision mesurées des deux approches, et comme les mesures de précision examinées sont toutes basées sur des erreurs, des valeurs inférieures représentent une meilleure précision.

Comme indiqué dans le tableau 2.3, le RMSE (Root Mean Squared Error) émerge comme la mesure de précision la plus fréquemment utilisée, suivie du MAE (Mean Absolute Error). Ces deux métriques présentent l'avantage d'être exprimées dans les mêmes unités que les données d'origine, ce qui facilite leur interprétation. Par exemple, lorsqu'il s'agit de prédire la latence réseau et que l'on obtient un MAE de 10, cela signifie que, en moyenne, les prédictions diffèrent des valeurs réelles de 10 millisecondes, ce qui est immédiatement compréhensible.

De plus, le RMSE fournit des informations approfondies sur les performances d'un modèle. Son calcul est similaire à la formule de l'écart-type (standard deviation), une mesure statistique décrivant la dispersion des valeurs par rapport à leur moyenne. Le RMSE mesure à quel point les valeurs prédites diffèrent des valeurs réelles. En comparaison, l'écart-type quantifie à quel point une valeur réelle s'écarte de la moyenne. Dans l'idéal, un bon modèle devrait, en moyenne, fournir des prédictions plus précises que l'estimation naïve de la moyenne pour l'ensemble des prédictions. Par conséquent, le RMSE devrait être inférieur à l'écart-type [46].

Si le RMSE d'un modèle est supérieur à l'écart-type de la distribution des données, cela suggère que le modèle est moins performant qu'une simple prédiction basée sur la moyenne des valeurs. En d'autres termes, le modèle n'apporte pas d'amélioration significative par rapport à une approche naïve.

Ainsi, le choix du RMSE et du MAE comme métriques d'évaluation se justifie par leur interprétabilité cohérente avec les unités des données, tandis que le lien entre le RMSE et l'écart-type reflète leur rôle dans l'analyse des performances du modèle.

Parmi les études examinées, il y a un bon nombre de ces études se limitent à n'utiliser qu'une mesure de précision spécifique [59, 22, 63, 60, 49]. Cependant, chaque mesure de précision possède ses propres particularités et avantages.

L'idée qui émerge est que l'utilisation d'une seule mesure de précision peut ne pas suffire à fournir une vue complète et nuancée des performances d'un modèle de prédiction. En effet, différentes mesures de précision mettent en évidence des aspects variés de la performance du modèle. Par exemple, le RMSE met l'accent sur la pénalisation des erreurs importantes en raison du carré, ce qui le rend utile pour évaluer la capacité du modèle à prédire avec précision les valeurs exactes. À l'opposé, le MAE considère toutes les erreurs avec la même importance, ce qui le rend plus résistant aux valeurs aberrantes. Le MAPE permet d'évaluer la précision relative du modèle indépendamment de l'échelle des données. Enfin, bien que le MSE soit similaire au RMSE, il ne prend pas la racine carrée, ce qui peut aggraver l'impact des erreurs significatives.

Dans cette optique, il est recommandé d'adopter plusieurs mesures de précision lors de l'évaluation d'un modèle de prédiction. En utilisant un ensemble varié de mesures, on peut obtenir une évaluation plus approfondie et plus équilibrée de la performance du modèle. Cela permet de prendre en compte différents scénarios et aspects, ce qui offre une meilleure compréhension globale de la pertinence et de l'efficacité du modèle.

2.4.2.2 Consommation de temps

Parmi les articles examinés, deux types de consommation de temps ont été mesurés et pris en compte : 1) le temps de génération du modèle et 2) le temps de production de la prédiction. Le premier temps correspond à la durée qu'une approche met pour générer (former) un modèle, tandis que le second mesure le durée nécessaire à un modèle pour produire une valeur de prédiction.

Selon le Tableau 2.3, la majorité des études examinées n'ont pas décrit leurs informations de consommation de temps et se sont concentrées uniquement sur la précision des prédictions. Cependant, une évaluation complète des performances doit prendre en compte à la fois la précision et le coût simultanément. Par exemple, dans les études [14, 50, 52], il a été rapporté que la programmation génétique (GP) peut atteindre la plus haute précision de prédiction pour des séries chronologiques univariées à un pas en avant. Cependant, la GP est également l'approche la plus consommatrice en temps. En d'autres termes, même si l'utilisation de la GP peut aboutir à la plus haute précision, le

temps nécessaire pour générer un modèle est également beaucoup plus long que pour les autres approches. Ainsi, si l'on prend en compte à la fois la précision et la consommation de temps, comme rapporté dans [50], les modèles ARIMA pourraient constituer un compromis entre ces deux considérations de performance, car les modèles ARIMA peuvent atteindre une précision légèrement inférieure à celle de la GP, mais la consommation de temps des modèles ARIMA est nettement inférieure à celle de la GP.

En résumé, il est idéal (voire nécessaire) de décrire à la fois les informations de précision et de consommation de temps dans une étude pour une évaluation complète des performances.

2.5 Conclusion

En conclusion, ce chapitre a examiné de manière approfondie l'état actuel de la modélisation et de la prédiction des séries chronologiques de QoS. En analysant les phases de modélisation et de prédiction, nous avons identifié des points clés tels que les types de données historiques, les méthodes et les modèles employés, les considérations relatives à l'horizon de prédiction, à la précision des modèles et à la mesure de consommation de temps. Cette vue d'ensemble détaillée nous balaye le terrain afin d'aboutir à notre objectif, à savoir la conception de modèles de prédiction adaptés aux particularités des environnements aussi complexes et dynamiques que ceux des véhicules autonomes.

DEUXIÈME PARTIE

Contribution

Chapitre 3

Étude de cas

3.1 Introduction

Dans le présent chapitre, nous introduirons le cadre général du projet, y compris ses fondements et architecture déjà en place. Notre objectif principal est de procéder à une analyse approfondie de l'environnement existant, en mettant particulièrement l'accent sur deux aspects : les schémas de conception adoptés (déjà mis en place) et les principaux défis identifiés. Cette analyse préliminaire nous servira de fondement essentiel pour notre proposition ultérieure.

3.2 Cadre du projet

Ce projet s'inscrit dans le cadre d'une initiative de recherche plus vaste visant à intégrer les véhicules autonomes dans des environnements urbains intelligents, en tirant parti de la technologie de l'Edge Computing. Il est mené au sein de l'équipe SARA (Systèmes et Architectures pour les Réseaux Avancés) du LAAS-CNRS (Laboratoire d'Analyse et d'Architecture des Systèmes du Centre National de la Recherche Scientifique) situé à Toulouse, en France. Dans la section suivante, nous allons présenter l'équipe SARA ainsi que le laboratoire d'accueil, le LAAS-CNRS, avant de présenter en détail les fondements déjà établis pour ce projet de grande envergure.

3.2.1 Présentation de l'organisme d'accueil : LAAS-CNRS

Le Laboratoire d'Analyse et d'Architecture des Systèmes (LAAS) est une unité de recherche affiliée au CNRS, fondée en 1967. Son objectif principal est d'approfondir la compréhension des systèmes complexes tout en tenant compte de leurs applications potentielles. Les domaines de recherche se concentrent sur 7 axes majeurs, à savoir l'énergie, l'espace, l'industrie du futur, la mobilité et le transport, ainsi que la santé et l'environnement. Le laboratoire est organisé en 25 équipes réparties au sein de 6 départements : (1) département RISC (Réseaux, Informatique, Systèmes de Confiance), (2) département Robotique, (3) département Décision et optimisation, (4) département Hyperfréquences et optique, (5) département Micro nano bio technologies et (6) département Gestion de lénergie. La figure 3.1 présente l'organigramme de l'organisme d'accueil.

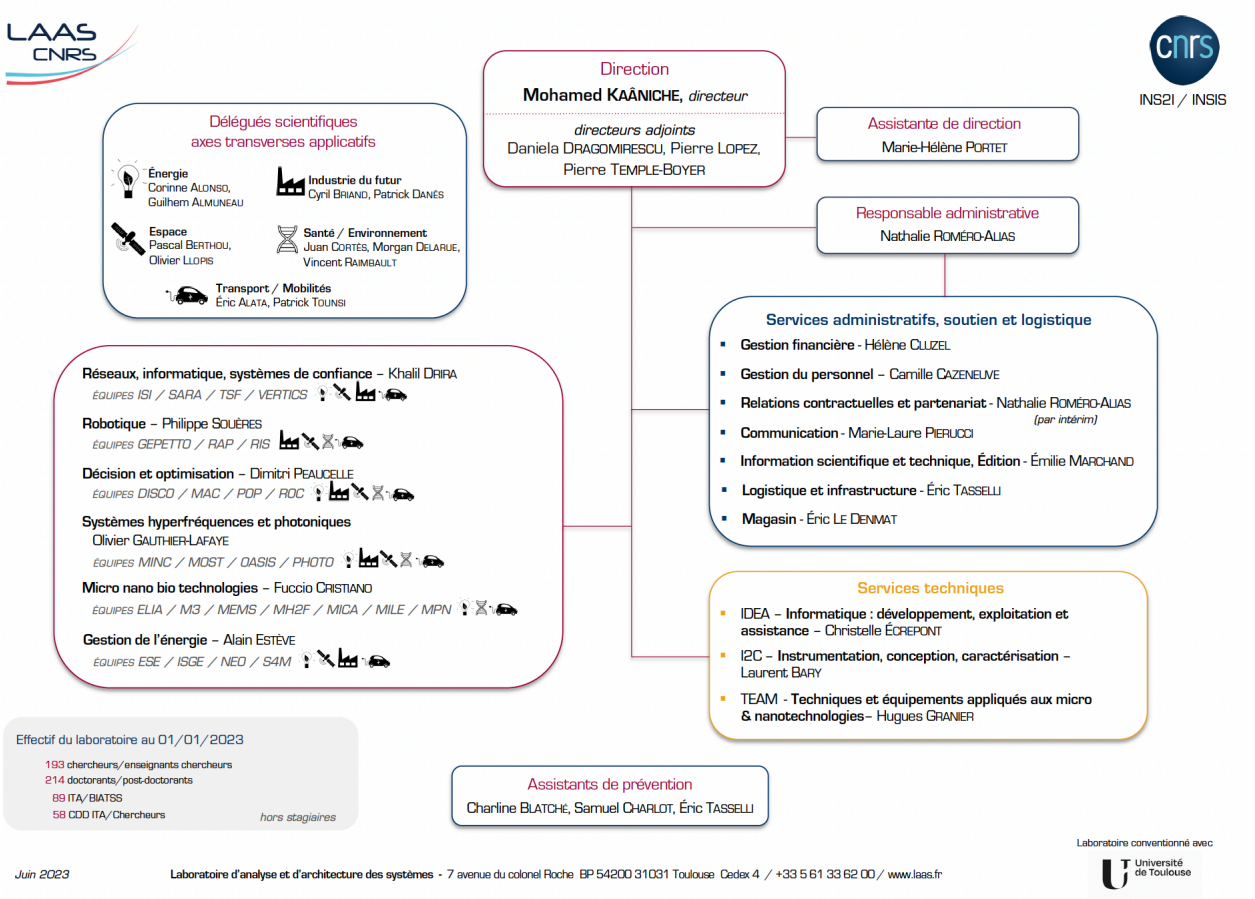


FIGURE 3.1 : Organigramme du LAAS-CNRS [74]

3.2.2 Département RISC

Notre travail se déroule au sein de l'équipe SARA, qui fait partie du département RISC (Réseaux, Informatique, Systèmes de Confiance). Le département RISC se consacre à la recherche et au développement de solutions innovantes visant à maîtriser tous les aspects de la conception, de l'architecture, de l'analyse, de la supervision et du déploiement opérationnel des réseaux et des systèmes de communication complexes, ainsi que des applications qui leur sont associées. Leurs recherches se concentrent sur les systèmes et les applications ayant des exigences élevées en matière de qualité de service (QoS), de contraintes temporelles, de sûreté de fonctionnement, de résilience et de sécurité. L'objectif est de contribuer à l'avancement des connaissances, en mettant l'accent sur les fondements théoriques et les approches méthodologiques basées sur des méthodes formelles et expérimentales, tout en veillant à leur mise en œuvre pratique.

3.2.2.1 L'équipe SARA

L'équipe SARA (Services et Architectures pour les Réseaux Avancés) se consacre à la recherche sur les réseaux et les systèmes de communication de nouvelle génération, ainsi que sur leurs applications. Cette orientation est motivée par la croissance constante de l'utilisation de ces systèmes, ce qui nécessite une prise en compte accrue de leur importance critique dans divers domaines d'activité humaine.

3.3 Présentation du projet de recherche

Les véhicules autonomes intègrent des systèmes de pilotage complexes, leur permettant de planifier des itinéraires vers des destinations spécifiques et d'ajuster leur conduite en conséquence. Cette planification doit tenir compte à la fois d'événements prévisibles, tels que des travaux routiers, et d'événements aléatoires, tels que des incidents imprévus.

Pour accomplir cette tâche, les véhicules dépendent d'informations sur l'état actuel de la circulation, obtenues grâce à des capteurs embarqués et des applications telles que Google Maps et Waze. Ces données sont transmises à des serveurs dans le cloud, qui les traitent et les transmettent aux véhicules autonomes. Ces informations sont essentielles pour adapter l'itinéraire en temps réel.

Cependant, la communication avec des serveurs distants présente des défis liés à la latence, ce qui peut entraîner des retards importants et compromettre la sécurité des usagers. Pour relever ces défis, la prochaine génération de véhicules autonomes doit adopter une approche axée sur la fiabilité et la performance.

Cela implique la mise en place d'une architecture de pilotage distribué, permettant aux véhicules de déléguer certains calculs à des unités externes plus puissantes. Ces unités doivent maintenir une communication à faible latence pour une interaction en temps réel. De plus, étant donné la mobilité des véhicules, ces unités externes doivent être déployées à proximité des véhicules pour réduire la latence, en utilisant des méthodes de migration pour garantir la continuité du service lors des déplacements.

Enfin, le réseau supportant ces interactions doit être flexible et capable de s'adapter dynamiquement pour garantir une connectivité sans interruption. Ce projet vise à réaliser cette vision en abordant ces défis cruciaux pour l'intégration réussie des véhicules autonomes dans les villes intelligentes.

3.4 Solution et Architecture Proposées

Pour répondre aux besoins préalablement identifiés, une architecture a été mise en place, se composant de trois domaines principaux : le domaine du véhicule autonome, le domaine du réseau 5G, et le domaine de la ville intelligente.

Dans le domaine du véhicule autonome, une architecture de microservices a été mis en place pour les applications liées à la navigation autonome. Ces microservices sont regroupés en trois modules clés : le Module de Perception (qui collecte des données de navigation à partir des capteurs embarqués tels que les caméras), le Module de Décision (qui prend des décisions en fonction des données de Perception), et le Module de Navigation (qui exécute ces décisions et détermine l'itinéraire optimal).

Dans le domaine du réseau 5G, nous distinguons le cœur du réseau, situé dans le cloud. Ce domaine héberge également des serveurs internes au réseau. À la périphérie du réseau se trouvent les nœuds edge, qui rapprochent le traitement des données des véhicules, améliorant ainsi la latence réseau.

Enfin, dans le domaine de la ville intelligente, la ville est divisée en zones du réseau 5G appelées Points of Presence (PoPs). Chaque PoP est équipé d'un d'un Système d'Antennes Distribuées, d'un Administrateur des nœuds edge, et d'un outil d'Orchestration. L'outil d'Orchestration assure le

déploiement et l'exécution des modules du véhicule autonome sur les nœuds edge.

Toutes ces composantes interagissent pour former une architecture globale visant à soutenir la navigation autonome des véhicules dans le contexte d'une ville intelligente. Le déploiement des microservices se fait dans des conteneurs, et l'orchestrateur utilisé est Kubernetes [73]. De plus, la communication est basée sur le protocole WebSockets basé sur TCP/HTTP, afin de supporter les communications en temps réel.

Cette architecture est illustrée dans la Figure 3.2.

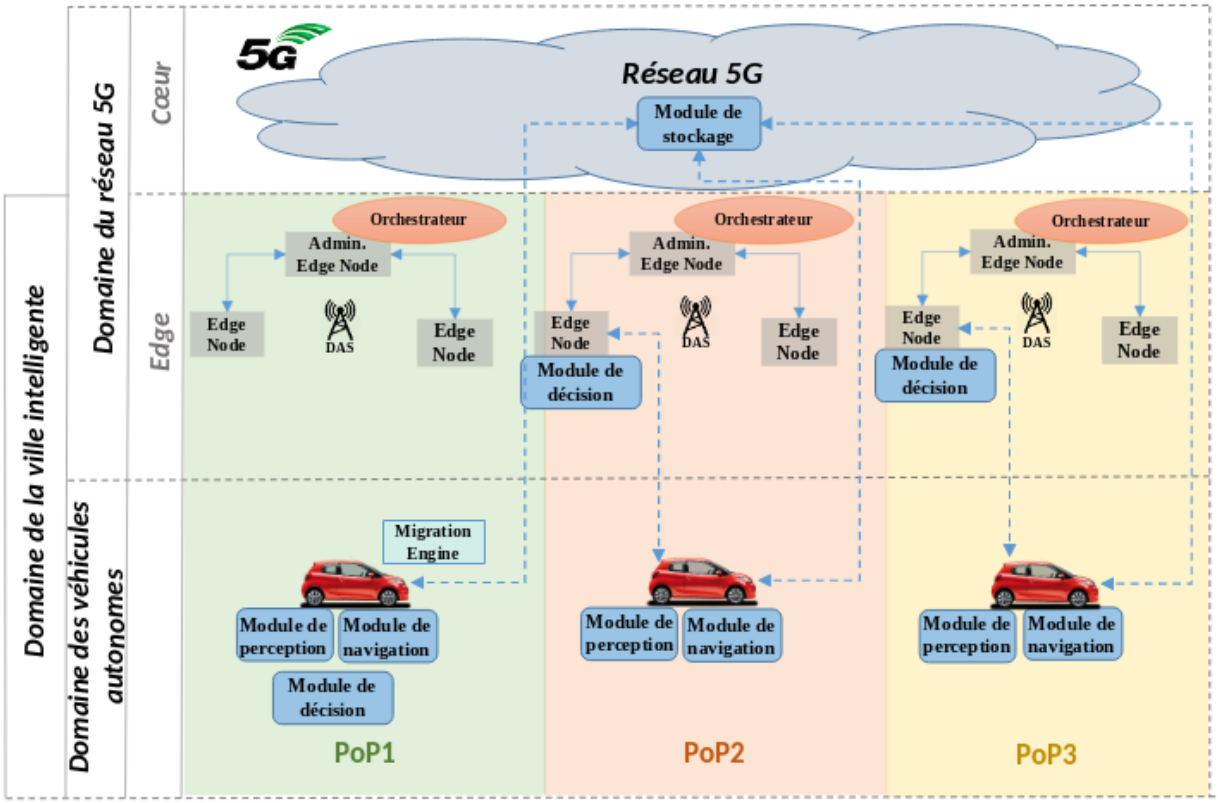


FIGURE 3.2 : Architecture proposée pour l'intégration des véhicules autonomes dans la ville intelligente [39]

Le diagramme de séquence présenté dans la Figure 3.3 offre une vue d'ensemble du fonctionnement de l'architecture en mettant en lumière les différentes étapes qui la composent, les parties intervenants impliqués, ainsi que les actions exécutées.

Il détaille les processus qui s'exécutent au sein de l'écosystème de la ville intelligente. Plus précisément, il illustre comment un véhicule autonome, grâce à l'edge computing, établit une connexion avec un nœud edge disponible, supervisé par un Orchestrateur. Cette connexion a pour objectif de délocaliser les opérations de traitement de données effectuées par le module de décision du véhicule.

Ces processus font usage d'outils appropriés, et la communication entre le nœud edge hébergeant le module de décision et le véhicule équipé des modules de perception et de navigation se fait par le biais d'une technologie adaptée, en l'occurrence le protocole WebSocket, qui favorise les échanges en temps réel.

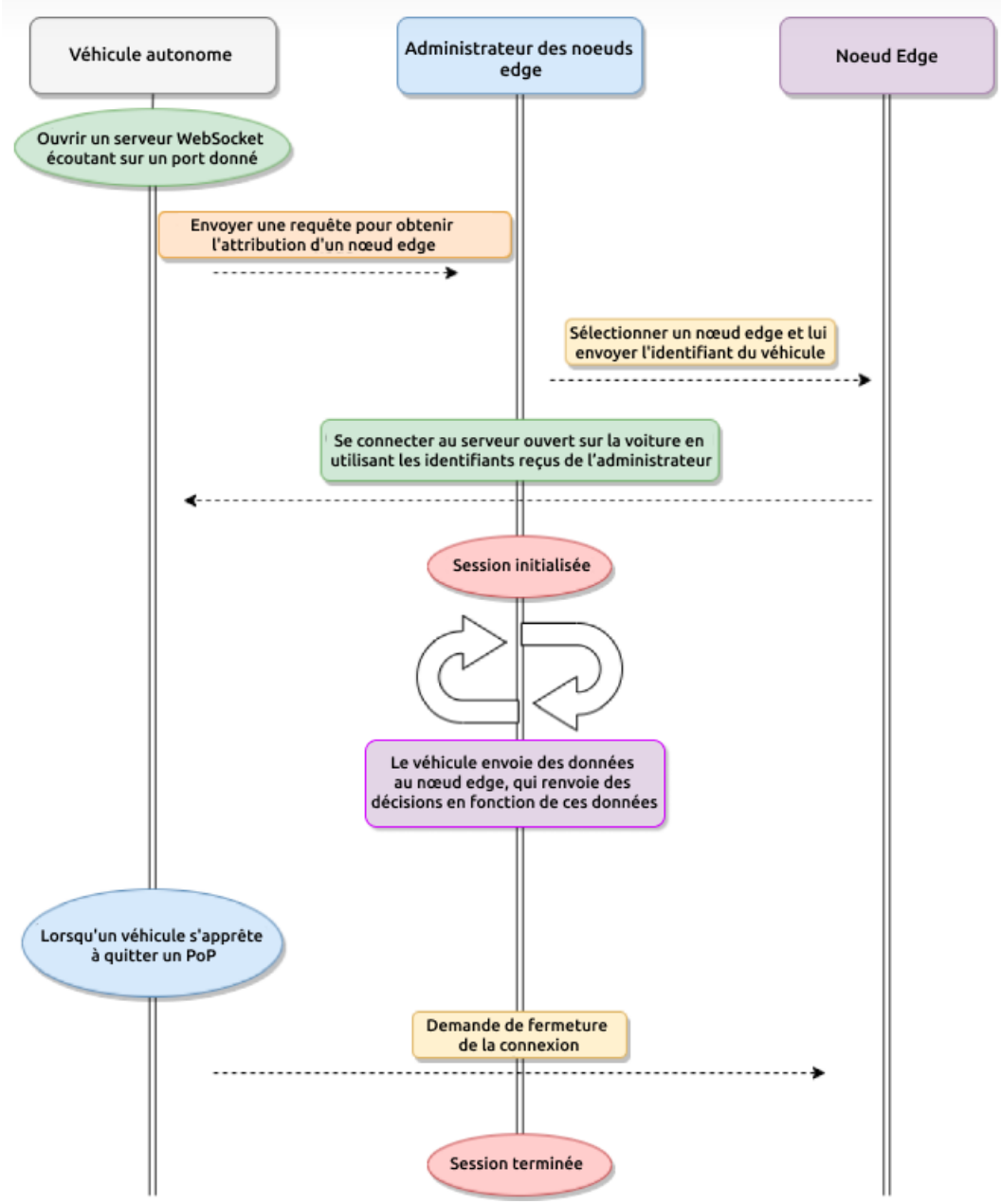


FIGURE 3.3 : Diagramme de séquence du fonctionnement de l'architecture

3.5 Analyse des Avantages et des Défis de l'Architecture Actuelle

Cette section examine les avantages et les défis auxquels l'architecture actuelle est confrontée.

3.5.1 Avantages

3.5.1.1 Tolérance aux Pannes

La répartition des tâches sur des modules distincts assure une tolérance aux pannes plus élevée, minimisant l'impact des défaillances sur l'ensemble du système. En cas de panne d'un module, les autres peuvent continuer à fonctionner sans perturbation majeure.

3.5.1.2 Modularité

L'architecture de microservices favorise la modularité, découpant le système en composants indépendants. Cela facilite les mises à jour spécifiques à chaque module, améliorant la maintenance et la flexibilité du système dans son ensemble. De plus, l'architecture de microservices combinée à la technologie de conteneurisation offre la possibilité de garder une certaine stabilité des métriques de QoS, en particulier la latence, malgré la mobilité du véhicule. Cela est possible grâce à la facilité de migration de service qu'elle offre.

3.5.1.3 Réduction du Temps de Traitement

La délocalisation des calculs vers des nœuds edges puissants permet de réduire significativement le temps de traitement des données. Cette optimisation contribue à des performances améliorées, permettant au système de réagir rapidement aux événements en temps réel. La Figure 3.4 illustre la variation du temps de traitement lorsque le module est exécuté à l'intérieur du véhicule par rapport au temps de traitement lorsqu'il est externalisé vers un nœud edge. L'expérience a montré que déplacer le module vers un nœud edge plus puissant permettait une réduction du temps de traitement de 50%.

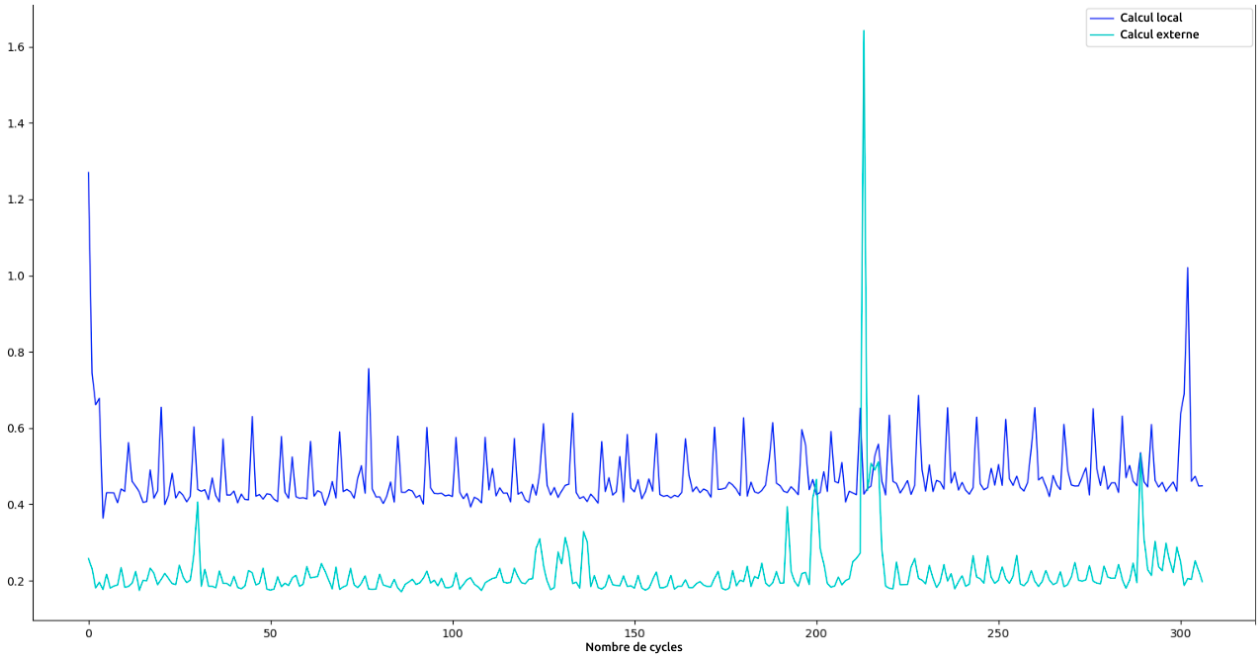


FIGURE 3.4 : Comparaison des temps de traitement entre exécution locale et externalisation vers un nœud edge

3.5.2 Défis

3.5.2.1 Latence due aux Flux TCP/HTTP

Les flux de connexion basés sur TCP et HTTP, utilisés par la technologie WebSocket, peuvent engendrer une latence non négligeable dans le système. La nature séquentielle de ces protocoles peut impacter la réactivité en temps réel des modules, ralentissant les échanges de données. Leur fiabilité inhérente peut entraîner des pertes de performances pour les systèmes temps réel.

3.5.2.2 Problèmes de Sécurité

La communication entre les modules et le déplacement des données entre différents nœuds peuvent exposer le système à des vulnérabilités en matière de sécurité. Une protection robuste doit être mise en place pour éviter les violations et les manipulations non autorisées.

3.5.2.3 Mécanisme de sélection des Nœuds Edges

Dans l'architecture actuelle du système, un défi majeur réside dans la gestion des nœuds edge au sein d'un Point de Présence (PoP) par le biais de l'outil de gestion des conteneurs, Kubernetes. Bien que ce dernier offre une manière efficace de gérer des nœuds et de planifier des charges de travail, il existe certaines limitations et points faibles à prendre en compte, notamment en ce qui concerne la communication entre les véhicules et les nœuds et les charges de travail variées au sein de ces nœuds.

L'un des points à considérer est que les nœuds peuvent accueillir d'autres types de charges de travail en dehors des conteneurs gérés par Kubernetes. Les nœuds edge peuvent également exécuter des tâches spécifiques à leur rôle dans l'écosystème (la ville). Ils ne sont pas limités aux conteneurs Kubernetes contenant les modules des véhicules autonomes.

Kubernetes est conçu pour gérer principalement les charges de travail des conteneurs qu'il gère dans un nœud. Cela signifie qu'il pourrait ne pas prendre en compte efficacement les charges de travail non-Kubernetes. Dans une ville, où des nœuds edge peuvent exécuter une variété de tâches distinctes, rend d'autant plus important de tenir compte de l'ensemble de ces facteurs.

De plus, l'approche actuelle néglige un aspect crucial : la communication réseau entre les véhicules autonomes et les nœuds edge. Les deux entités sont mobiles, ce qui signifie que la qualité et la latence du réseau peuvent varier considérablement. Le processus de sélection des nœuds edge ne tient pas compte des conditions de réseau. Pourtant, la communication efficace entre les véhicules et les nœuds edge est essentielle pour garantir des performances optimales et des décisions en temps réel.

De manière générale, la stratégie de sélection des nœuds edge nécessite une réflexion plus approfondie. Prendre en compte les charges et performances des nœuds disponibles et des contraintes de communication réseau devrait jouer un rôle central dans le processus de prise de décision pour garantir des performances optimales et une réactivité en temps réel, même dans des conditions changeantes.

3.6 Positionnement du PFE dans le contexte de recherche

Notre Projet de Fin d'Études, intégré dans un projet de recherche plus vaste, se concentrant principalement sur l'amélioration du mécanisme de sélection des nœuds Edge. Plus précisément, notre objectif est d'anticiper les dégradations de la qualité de service (QoS) au niveau de ces nœuds Edge, en tenant compte de deux aspects cruciaux : la charge de travail des nœuds Edge et la communication réseau entre les véhicules autonomes et ces nœuds. Cette démarche permettra une sélection plus précise des nœuds capables de répondre aux exigences de QoS des véhicules autonomes, garantissant ainsi leur bon fonctionnement.

Pour y parvenir, nous devons mettre en place un système de prédiction continue des paramètres de la QoS pour l'ensemble des nœuds Edge. Cela implique la surveillance en temps réel des paramètres liés à la communication réseau entre les véhicules autonomes et les nœuds, ainsi que la

charge de travail de ces nœuds, susceptible d'impacter négativement les temps de traitement. En faisant cela, il serait possible d'alerter le système à l'avance en cas de risque de dégradation potentielle d'un nœud Edge ou de perturbation de la communication avec celui-ci.

Cette tâche de prédiction requiert un modèle d'une grande précision et capable de prédire rapidement, ce qui permet au système de réagir efficacement. Notre projet de fin d'études s'inscrit précisément dans cette perspective, il se concentre principalement sur le développement et l'évaluation de divers modèles de prédiction de la QoS.

Pour étayer notre recherche, un ensemble de données a été constitué, contenant les métriques de QoS les plus pertinentes, ainsi que les métriques liées aux nœuds Edge. Ces données ont été collectées en mettant en place un prototype de l'architecture envisagée pour l'intégration des véhicules autonomes dans les villes intelligentes. Cette approche garantit que les données recueillies reflètent de manière étroite les caractéristiques réelles, telles qu'elles seraient mises en œuvre dans une situation réelle.

La prochaine section détaillera le jeu de données collecté grâce à la mise en place du prototype. Ce jeu de données est indispensable pour les étapes de modélisation et d'évaluation de nos modèles de prédiction de la QoS.

3.6.1 Collecte du jeu de données

La Figure 3.5 ci-dessous présente le setup déployé pour la réalisation concrète du prototype. Il montre un véhicule autonome ainsi que trois nœuds, chacun assigné à une zone (Point of Presence - PoP), formant ainsi un total de trois zones. Dans ce contexte, le module de décision du véhicule a été délocalisé pour être exécuté au niveau des nœuds Edge.

Le véhicule circule entre ces trois zones et établit des connexions avec les différents nœuds pour communiquer avec son module de décision. Le serveur de présence initialise des sessions entre le véhicule et les nœuds Edge, permettant ainsi leur communication. Le premier nœud Edge est mobile, tandis que les 2ème et 3ème nœuds sont stationnaires.

De plus, le véhicule est équipé d'une caméra pour la détection de dangers potentiels. Grâce à l'exécution de ce prototype, une mise en place concrète de l'architecture envisagée a été réalisée, ce qui a permis de collecter un jeu de données exhaustif, comprenant de multiples caractéristiques. Ce jeu de données a été généré en surveillant à la fois la communication entre le véhicule et les nœuds Edge (le délais), ainsi que la charge de travail de ces nœuds (RAM et CPU).



FIGURE 3.5 : Setup du prototype avec un véhicule et trois nœuds

L'ensemble de données collectées se compose de 20 000 observations espacées de 3 secondes, extraites d'un environnement dans lequel nous avons une voiture autonome et 3 nœuds edge. La voiture autonome envoie des requêtes aux 3 nœuds toutes les 3 secondes. La voiture et le nœud 1 sont en mouvement dans l'espace, tandis que le nœud 2 et le nœud 3 sont stationnaires. Chaque observation est décrite par l'instant t auquel elle est observée, ainsi que par 11 métriques :

- ***node1_delay, node2_delay et node3_delay.*** Le temps de ping (uniquement le temps de trajet, le délai) pour la requête envoyée par la voiture aux nœuds 1, 2 et 3 respectivement.
- ***node1_cpu, node2_cpu et node3_cpu.*** Charge CPU des nœuds 1, 2 et 3 lors de la réception de la requête de la voiture, respectivement.
- ***node1_ram, node2_ram et node3_ram.*** Charge mémoire des nœuds 1, 2 et 3 lors de la réception de la requête de la voiture, respectivement.
- ***node1_x, node1_y.*** La position du nœud 1.

Cet ensemble de données contient 9 séries chronologiques sur lesquelles nous nous concentrons pour la prédiction : *node1_delay*, *node2_delay*, *node3_delay*, *node1_ram*, *node2_ram*, *node3_ram*, *node1_cpu*, *node2_cpu* et *node3_cpu*. Ces séries incluent des caractéristiques concernant la communication réseau entre le véhicule et les nœuds edge (délais) ainsi que des caractéristiques relatives à la charge de travail des nœuds edge (RAM et CPU de chaque nœud). Les coordonnées du nœud 1 sont incluses pour déterminer si elles peuvent contribuer à prédire le délai de ce nœud. Cependant, comme le montre la Figure 3.6, il n'y a aucune corrélation entre les coordonnées et le délai du nœud 1. Cela est tout à fait logique car le processus de collecte et le prototype a été effectué dans une salle de laboratoire dont la taille ne dépasse pas $50m^2$. Par conséquent, la distance dans ce cas est négligeable et n'affectera pas

le délai de communication réseau entre le véhicule et le premier nœud edge. C'est pourquoi nous ne tiendrons pas compte de ces deux caractéristiques. Le tableau 3.1 décrit les caractéristiques du jeu de données.

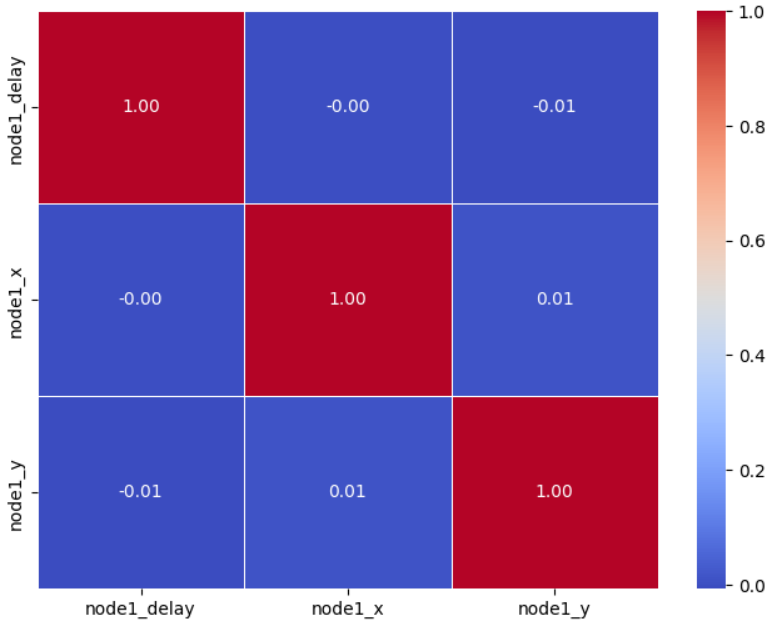


FIGURE 3.6 : Corrélation entre le délai de communication du premier nœud edge et ses coordonnées spatiales

Variable	Compteur	Moyenne	Écart-type	Min	25%	50%	75%	Max
node1_delay	20000.0	0.323828	0.152275	0.059012	0.191918	0.324079	0.456506	0.586923
node2_delay	20000.0	0.200986	0.053737	0.107939	0.154545	0.200765	0.247665	0.294275
node3_delay	20000.0	0.215486	0.086349	0.065931	0.140327	0.215022	0.290110	0.364288
node1_ram	20000.0	0.548665	0.149550	0.300000	0.400000	0.500000	0.700000	0.800000
node2_ram	20000.0	0.550655	0.149939	0.300000	0.400000	0.600000	0.700000	0.800000
node3_ram	20000.0	0.548180	0.149552	0.300000	0.400000	0.500000	0.700000	0.800000
node1_cpu	20000.0	0.534699	0.252282	0.100000	0.310000	0.530000	0.760000	0.970000
node2_cpu	20000.0	0.536648	0.251351	0.100000	0.320000	0.540000	0.750000	0.970000
node3_cpu	20000.0	0.535996	0.251008	0.100000	0.320000	0.540000	0.750000	0.970000
node1_x	20000.0	2.489950	1.712643	0.000000	1.000000	2.000000	4.000000	5.000000
node1_y	20000.0	2.501800	1.706266	0.000000	1.000000	2.000000	4.000000	5.000000

TABLE 3.1 : Description des caractéristiques du premier ensemble de données.

Chaque variable dans le tableau contient 20 000 observations. La moyenne indique la valeur moyenne de ces observations, tandis que l'écart-type mesure la dispersion des données par rapport à la moyenne. Un écart-type plus petit indique que les valeurs sont regroupées plus près de la moyenne, tandis qu'un écart-type plus grand suggère une plus grande dispersion des valeurs. Les valeurs "Min" et "Max" représentent respectivement les valeurs minimales et maximales observées pour chaque variable.

Les quartiles (25%, 50%, et 75%) indiquent la répartition des valeurs en quatre parties égales. Par exemple, le 25e percentile (Q1) sépare les 25% des valeurs les plus basses des 75% restantes. De même, le 50e percentile (Q2) est la médiane, qui divise les données en deux parties égales. Le 75e percentile (Q3) sépare les 75% des valeurs les plus basses des 25% restantes.

Ces statistiques aident à comprendre la tendance centrale, la dispersion et la distribution des données pour chaque variable.

3.6.2 Importance de la prédiction de QoS : Scénario Illustrant

Pour mieux comprendre les conséquences des variations de QoS et pour mettre en évidence l'importance de la prédiction des paramètres de QoS, nous présentons ci-dessous un scénario démontrant comment une approche prédictive peut atténuer les risques et améliorer la sécurité de la conduite autonome.

Imaginons que nous ayons un véhicule connecté via le réseau. Le module de décision du véhicule, responsable de toutes les politiques et décisions de conduite, est migré dans un nœud edge car il effectue des calculs complexes basés sur les données de capteurs reçues. Le module de décision est sensible à la latence et est un service crucial dans le moteur de conduite car il doit être disponible et fonctionnel tant que la voiture est en mouvement.

Le module de décision envoie des messages de commande au véhicule toutes les 20 ms :

1. Le module de décision reçoit les conditions de la route (par exemple, les obstacles) et les informations sur l'état des véhicules voisins (par exemple, la localisation, la vitesse, la dynamique, etc.), les informations sur l'état du véhicule (par exemple, la vitesse, la localisation) et les conditions de circulation.

2. En fonction des informations reçues, le module de décision construit le modèle de l'environnement (c'est-à-dire la perception de l'environnement du véhicule) et, en tenant compte du point de destination, sélectionne la trajectoire et les instructions de manœuvre.

3. Le véhicule reçoit du module de décision des instructions de trajectoire et/ou de manœuvre et les exécute.

4. Le délai de service pour les messages de commande du module de décision au véhicule doit être inférieur à 20 ms, car une commande est reçue toutes les 20 ms.

5. Le modèle de prédiction prévoit que la latence pourrait se dégrader dans une fourchette de 20 à 30 ms dans les 30 prochaines secondes.

6. Le système est notifié d'une éventuelle dégradation de QoS.

7. Le système signale au module de décision de choisir un autre nœud edge garantissant une meilleure latence, de réduire la vitesse du véhicule s'il peut toujours continuer à envoyer en

toute sécurité des commandes au véhicule malgré la latence plus élevée (en fonction, par exemple, de la vitesse et de l'environnement environnant), ou d'arrêter le véhicule en toute sécurité.

Toutefois, en l'absence de prédictions, en raison des fluctuations potentielles du réseau, des conditions environnementales ou d'autres facteurs imprévisibles, la latence pourrait varier. Dans certaines circonstances, cette latence pourrait excéder les 20 ms, entraînant ainsi des délais dans la réception des instructions de conduite. Lorsque la latence devient élevée, le véhicule pourrait éprouver des difficultés à exécuter les instructions, ce qui risquerait de perturber la sécurité de la conduite. Ces délais pourraient induire des erreurs de réaction, des situations dangereuses ou même une perte de contrôle temporaire, mettant ainsi en danger la sécurité des passagers à bord ainsi que celle des autres usagers de la route.

3.7 Conclusion

Ce chapitre a posé les fondations essentielles pour la compréhension de notre projet en présentant en détail le cadre général du projet, ainsi que l'architecture proposée pour l'intégration des véhicules autonomes dans les villes intelligentes. Cette intégration repose sur le concept de rapprochement des ressources de calcul auprès des véhicules grâce à l'edge computing.

L'architecture actuelle présente plusieurs défis, parmi lesquels nous avons notamment mentionné le mécanisme de sélection des nœuds edge. Cette étape revêt une importance cruciale, car elle détermine quel nœud edge sera chargé du traitement des données pour un véhicule donné. La QoS offerte par ces nœuds edge joue un rôle déterminant dans le bon fonctionnement des véhicules autonomes, en particulier dans des situations critiques, telles que la prise de décision en temps réel.

En mettant en lumière ces éléments, ce chapitre a donc établi un contexte solide pour notre travail à venir en mettant en évidence le rôle crucial de la prédiction de la QoS dans la sélection des nœuds edge. Cette prédiction permet d'anticiper les dégradations potentielles de la performance des nœuds edge au sein de cette architecture.

Dans le chapitre suivant, nous présentons notre solution, en examinant en détail le processus que nous proposons pour le développement et l'évaluation des modèles de prédiction de la QoS.

Chapitre 4

Construction des modèles de prédiction de la QoS

4.1 Introduction

L'objectif central de notre travail réside dans le développement et l'évaluation des modèles de prédiction de la QoS. Notre démarche vise à construire des modèles ayant l'intrinsèque capacité de fournir des prédictions d'une grande précision concernant les indicateurs clés de performance (KPI) de la QoS. Notamment, étant donné que les données de QoS sont de nature très complexe, comme nous l'avons observé dans l'état de l'art. De plus, il est essentiel que ces modèles puissent générer des prédictions rapidement, c'est-à-dire offrir des temps de réponse réduits. Cette capacité est cruciale pour offrir suffisamment de temps au système afin de réagir de manière efficace et en temps réel face à d'éventuelles dégradations de la QoS.

Ce chapitre se penche sur la conception d'une solution de prédiction de la QoS pour les véhicules autonomes. Nous y détaillerons le processus adoptée pour développer des modèles de prédiction robustes.

4.2 Cadre d'application

Cette section établit le cadre dans lequel notre solution est déployée, elle offre une vue d'ensemble détaillée de l'architecture du système :

4.2.1 Vue d'ensemble de l'architecture du système

L'architecture du système, illustrée dans la Figure 4.1, repose sur Kubernetes, avec chaque nœud de travail (worker node) étant également un nœud edge ou une machine virtuelle dans un nœud edge. Kubernetes supervise la gestion du service de monitoring ainsi que du moteur de migration. Le module de décision pour les voitures autonomes et potentiellement d'autres services sont intégrés dans les nœuds de travail.

Le service de monitoring surveille en temps réel les ressources du nœud (comme la RAM et le CPU) ainsi que la performance des communications réseau avec le véhicule autonome (comme la

latence). C'est également là que se trouve notre module de prédiction de séries chronologiques.

Si le service de monitoring anticipe une dégradation de la qualité de service (QoS), il informe le moteur de migration. Ce dernier prend alors la décision de transférer le module de décision du véhicule vers un autre nœud. Cette décision est basée sur les prédictions des services de monitoring des autres nœuds edge. Ainsi, le service de monitoring joue un rôle central dans cette architecture.

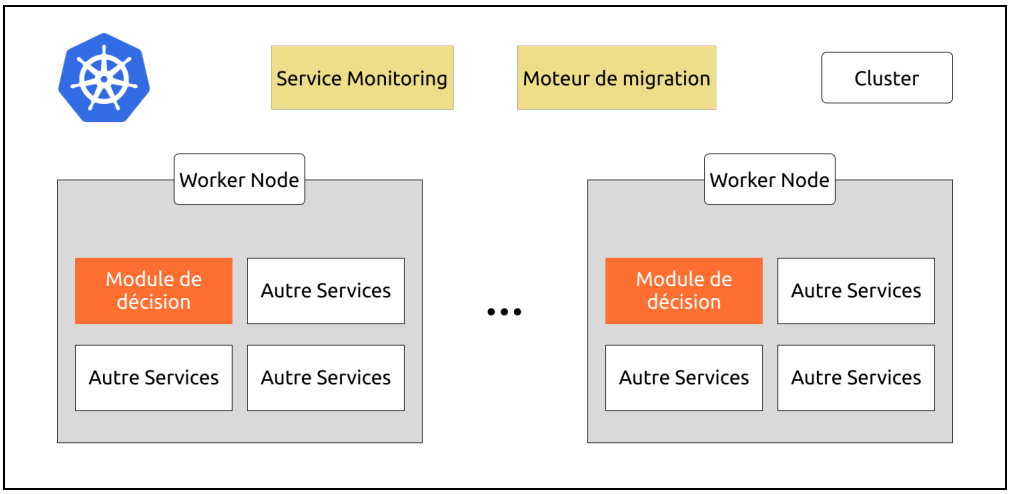


FIGURE 4.1 : Vue d'ensemble de l'architecture du système

4.2.2 Service de monitoring

Le service de monitoring repose principalement sur un modèle de prédiction des séries chronologiques. En temps réel, il reçoit des mesures des différents attributs ou indicateurs clés de performance (KPI) ciblés, puis génère des prédictions en continu. L'aspect "en temps réel et en ligne" signifie que les prédictions sont élaborées instantanément à mesure que de nouvelles données arrivent.

Ces prédictions sont ensuite transmises à un module de surveillance continue des prédictions. Ce module génère des notifications destinées au moteur de migration si une dégradation de QoS est anticipée. Une dégradation de QoS est signalée lorsqu'une valeur prédite d'un KPI dépasse le seuil prédéfini, appelé le seuil de dégradation.

L'ensemble de cette procédure repose sur un modèle de prédiction performant capable de saisir les dépendances des données de QoS et de générer des prédictions précises. Dans le cadre de ce projet, notre objectif se concentre principalement sur la première phase, qui est réalisée hors ligne (offline). Cette phase consiste à développer des modèles de prédiction basés sur des données historiques de QoS, puis à évaluer ces modèles. L'évaluation ne se limite pas seulement à l'erreur ou à la précision des prédictions, mais inclut également la prise en compte du temps de traitement des modèles. Ce facteur temporel est crucial dans notre contexte, où chaque milliseconde est importante.

La Figure 4.2 illustre le fonctionnement du service de monitoring ainsi que la phase offline de génération des modèles de prédiction, montrant comment ces éléments s'intègrent dans l'architecture globale.

Dans la section 4.3, nous examinerons en détail le processus de développement et d'évaluation des modèles de prédiction de la QoS.

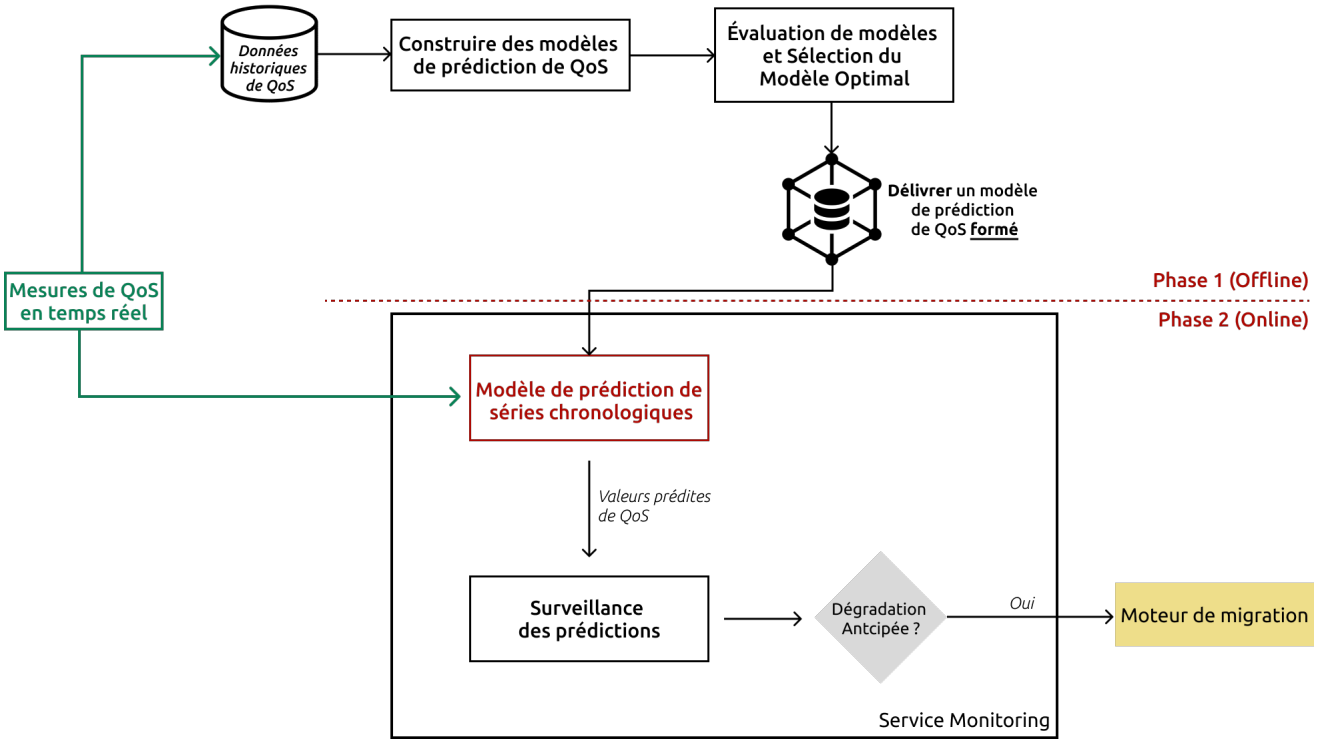


FIGURE 4.2 : Service de monitoring

4.3 Processus de construction des modèles de prédiction

Dans cette section, nous dévoilons le processus que nous avons adopté pour le développement des modèles de prédiction de la QoS. De la collecte des données à l'évaluation des modèles. Nous détaillons chaque étape cruciale du processus. L'objectif est de créer des modèles précis, tout en prenant en compte les contraintes temporelles essentielles pour les applications des véhicules autonomes.

La structure générale du processus proposé est présenté dans la Figure 4.3. Notre structure conceptuelle se compose de trois étapes : le prétraitement des données, la construction des modèles et l'évaluation des modèles.

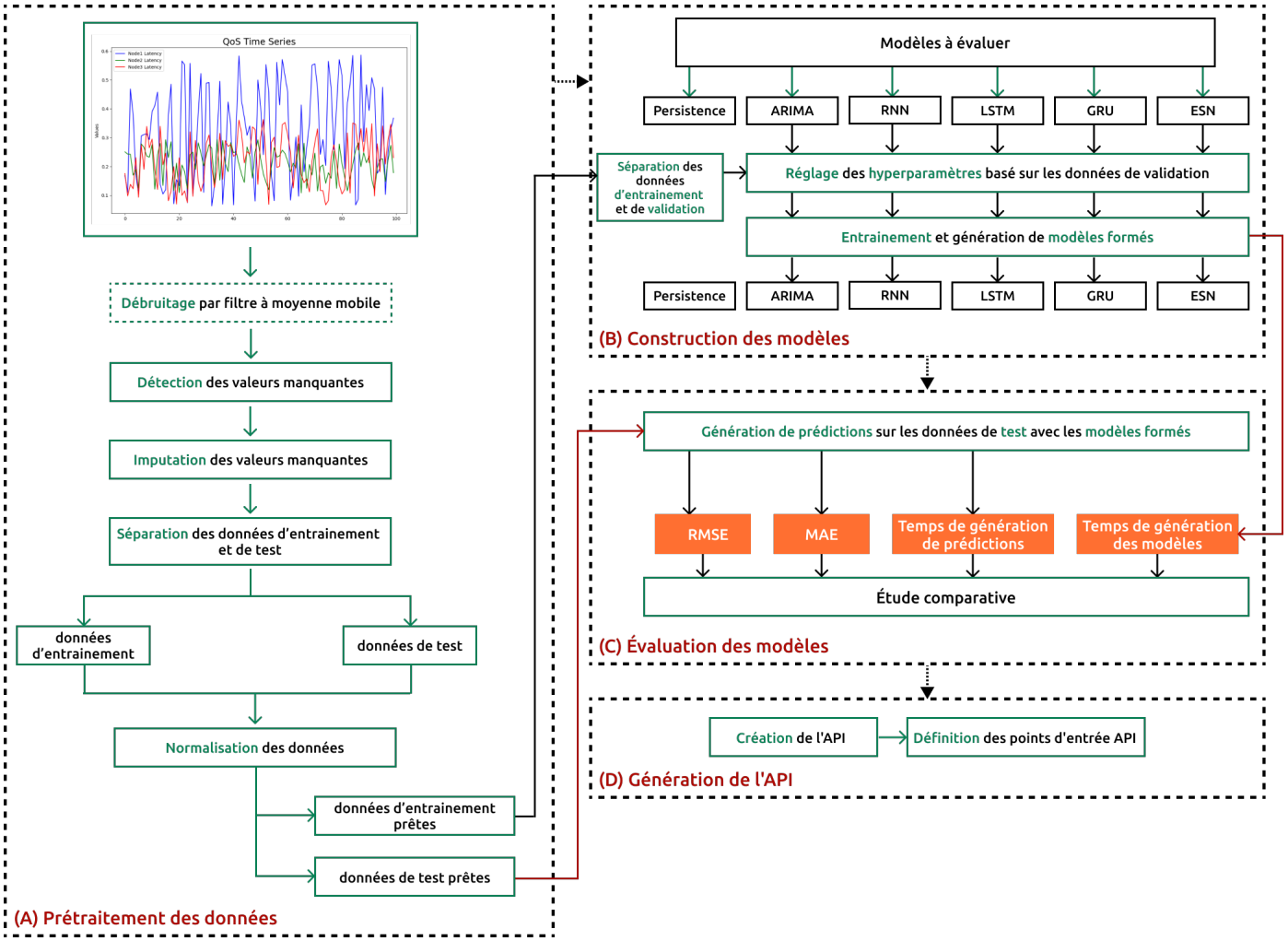


FIGURE 4.3 : Processus adopté pour la construction des modèles de prédiction de la QoS

4.3.1 Collecte de données

Avant d'entreprendre toute démarche d'analyse ou de prétraitement des données, il est impératif de disposer d'un jeu de données approprié pour garantir la solidité de nos évaluations et la pertinence de nos modèles. Cette section se consacre à la collecte des données, une étape cruciale dans le processus global de développement et d'évaluation de modèles de prédiction de la QoS.

Optant pour une approche rigoureuse, l'utilisation de données collectées à l'aide d'un prototype mettant en œuvre l'architecture envisagée de l'intégration des véhicules autonomes dans les villes intelligentes nous a permis de nous assurer que les caractéristiques des données recueillies reflètent au mieux des scénarios réels, ce qui est essentiel pour obtenir des résultats fiables dans cette étude.

Nous avons fourni une description complète du processus de collecte de données dans la section 3.6.1 du chapitre 3.

4.3.2 Prétraitement de données

Dans cette étape, nous mettons en œuvre des techniques de prétraitement pour garantir la qualité et la fiabilité des données.

4.3.2.1 Débruitage par filtre à moyenne mobile.

Tout d'abord, nous employons un filtre à moyenne mobile, "Moving Average" en anglais, afin de réduire les bruits indésirables présentes dans les données. Cette méthode simple de réduction de bruit consiste à calculer la moyenne au sein d'une fenêtre de points de données, dans le but d'atténuer les interférences non souhaitées. Le fonctionnement du filtre à moyenne mobile est illustré dans la Figure 4.4. Lorsqu'une fenêtre de taille 2 est employée, les paires de valeurs consécutives sont moyennées puis déplacées d'une unité, et ce processus est répété jusqu'à ce qu'un signal débruité soit obtenu. La dimension de la fenêtre est le seul paramètre nécessitant estimation dans ce filtre. C'est la raison pour laquelle nous avons opté pour ce filtre en particulier : il est simple à implémenter, très efficace, et ne requiert qu'un seul paramètre à estimer.

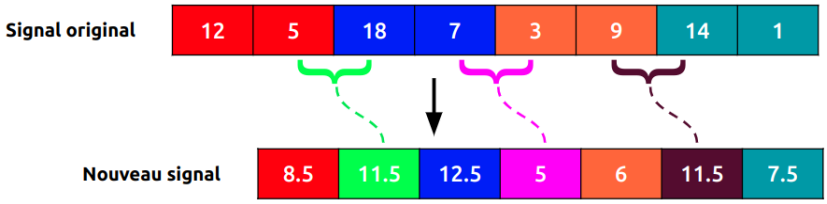


FIGURE 4.4 : Fonctionnement du filtre à moyenne mobile pour la réduction de bruit

Pour illustrer l'importance de la réduction du bruit dans les données, le graphique présenté dans la figure 4.5 montre les prédictions du meilleur modèle, celui ayant affiché le RMSE le plus bas sur les données de test. Cela a été effectué sur l'une de nos séries chronologiques caractérisée par un niveau de bruit très élevé. Cependant, le défi réside dans le fait que l'amplitude du signal propre (signal significatif) est inférieure à celle des perturbations, ce qui entraîne une prédominance du bruit. Étant donné que les valeurs du signal se situent également autour de 0, elles sont faibles en amplitude. Le bruit (ou bruit blanc) est défini comme un signal de moyenne 0 et imprévisible, suivi d'une distribution normale. Par conséquent, le modèle s'efforce de prédire le bruit aléatoire prédominant dans la série chronologique, car il constitue le signal dominant. Ce bruit échappe à toute anticipation par un modèle (signal imprévisible) et ne peut même pas être capturé à l'échelle des données. Les prédictions (en orange) présentent des perturbations minimes, ce qui implique la présence d'informations cachées dans ce signal.

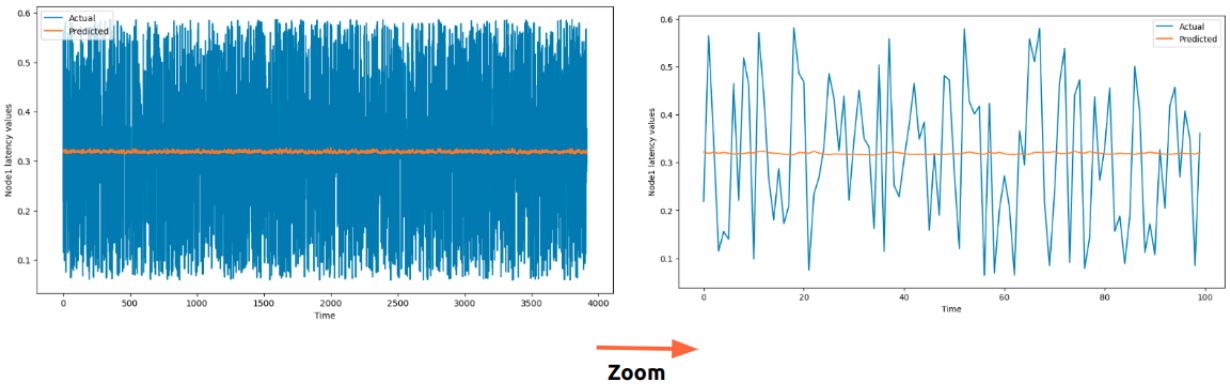


FIGURE 4.5 : Prédictions sur une série chronologique à forte teneur en bruit

Pour démontrer la pertinence des informations véhiculées par cette série, nous avons recouru à une méthode statistique nommée le test de Durbin-Watson. Fondamentalement, ce test vise à déceler les corrélations entre les valeurs successives d'une série, communément appelée "autocorrélation". Il s'applique aux résidus d'un modèle de prédiction de séries chronologiques, où les résidus représentent la différence entre la série originale et celle prédite par le modèle. L'objectif de cette démarche est d'évaluer si les résidus manifestent une corrélation significative, susceptible d'indiquer une dépendance temporelle non capturée par le modèle. Cette évaluation engendre une statistique appelée "statistique de Durbin-Watson", qui se situe entre 0 et 4. Une valeur proche de 2 traduit l'absence d'autocorrélation, signifiant ainsi que les résidus équivalent au bruit et demeurent imprédictibles, ce qui est caractéristique d'un modèle adéquat. En revanche, des valeurs déviant considérablement de 2 pointent vers la présence d'autocorrélations positives ou négatives, révélant que le modèle n'a pas réussi à saisir toutes les informations contenues dans les données. Dans notre étude, nous avons soumis la série elle-même (celle présentée dans la figure 3) à ce test, et obtenu une valeur de 0.36. Cette valeur indique une forte autocorrélation au sein de la série, suggérant la présence d'un signal prédictible sous-jacent que le modèle n'a pas réussi à capter en raison du bruit intense qui prédomine. De façon similaire, nous avons effectué ce test sur un signal de bruit (généré selon une distribution normale) de la même taille que la série, aboutissant à une valeur de 1.99, très proche de 2. Cette proximité indique que le signal consiste en du bruit non prédictible.

Cependant, le choix de la fenêtre pour le filtre n'est pas une tâche facile. En effet, une fenêtre plus grande masque tout le bruit dans les données, mais peut entraîner une perte d'informations. Une fenêtre plus petite, en revanche, n'élimine pas efficacement les bruits s'ils existent. Pour résoudre ce problème, nous avons développé une méthode permettant de sélectionner automatiquement la taille optimale de la fenêtre pour le filtre. Une fenêtre optimale est celle qui produit un signal sans bruit tout en minimisant la perte d'informations.

Deux mesures statistiques sont utilisées dans ce contexte : la rugosité (roughness) et le kurtosis [42]. La rugosité fait référence à la variabilité ou aux fluctuations dans les données d'une série. Le kurtosis, quant à lui, est une mesure statistique qui décrit la forme de la distribution d'un ensemble de données. Dans le contexte du débruitage des séries chronologiques, préserver le kurtosis signifie conserver les propriétés de la distribution des données. Plus nous minimisons la rugosité, plus le signal est lisse, mais cela peut entraîner une perte d'informations. Ainsi, si nous nous concentrons uniquement sur la minimisation de la rugosité, la série résultante aura tendance à être constituée de lignes droites et ne représentera aucune information significative. Par conséquent, nous avons besoin d'une mesure de préservation qui évalue dans quelle mesure les informations à l'intérieur de la série chronologique sont préservées. C'est là que le kurtosis entre en jeu : si le kurtosis de la série originale est égal ou supérieur à celui de la série débruitée, cela signifie que la proportion d'informations dans la série débruitée n'est pas inférieure à celle de la série originale. Par conséquent, nous devons veiller à ce que le kurtosis de la série débruitée soit toujours inférieur à celui de la série originale. Ainsi, notre fenêtre optimale est celle qui offre la mesure de rugosité la plus faible tout en respectant la contrainte du kurtosis [42].

La démarche pour sélectionner la fenêtre optimale pour le débruitage est présentée dans l'algorithme 1.

Une version optimisée de l'algorithme a été présentée dans [42]. Cette optimisation vise principalement à simplifier le calcul du kurtosis et de la rugosité, tout en évitant la recherche séquentielle

Algorithm 1 Sélection de la fenêtre optimale pour le débruitage

```
function FENETREOPTIMALE(signal_original)
    kurtosis_original ← CalculerKurtosis(signal_original)
    fenetre_optimale ← 0
    meilleure_rugosite ← ∞
    taille_maximale_de_fenetre ← Longueur(signal_original)
    for taille_de_fenetre from 2 to taille_maximale_de_fenetre do
        signal_debruite ← AppliquerFiltrage(signal_original, taille_de_fenetre)
        kurtosis_debruite ← CalculerKurtosis(signal_debruite)
        rugosite ← CalculerRugosite(signal_debruite)
        if kurtosis_debruite < kurtosis_original AND rugosite < meilleure_rugosite then
            meilleure_rugosite ← rugosite
            fenetre_optimale ← taille_de_fenetre
        end if
    end for
    Retourner fenetre_optimale
end function
```

grâce à une approche de recherche binaire. Nous avons intégré cet algorithme optimisé en tant qu'alternative dans notre processus.

Une fois que nous avons choisi la meilleure fenêtre pour la série présentée dans la Figure 4.5 et appliqué le filtre, nous avons procédé à l'entraînement du modèle. Voici comment le modèle a réussi à capturer les informations qui étaient dissimulées dans la série originale, comme illustré dans la figure 4.

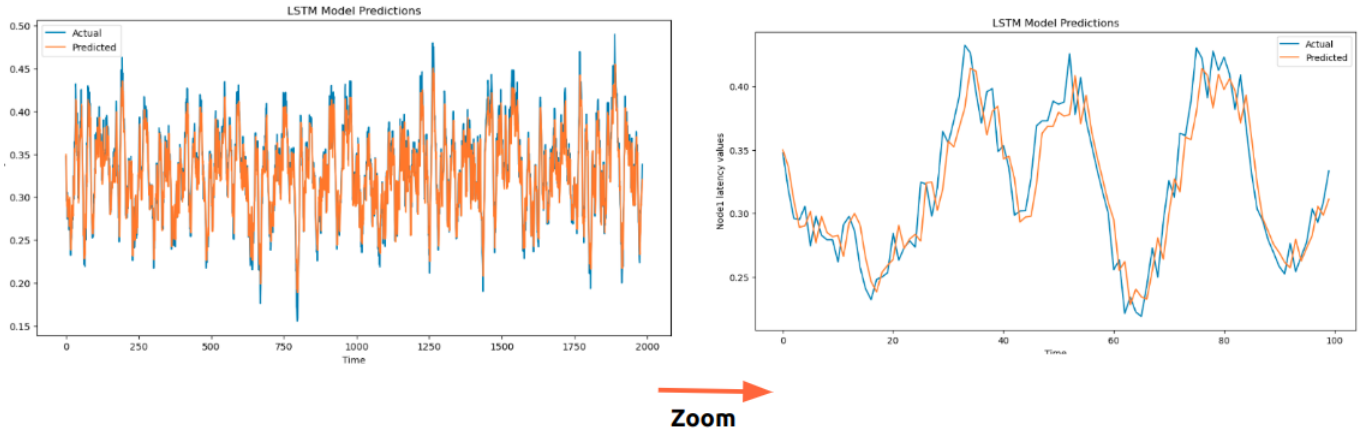


FIGURE 4.6 : Prédictions sur la série chronologique débruitée

Il convient également de noter que pour la série filtrée, nous avons obtenu une valeur de test de Durbin-Watson de 0, tandis que les résidus (représentant la différence entre la série originale et la série filtrée) affichent une valeur de 2.01, qui est très proche de 2. Cela indique que ces résidus ne sont que du bruit non prédictible. Par conséquent, on peut conclure que le filtre a réussi à éliminer complètement le bruit présent dans la série, tout en conservant les informations pertinentes.

Il est tout à fait naturel de se demander comment la taille de la fenêtre serait affectée si la série ne présentait pas de bruit ou si le bruit était si minime qu'il n'impactait pas la performance du modèle dans la capture des informations du signal. Cette situation reflète une expérience similaire que nous

avons menée avec une autre série, comportant de légers bruits sans altérer la capacité du modèle à saisir les motifs dans la série. Dans ce cas, une fenêtre de taille égale à 1 a été obtenue, indiquant ainsi que la série originale ne nécessite pas de débruitage. Dans le cas contraire, procéder au débruitage pourrait entraîner une perte d'informations significatives au sein de la série.

4.3.2.2 Détection et Imputation des valeurs manquantes.

En raison de diverses causes telles que les problèmes d'acquisition de données, les réseaux de transmission instables et les pannes de stockage, les données utilisées sont souvent incomplètes. Ces données manquantes peuvent fausser les estimations des paramètres, causer une perte d'informations essentielles, augmenter les erreurs standards et réduire la généralisabilité des résultats [11]. La présence de valeurs manquantes a un impact significatif sur les modèles testés dans l'expérience. Pour traiter les données manquantes dans les séries chronologiques, différentes méthodes sont couramment utilisées, notamment l'imputation par la moyenne, l'imputation linéaire et l'imputation multiple.

Dans notre cas, si une seule valeur de données est manquante, nous adopterons la méthode d'interpolation de la moyenne. L'implémentation de cette méthode est montrée dans la formule suivante :

$$x_{vm} = \frac{x_{t-1} + x_{t+1}}{2} \tag{4.1}$$

On suppose qu'il y a une valeur manquante x_{vm} au temps actuel t, et que x_{t-1} et x_{t+1} représentent les données complètes au temps précédent et au temps suivant la donnée manquante, respectivement.

En présence de plusieurs données consécutives manquantes dans la série chronologique, nous recourons à l'interpolation linéaire pour résoudre cette situation. Supposons que nous ayons n valeurs manquantes consécutives entre t et $t+(n+1)$, et que les valeurs à l'instant t et $t+(n+1)$ sont connus : x_t et $x_{t+(n+1)}$. L'interpolation linéaire pour estimer les valeurs manquantes x_{t+1} à x_{t+n} serait :

$$x_{t+i} = x_t + \frac{x_{t+(n+1)} - x_t}{n+1} \cdot i \tag{4.2}$$

où x_{t+i} est la valeur estimée à l'instant $t+i$, i varie de 1 à n, et n est le nombre total de valeurs manquantes entre t et $t+(n+1)$. Cette formule permet d'estimer les valeurs manquantes en traçant une ligne droite entre les deux valeurs connues à t et $t+(n+1)$, en utilisant la pente (différence entre $x_{t+(n+1)}$ et x_t) et en ajustant les positions relatives par rapport à x_t.

Si la série a des valeurs manquantes au début ou à la fin, nous les retirerons simplement. Les étapes complètes de détection et d'imputation des valeurs manquantes sont illustrées dans l'algorithme 2.

Algorithm 2 Détection et imputation des valeurs manquantes

```
1 : function IMPUTERVALEURSMANQUANTES(serie)
2 :     while estValeurManquante(serie[0]) do
3 :         retirerValeurManquante(serie[0])
4 :     end while
5 :     while estValeurManquante(serie[longueur(serie) − 1]) do
6 :         retirerValeurManquante(serie[longueur(serie) - 1])
7 :     end while
8 :     i ← 0
9 :     while i < longueur(serie) do
10 :        if estValeurManquante(serie[i]) then
11 :            nombre_valeurs_manquantes ← 1
12 :            j ← i + 1
13 :            while j < longueur(serie) et estValeurManquante(serie[j]) do
14 :                nombre_valeurs_manquantes ← nombre_valeurs_manquantes + 1
15 :                j ← j + 1
16 :            end while
17 :            valeur_debut ← serie[i − 1]
18 :            valeur_fin ← serie[j]
19 :            for k de 1 à nombre_valeurs_manquantes do
20 :                valeur_imputee ← valeur_debut + (valeur_fin−valeur_debut)/(nombre_valeurs_manquantes+1) · k
21 :                insérerValeurImputee(serie, i + k − 1, valeur_imputee)
22 :            end for
23 :            i ← j
24 :        end if
25 :        i ← i + 1
26 :    end while
27 :    Retourner serie
28 : end function
```

4.3.2.3 Séparation des données d'entraînement et de test.

Après avoir préparé les données, elles sont ensuite divisées en deux ensembles distincts : l'ensemble d'entraînement et l'ensemble de test. Cette division permet d'évaluer les modèles de manière objective.

L'ensemble d'entraînement est utilisé pour former les modèles sur les données disponibles. Les algorithmes d'apprentissage utilisent ces données pour apprendre à identifier les modèles et les relations au sein des données. Cela permet aux modèles de s'ajuster aux caractéristiques spécifiques des données et de mieux généraliser lorsqu'ils sont confrontés à de nouvelles observations.

D'autre part, l'ensemble de test est réservé pour évaluer les performances des modèles. Ces données n'ont pas été utilisées pendant la phase d'entraînement, ce qui permet d'évaluer la capacité des modèles à faire des prédictions précises sur des données qu'ils n'ont jamais vues auparavant. L'ensemble de test agit comme une simulation de la manière dont les modèles se comporteraient dans le monde réel lorsqu'ils sont confrontés à de nouvelles données.

Une approche courante est de diviser les données en environ 90% pour l'ensemble d'entraîne-

ment et 10% pour l'ensemble de test. Cette proportion peut varier en fonction de la taille du jeu de données. L'utilisation d'un ensemble de test distinct permet d'obtenir une évaluation impartiale des performances des modèles, garantissant ainsi leur validité et leur utilité dans des situations réelles.

4.3.2.4 Normalisation des données.

Avant d'alimenter les données dans les modèles, il est nécessaire de les normaliser. Cette étape vise à éliminer l'influence des dimensions des caractéristiques des données sur les modèles, afin d'accélérer la convergence des modèles et d'améliorer leur capacité de prédiction. Dans nos expérimentations, nous avons adopté la méthode de normalisation max-min :

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{4.3}$$

où x' représente la valeur normalisée, et x_{max} ainsi que x_{min} correspondent respectivement aux valeurs maximales et minimales dans les données d'entraînement. Les ensembles d'entraînement et de test subissent une normalisation en utilisant les valeurs minimales et maximales provenant des données d'entraînement. Cette approche garantit que les valeurs des données de test sont également alignées dans la même plage de valeurs. Cette méthode est en accord avec le processus d'entraînement des modèles et prévient toute introduction biaisée des données de test dans le processus de prédiction. Ainsi, les modèles ne sont pas exposés à des valeurs inattendues ou peu représentatives lors de l'évaluation sur les données de test, ce qui contribue à éviter toute distorsion des résultats et à préserver la qualité de la prédiction.

Cette normalisation garantit que les valeurs des caractéristiques se situent dans une plage commune, ce qui facilite la convergence des modèles et améliore leur performance prédictive. En conséquence, les modèles sont en mesure de mieux saisir les relations et les motifs au sein des données normalisées, ce qui se traduit par des prédictions plus précises sur de nouvelles données.

Ainsi, nous avons achevé la première étape essentielle de notre méthodologie, à savoir le prétraitement des données. Grâce à cela, nous disposons désormais de deux ensembles de données parfaitement préparés : les données d'entraînement, destinées à la construction de nos modèles, et les données de test, conçues spécifiquement pour l'évaluation de ces modèles.

4.3.3 Construction et entraînement des modèles

Dans cette étape, nous créons des modèles de prédiction pour capturer les tendances et les motifs des séries chronologiques :

4.3.3.1 Séparation des données d'entraînement et de validation.

Au cours de cette étape, les données d'entraînement sont subdivisées en deux ensembles distincts : les données d'entraînement proprement dites et les données de validation. Pour affiner les paramètres des modèles de manière optimale, une partie des données d'entraînement est réservée à l'ensemble de validation. Cet ensemble de validation est utilisé pour ajuster minutieusement les hyperparamètres des modèles, ce qui permet de rechercher les configurations optimales pour obtenir les meilleures performances.

4.3.3.2 Modèles à évaluer.

Nous avons choisi soigneusement 6 modèles distincts pour constituer notre ensemble de comparaison. Ces choix reflètent une approche méthodique pour couvrir une large gamme de techniques et d'approches de modélisation.

Le premier modèle que nous avons inclus est le modèle *Persistence*, qui sert de point de référence fondamental. Ce modèle adopte une approche simple mais cruciale : il prédit la prochaine valeur de la série comme étant égale à la valeur actuelle. Bien que cette méthode puisse sembler rudimentaire, elle établit une base solide à partir de laquelle les autres modèles seront évalués.

Nous avons également incorporé le modèle *ARIMA*, une figure de proue dans le domaine de la prédiction de séries chronologiques. Les propriétés statistiques et la prise en compte des tendances et des saisons font de *ARIMA* un choix incontournable pour notre analyse comparative.

Le modèle *LSTM* s'intègre également à notre ensemble, démontrant des performances prometteuses lorsque des volumes importants de données sont en jeu. Sa capacité à capturer des motifs complexes dans les séquences temporelles en fait un concurrent sérieux. De plus, LSTM est utilisé comme une référence d'efficacité pour l'évaluation des performances de prédiction, car il est largement reconnu que lorsque la quantité de données est suffisante, les LSTM ont tendance à surpasser toute autre approche en matière de prédiction de séries chronologiques [10].

Nous avons également introduit le modèle *GRU*, une version simplifiée et plus légère du LSTM. Sa structure allégée lui confère une vitesse d'entraînement supérieure par rapport à d'autres modèles plus complexes comme les LSTM. Notre choix d'inclure le modèle GRU vise à établir s'il peut maintenir un équilibre entre efficacité et coût de prédiction sur nos ensembles de données.

En ce qui concerne le modèle *RNN*, c'est le modèle de base des deux modèles précédemment mentionnés. Son point fort réside dans sa capacité à capturer les dépendances temporelles dans les séries chronologiques. Cependant, il peut éprouver des difficultés liées à la disparition du gradient, ce qui limite son efficacité pour les dépendances à long terme. Nous l'avons inclus dans notre comparaison pour évaluer si un modèle apte à saisir des dépendances à long terme s'avère nécessaire pour les données QoS.

Finalement, nous portons une attention particulière à l'approche novatrice *ESN*. Sa caractéristique marquante réside dans sa rapidité d'entraînement et d'inférence, attribuée à sa structure simplifiée qui évite les calculs complexes. Dans certains cas, l'ESN s'avère même plus rapide que des modèles structurés tels que ARIMA. Cette efficacité en fait un choix attrayant pour les applications en temps réel ou impliquant des volumes considérables de données. Cependant, il est essentiel de souligner que la performance de l'ESN dépend étroitement de la nature des données. Ainsi, dans notre étude, nous l'avons sélectionné pour déterminer s'il peut prédire avec précision les données QoS et tirer parti de sa simplicité et de sa rapidité.

Le tableau 4.1 résume les motivations derrière le choix de chaque modèle dans notre étude comparative.

4.3.3.3 Réglage des hyperparamètres basé sur les données de validation.

Après avoir sélectionné les modèles, nous passons à l'étape cruciale du réglage des hyperparamètres. Cette étape vise à ajuster les paramètres internes des modèles pour optimiser leurs performances sur les données de validation. Cependant, le modèle de référence *Persistence* ne nécessite ni entraînement ni réglage de paramètres spécifiques, car il repose sur une approche simple de prédiction basée sur la valeur actuelle.

Modèle	**Objectif de Sélection**
Persistence	Établir une référence fondamentale pour l'évaluation des modèles.
ARIMA	Incorporer une méthode de référence largement utilisée.
LSTM	Tester les performances avec des quantités importantes de données et évaluer la capacité à capturer des motifs complexes dans les données de QoS.
GRU	Évaluer la balance entre efficacité et coût de prédiction.
RNN	Examiner la capture des dépendances temporelles et à long terme.
ESN	Explorer une approche rapide et novatrice, en dépit de la nature des données.

TABLE 4.1 : Modèles sélectionnés et objectifs

Pour les autres modèles, le processus de réglage des hyperparamètres commence par le choix d'une combinaison initiale d'hyperparamètres, suivie de l'entraînement du modèle sur l'ensemble d'entraînement. Ensuite, le modèle est évalué sur l'ensemble de validation, qui n'a pas été utilisé pendant l'entraînement. Cette évaluation mesure les performances du modèle en utilisant la combinaison d'hyperparamètres actuelle.

L'objectif est de trouver la combinaison optimale d'hyperparamètres qui minimise l'erreur (RMSE) sur les données de validation. Pour ce réglage des hyperparamètres, nous utilisons la bibliothèque *Optuna*, un framework d'optimisation d'hyperparamètres basé sur *Python*. *Optuna* automatise la recherche des meilleurs hyperparamètres en utilisant des méthodes d'optimisation séquentielle pour explorer l'espace des hyperparamètres et identifier ceux qui minimisent l'erreur de validation.

Pour les modèles non structurés basés sur les réseaux de neurones, à savoir LSTM, RNN, GRU et ESN, une étape additionnelle s'impose pour transformer les données de séries chronologiques en un format tabulaire. Cette conversion permet de rendre le problème compatible avec une approche supervisée. Ces modèles requièrent que les données soient organisées en vecteurs d'entrée (X) et en vecteurs de sortie (y), comme expliqué en détail dans la section 2.3.3. Pour effectuer cette transformation, nous faisons appel à l'algorithme 3.

4.3.3.4 Entraînement et génération de modèles formés.

Une fois les hyperparamètres optimisés pour chaque modèle, la phase suivante implique de ré-entraîner les modèles en utilisant l'ensemble d'entraînement complet, y compris les données de validation. Cette approche permet aux modèles de bénéficier de l'ensemble le plus complet de données d'apprentissage disponibles.

En incorporant les données de validation dans l'ensemble d'entraînement, les modèles finaux sont en mesure de capturer une gamme plus large de motifs, de structures et de caractéristiques présentes dans les données. Cette stratégie favorise une meilleure généralisation et permet aux modèles de mieux s'adapter aux diverses variations et complexités des données de QoS.

Une fois ce processus de ré-entraînement achevé, les modèles finaux sont prêts à être soumis à l'évaluation sur les données de test indépendantes. Ces modèles optimisés et finement ajustés sont en mesure de fournir des prédictions plus précises lorsqu'ils sont confrontés à de nouvelles données.

Algorithm 3 Création d'un ensemble de données tabulaire

```
1 : function CreerDonneesTabulaire(data, fenetre_temporelle, horizon_de_prédiction)
2 :     X ← ListeVide()
3 :     y ← ListeVide()
4 :     indice_start ← 0
5 :     for étape de 0 à longueur(data) − 1 do
6 :         indice_end ← indice_start + fenetre_temporelle
7 :         out_end ← indice_end + horizon_de_prédiction
8 :         if out_end ≤ longueur(data) then
9 :             x_input ← data[indice_start : indice_end]
10 :            ajouter x_input à X
11 :            ajouter data[indice_end : out_end] à y
12 :        end if
13 :        indice_start ← indice_start + 1
14 :    end for
15 :    Retourner X, y
16 : end function
```

4.3.4 Évaluation des modèles

Une fois que nous disposons de modèles entraînés et prêts à être utilisés, nous entamons la dernière phase du processus : l'évaluation de ces modèles. L'objectif de cette phase est de déterminer comment chaque modèle se comporte dans la tâche de prédiction en analysant ses performances.

Pour ce faire, les modèles finalement formés sont mis à l'épreuve en utilisant les données de test. Chaque modèle génère des prédictions pour les valeurs de QoS inconnues dans l'ensemble de test. Ensuite, une étude comparative approfondie est menée en mesurant plusieurs métriques clés :

4.3.4.1 RMSE et MAE.

Nous avons choisi le RMSE (Root Mean Squared Error) comme métrique d'évaluation principale en raison de sa capacité à mesurer l'écart entre les valeurs prédites et les valeurs réelles tout en prenant en compte magnitude de ces écarts. Le RMSE exprime ces écarts dans les mêmes unités que les données originales, facilitant leur interprétation intuitive. En outre, le RMSE peut être comparé directement à l'écart type, fournissant ainsi une référence pertinente pour évaluer la performance du modèle.

Le MAE (Mean Absolute Error) a également été sélectionné pour évaluer la précision des modèles, car il fournit une mesure directe de l'écart moyen entre les prédictions et les valeurs réelles. Cette métrique est facilement interprétable, car elle représente simplement la moyenne des erreurs absolues.

En combinaison avec le RMSE, le MAE offre une vue complète et précise de la précision de chaque modèle de prédiction, avec le RMSE mettant l'accent sur l'amplitude des erreurs et le MAE fournissant une vue simple de l'erreur moyenne.

Cependant, le calcul du RMSE et du MAE dans le cas où l'on prédit une valeur future semble trivial. L'erreur ponctuelle est la différence entre la valeur prédite et la valeur réelle. En revanche, calculer le RMSE et le MAE dans le cadre d'une prédiction à plusieurs pas en avant n'est pas aussi trivial que dans le cas de la prédiction à un seul pas en avant. Cela nécessite de prendre en compte plusieurs prédictions à différents moments dans le futur.

L'algorithme que nous utilisons pour calculer le RMSE et le MAE dans ce contexte consiste à

générer des prédictions pour un horizon donné, puis à les comparer aux valeurs réelles pour cet horizon. Ensuite, nous décalons notre fenêtre de comparaison d'un pas en avant dans les données de test et répétons le processus jusqu'à ce que nous ayons parcouru l'ensemble de test en entier. En fin de compte, nous calculons la moyenne des erreurs absolues (MAE) et la racine carrée de la moyenne des erreurs au carré (RMSE) sur l'ensemble des prédictions pour obtenir des mesures globales de la performance du modèle.

Cet algorithme est illustré plus en détail dans l'Algorithme 4. Il nous permet de quantifier la précision de nos modèles de prédiction en prenant en compte l'horizon de prédiction et en évaluant leur performance sur l'ensemble de test, ce qui est essentiel pour comprendre comment nos modèles se comportent dans des situations de prédiction à plus long terme.

Algorithm 4 Évaluation des précisions des prédictions à plusieurs pas en avant

```
1 : function CALCULERMAEETRMSE(vraiesValeurs[1..N], model, horizon)
2 :     sommeRMSE ← 0, sommeMAE ← 0          ▷ Initialiser les variables pour le RMSE et le MAE
3 :     for i ← 1 to N − horizon do
4 :         predictionsEnAvant[1..horizon] ← model.predict(vraiesValeurs[1 : i], horizon)
5 :         vraiesValeursEnAvant[1..horizon] ← vraiesValeurs[i : i + horizon]
6 :         for j ← 1 to horizon do
7 :             erreurAbsolue ← |vraiesValeursEnAvant[j] − predictionsEnAvant[j]|
8 :             erreurCarrée ← (vraiesValeursEnAvant[j] − predictionsEnAvant[j])^2
9 :             sommeMAE ← sommeMAE + erreurAbsolue
10 :            sommeRMSE ← sommeRMSE + erreurCarrée
11 :        end for
12 :    end for
13 :    MAE ← sommeMAE/(N − horizon)
14 :    RMSE ← √(sommeRMSE/(N − horizon))
15 :    Retourner MAE, RMSE
16 : end function
```

4.3.4.2 Temps de Génération des Modèles.

Le temps nécessaire pour former chaque modèle peut avoir un impact significatif sur l'efficacité globale du processus de prédiction, surtout dans des environnements où les données peuvent changer de caractéristiques avec le temps, ce qui est fréquent dans des scénarios réels. De plus, la performance d'un modèle peut se dégrader au fil du temps, ce qui nécessite un éventuel ré-entraînement périodique. Dans des applications en temps réel, une longue période de génération peut ne pas être pratique, car elle pourrait entraîner des retards indésirables dans la prise de décision. Il est donc crucial de prendre en considération le temps de génération des modèles.

4.3.4.3 Temps de Génération de Prédictions.

Cette mesure évalue la rapidité avec laquelle chaque modèle peut générer des prédictions pour l'ensemble de test. Il est important de tenir compte du temps de traitement, surtout lorsque des quantités importantes de données sont prises en compte. Cela se révèle particulièrement essentiel dans notre cas d'application, en l'occurrence les véhicules autonomes, où des prédictions rapides sont indispensables.

L'analyse globale de ces métriques nous permet d'établir les performances relatives de chaque modèle dans la prédiction de la QoS. En suivant cette méthodologie rigoureuse, nous sommes en

mesure de proposer des évaluations exhaustives des capacités de prédiction des différents modèles, en tenant compte de la précision ainsi que des temps de traitement. Ces évaluations nous permettront de réaliser une étude comparative approfondie pour évaluer et comparer l'efficacité de chaque modèle dans la prédiction de la QoS.

4.3.5 Génération de l'API

Notre étude vise à développer et évaluer des modèles de prédiction de séries chronologiques de QoS. Afin de rendre notre processus de développement et d'évaluation accessible et fonctionnel au sein du système global, nous avons créé une API (Interface de Programmation d'Application). Cette API permet d'exécuter l'ensemble des étapes et des modèles de notre processus via le protocole HTTP.

4.3.5.1 Création de l'API

L'API offre la possibilité au système global, ainsi qu'à d'autres systèmes externes, d'accomplir plusieurs actions essentielles. Elle peut s'intégrer de manière transparente dans un écosystème plus vaste, permettant ainsi une collaboration fluide avec d'autres composants. Elle permet tout d'abord le chargement des données initiales d'entraînement et la sélection des variables cibles pour les prédictions. De plus, elle autorise la définition de l'horizon de prédiction, la sélection de l'une des cinq stratégies de prédiction à plusieurs pas en avant lorsque l'horizon est supérieur à 1, ainsi que le choix du modèle de prédiction. L'optimisation des hyperparamètres est également possible, sinon l'utilisation des paramètres par défaut est une option.

Après la configuration initiale, l'API permet de lancer l'entraînement du modèle sélectionné sur les données téléchargées. Ensuite, elle offre la possibilité de générer des prédictions en temps réel pour les variables ciblées. De plus, à tout moment, le système peut consulter la précision de prédiction du modèle en temps réel, mesurée par le RMSE et le MAE. Cette fonctionnalité permet de déterminer si une révision du modèle est nécessaire en cas de dégradation des performances.

L'architecture globale de cette API est illustrée dans la Figure 4.7.

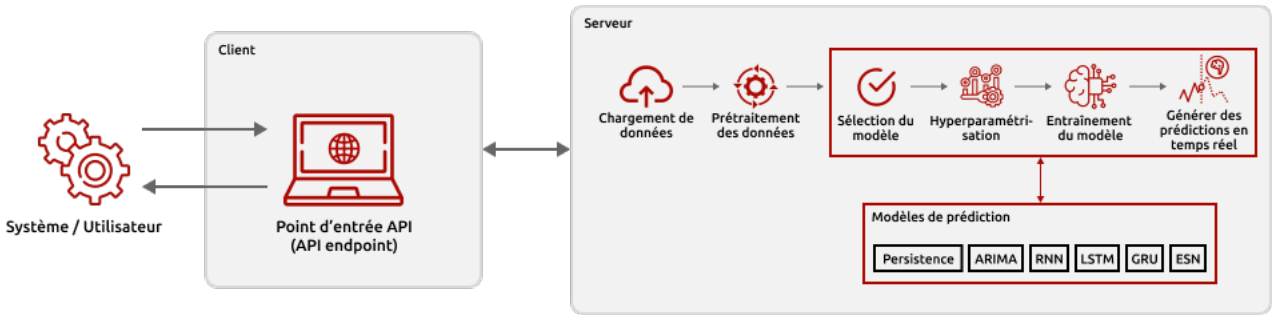


FIGURE 4.7 : Architecture globale de l'API

4.3.5.2 Définition des points d'entrée API

Notre API offre une gamme complète de fonctionnalités pour personnaliser et exécuter des prédictions de QoS. Pour rendre ce processus transparent et accessible, nous avons défini plusieurs points d'entrée qui vous permettent de configurer, entraîner et utiliser les modèles de prédiction de manière flexible. Le tableau 4.2 présente l'ensemble des points d'entrées.

Tâche	**Endpoint**	**Méthode HTTP**	**Description**
Charger les données	`/`	POST	Permet le chargement des données initiales d'entraînement.
Sélectionner les colonnes	`/columns`	POST	Permet de spécifier les colonnes ou variables cibles pour les prédictions.
Sélectionner l'horizon	`/horizon`	POST	Permet de déterminer l'horizon de prédiction souhaité.
Sélectionner la stratégie	`/strategy`	POST	Permet de choisir parmi les 5 stratégies de prédiction à plusieurs pas en avant lorsque l'horizon est supérieur à 1.
Sélectionner le modèle	`/model`	POST	Permet de choisir le modèle de prédiction souhaité parmi plusieurs options disponibles.
Optimisation des hyperparamètres	`/hypopt`	GET	Offre la possibilité d'optimiser les hyperparamètres du modèle sélectionné.
Entraîner le modèle	`/train`	GET	Permet de lancer le processus d'entraînement du modèle sélectionné sur les données d'entraînement.
Générer les prédictions	`/predict`	GET	Permet de générer des prédictions en temps réel en fournissant des valeurs actuelles. Le modèle renvoie des prédictions de QoS en temps réel. Si "stop" est envoyé, il retourne également la précision actuelle du modèle (RMSE et MAE).

TABLE 4.2 : Points d'entrée de l'API

Chacun de ces points d'entrée API revêt une importance cruciale en offrant au système la flexibilité nécessaire pour personnaliser les prédictions en fonction des besoins spécifiques et du contexte de la tâche de prédiction.

La Figure 4.8 présente un diagramme de séquence illustrant l'utilisation de notre API. Ce schéma offre un aperçu visuel du processus d'interaction entre le système et l'API.

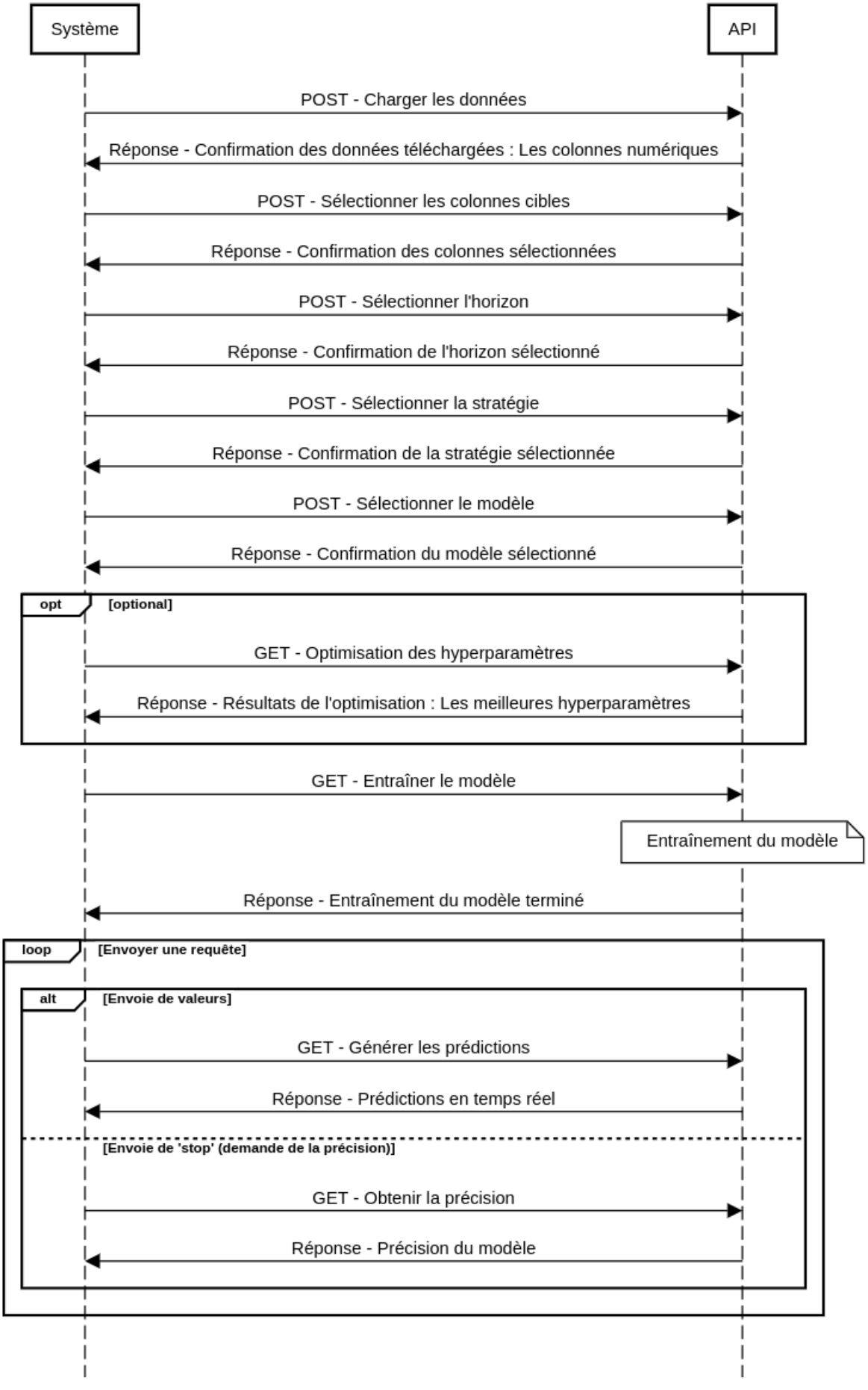


FIGURE 4.8 : Diagramme de séquence des interactions entre le système et l'API

4.4 Conclusion

Ce chapitre revêt une importance capitale dans notre démarche de développement et d'évaluation des modèles de prédiction de la QoS. Notre objectif principal demeure la création de modèles précis capables de fournir des prédictions en temps réel pour la QoS, même en tenant compte de la complexité inhérente des données de QoS.

Nous avons exploré en profondeur le processus de construction de ces modèles de prédiction, en mettant en évidence l'importance critique de chaque étape, de la collecte de données pertinente au prétraitement pour garantir la qualité des données d'entrée, en passant par la construction de modèles précis capables d'apprendre efficacement à partir de ces données, tout en tenant compte de la dimension temporelle des prédictions et de leur précision dans les évaluations.

Dans le cadre de cette exploration, nous avons également présenté en détail l'API que nous avons développée pour rendre notre processus accessible et fonctionnel. Cette API offre une interface flexible pour configurer, entraîner et utiliser les modèles de prédiction, ce qui la rend essentielle non seulement pour le système global, mais aussi pour d'autres systèmes externes.

Dans le prochain chapitre, nous mettrons en œuvre et testerons toutes les étapes détaillées dans ce chapitre.

Chapitre 5

Implémentation et tests

5.1 Introduction

Ce chapitre se penche sur l'aspect pratique de notre étude, en mettant en lumière l'implémentation et les résultats de l'évaluation des modèles sélectionnés pour la prédiction de la QoS. Après avoir minutieusement défini le processus de construction de modèles dans le chapitre précédent, nous sommes à présent prêts à examiner comment ces modèles se comportent dans un contexte réel de prédiction. Notre objectif est d'apporter une compréhension profonde des performances de chaque modèle et de déterminer leur pertinence dans le domaine des séries chronologiques de QoS, en particulier dans le contexte des véhicules autonomes. Nous détaillerons l'approche d'implémentation adoptée pour chaque modèle, en expliquant les étapes clés et les choix techniques effectués pour garantir une mise en œuvre précise et cohérente. Nous examinerons également les résultats obtenus à travers une évaluation complète de ces modèles.

Nos expérimentations se divisent en 3 séries d'expérimentations distinctes, chacune visant à explorer différents aspects de notre étude. Précisément, nous abordons les prédictions univariées à un pas en avant, les prédictions univariées à N pas en avant et les prédictions multivariées. Cette segmentation nous permettra de fournir une vue exhaustive des performances des modèles sous diverses conditions.

5.2 Configuration des expérimentations

Les expérimentations ont été réalisées en utilisant le langage de programmation *Python* [76], en conjonction avec la bibliothèque *Keras* [72]. Elles ont été exécutées sur la plateforme *Google Colab* [71] à travers l'environnement de développement *Jupyter Notebook*. Le choix de la bibliothèque *Keras* a été motivé par sa modularité, qui offre la possibilité de décomposer notre processus expérimental en éléments distincts et interconnectés, facilitant ainsi la gestion et la personnalisation de chaque composant. De plus, l'interface utilisateur conviviale de Keras simplifie considérablement la création, la configuration et la formation des modèles de réseaux neuronaux, réduisant ainsi la complexité technique et permettant une plus grande concentration sur les aspects conceptuels de notre étude. Il est important de noter également que Keras est une surcouche de haut niveau pour les réseaux

neuronaux, construite sur la base de *TensorFlow* [79], un puissant framework d'apprentissage automatique développé par Google, qui offre une grande flexibilité et des performances élevées pour la construction et l'entraînement de modèles de réseaux neuronaux.

5.3 Jeu de données

Pour évaluer nos modèles de manière robuste sur une variété de données, nous avons employé un jeu de données recueilli par un prototype mettant en œuvre l'architecture envisagée. Cette démarche visait à assurer que les caractéristiques du jeu de données correspondent au mieux aux situations réelles, confirmant ainsi sa pertinence pour nos tests. Une description approfondie de ce jeu de données a été détaillée dans la section 3.6.1 du chapitre 3.

Ce jeu de données est stocké au format *CSV (Comma-Separated Values)* et a une taille de *1.5 MiB (Mebibytes)*. Cette représentation au format CSV offre une flexibilité considérable pour stocker des données tabulaires, ce qui est essentiel pour notre analyse. En outre, le jeu de données est facilement partageable et interopérable avec un large éventail d'outils d'analyse de données.

Pour importer et manipuler efficacement ce jeu de données, nous avons utilisé la bibliothèque *pandas* [75] de Python. Pandas offre une gamme complète de fonctionnalités pour la gestion de données tabulaires, notamment l'importation de fichiers CSV, la manipulation des données, la transformation, la fusion et bien d'autres opérations. Cette bibliothèque s'est avérée essentielle pour prétraiter nos données, les explorer, et les adapter aux besoins de nos modèles de prédiction.

En utilisant pandas, nous avons pu charger rapidement le jeu de données, examiner ses premières lignes pour comprendre sa structure, effectuer des opérations de nettoyage des données, et préparer les données d'entraînement et de test pour nos modèles. L'utilisation de cette bibliothèque a grandement simplifié le processus de gestion de données, ce qui nous a permis de nous concentrer davantage sur le développement et l'évaluation de nos modèles de prédiction.

5.4 Hyperparamétrisation

Avant d'entraîner un modèle quelconque, la première étape consiste à déterminer les meilleurs hyperparamètres à appliquer au modèle lors de la phase d'apprentissage. Les hyperparamètres varient d'un modèle à l'autre. Par exemple, pour un modèle LSTM, GRU ou RNN, il s'agit du nombre de neurones, du nombre de couches cachées, du nombre d'itérations (epochs), du taux d'apprentissage (learning rate) et de la taille du lot (batch size). Dans le cas d'un modèle ARIMA, les hyperparamètres consistent principalement en l'ordre du modèle, à savoir p, d et q. Cependant, pour un modèle ESN, les hyperparamètres incluent le taux de fuite (Leaking Rate), la taille du réservoir (Reservoir Size), le bruit et le rayon spectral. Un hyperparamètre commun dans la prédiction des séries chronologiques est la fenêtre temporelle (look back), qui correspond au nombre d'observations passées que le modèle doit prendre en compte pour générer des prédictions futures. Le tableau 5.1 présente les hyperparamètres de chaque modèle avec leurs explications.

Nous avons utilisé la bibliothèque Optuna pour sélectionner la meilleure combinaison d'hyperparamètres pour chaque modèle. Optuna est une bibliothèque qui permet l'optimisation automatique des hyperparamètres des modèles d'apprentissage automatique. Elle permet d'identifier facilement

Modèles	Hyperparamètres	Explication
LSTM, GRU, RNN	Nombre de neurones	Le nombre des unités dans chaque couche cachée, il détermine la capacité du modèle à capturer des dépendances complexes.
	Nombre de couches cachées	Permet d'ajouter de la profondeur au modèle.
	Nombre d'itérations	Contrôle combien de fois le modèle parcourt l'ensemble de données d'entraînement.
	Taux d'apprentissage	Détermine la taille des pas lors de la mise à jour des poids.
	Taille du lot (batch size)	La taille du lot spécifie combien d'exemples d'entraînement sont utilisés en même temps pour la mise à jour des poids.
ARIMA	Ordre du modèle (p, d, q)	p, d et q sont les ordres de l'autorégression, de la différenciation et de la moyenne mobile respectivement.
ESN	Taux de fuite (Leaking Rate)	Contrôle la mémoire de court terme du modèle.
	Taille du réservoir (Reservoir Size)	Le nombre des unités (ou neurones) dans le réseau.
	Bruit	Ajoute une perturbation aléatoire au modèle.
	Rayon spectral (Spectral Radius)	Le rayon spectral influe sur la dynamique du réservoir.
Tous les modèles	"Look Back"	Spécifie combien d'observations passées le modèle doit prendre en compte.

TABLE 5.1 : Hyperparamètres pour chaque modèle avec leur explication.

les hyperparamètres optimaux en effectuant plusieurs tests avec différentes combinaisons d'hyperparamètres. Optuna utilise un historique des essais pour déterminer quelles valeurs d'hyperparamètres essayer ensuite, facilitant ainsi la recherche de la meilleure combinaison d'hyperparamètres. Cependant, Optuna nécessite trois éléments clés : la plage de valeurs pour chaque hyperparamètre, l'objectif à minimiser ou à maximiser (la fonction objectif), et le nombre d'essais à effectuer.

Après avoir effectué plusieurs essais, nous avons constaté que 100 essais étaient un nombre suffisant pour trouver une excellente combinaison d'hyperparamètres pour les modèles. Nous avons choisi de minimiser la valeur du RMSE comme fonction objectif. Les plages de valeurs que nous avons sélectionnées pour chaque hyperparamètre sont illustrées dans le tableau 5.2.

L'optimisation des hyperparamètres est une tâche très coûteuse en temps, qui peut prendre des heures, en particulier pour des modèles plus complexes tels que les réseaux de neurones LSTM, GRU et RNN. Dans notre cas, pour chaque caractéristique et à chaque série d'expérimentations, nous répétons le processus d'optimisation des hyperparamètres de nos modèles afin d'obtenir les performances optimales, ou du moins les approximer, pour chaque modèle, ce qui nous permet de réaliser des comparaisons constructives. Le tableau 5.3 résume la durée moyenne nécessaire pour l'optimisation des hyperparamètres de chacun des modèles que nous avons sélectionnés au cours de nos séries d'expérimentations.

Il est évident que les modèles RNN, GRU et LSTM sont les plus gourmands en temps lorsqu'il s'agit de l'optimisation de leurs hyperparamètres, et ce, dans cet ordre. Cette observation découle du grand nombre de paramètres que ces modèles cherchent à optimiser, en plus de la plage de valeurs dans laquelle ces paramètres évoluent. Plus précisément, RNN et GRU requièrent plus de temps que LSTM pour l'optimisation de leurs hyperparamètres. Cette différence peut s'expliquer par la

Modèle	Hyperparamètre	Plage des valeurs
LSTM, GRU, RNN	Nombre de neurones	[8-256]
	Nombre de couches cachées	[1-10]
	Nombre d'itérations	[10-200]
	Taux d'apprentissage	$[10^{-5}$-$10^{-1}]$
	Taille du lot (batch size)	[5-300]
ARIMA	Ordre du modèle (p, d, q)	[0-3]
ESN	Taux de fuite (Leaking Rate)	[0.01, 0.1, 0.2, 0.3, 0.4, 0.5]
	Taille du réservoir (Reservoir Size)	[10-1000]
	Bruit	[0.001, 0.002, 0.003, 0.004, 0.005, 0.006, 0.007, 0.008, 0.009]
	Rayon spectral (Spectral Radius)	[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1]
Tous les modèles	"Look Back"	[5-300]

TABLE 5.2 : Plage des valeurs pour chaque hyperparamètre.

Modèle	Durée moyenne d'optimisation des hyperparamètres
LSTM	9824.7 secondes $\approx$ **2.7 heures**
GRU	17599.4 secondes $\approx$ **4.9 heures**
RNN	18805.5 secondes $\approx$ **5.2 heures**
ESN	642.5 secondes $\approx$ **10.7 minutes**
ARIMA	2221.7 secondes $\approx$ **37 minutes**

TABLE 5.3 : Durée moyenne d'optimisation des hyperparamètres des modèles.

nature intrinsèquement plus complexe de LSTM par rapport à RNN et GRU. En général, lors de la recherche des meilleures valeurs pour la taille du lot (batch size), nous observons que la plage de valeurs optimale est plus grande pour LSTM. Cela s'explique par la capacité de LSTM à traiter un grand volume de données en une seule itération, lui permettant ainsi de parcourir rapidement l'ensemble des données d'entraînement.

Cette différence est clairement illustrée dans la figure 5.1, qui représente une étape du processus d'optimisation des hyperparamètres avec LSTM (en haut), GRU (au milieu) et RNN (en bas). On peut clairement constater la disparité entre les plages de valeurs de la taille du lot, avec environ **115** pour LSTM, tandis que pour GRU et RNN, elles se situent respectivement autour de **39** et **30**. Il est important de noter que la taille du lot a un impact significatif sur le temps nécessaire à l'entraînement d'un modèle, ainsi que sur la durée globale du processus d'optimisation des hyperparamètres. En effet, la taille du lot détermine la vitesse à laquelle le modèle peut parcourir l'ensemble des données, ainsi que la fréquence avec laquelle il met à jour ses paramètres.

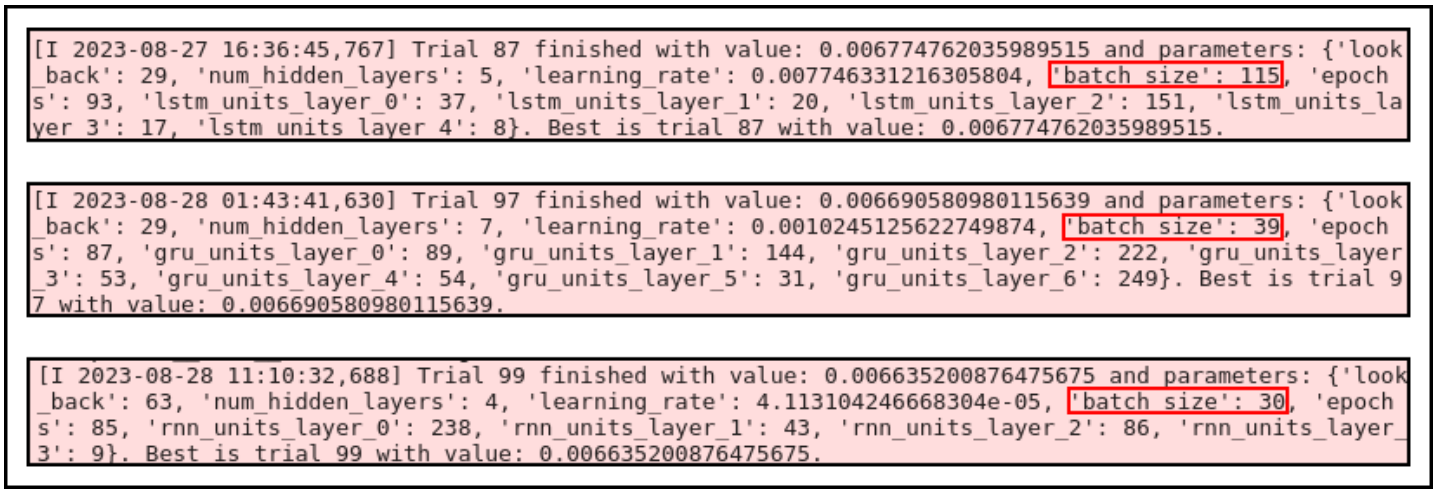


FIGURE 5.1 : Taille de lot (Batch Size) lors de l'optimisation des hyperparamètres pour LSTM (en haut), GRU (au milieu) et RNN (en bas)

En revanche, le modèle ESN se distingue par sa rapidité grâce à sa structure qui ne met à jour que la couche de sortie. Bien qu'il ait un nombre significatif de paramètres à optimiser et une large plage de valeurs à explorer, il ne prend en moyenne que **10 minutes** pour trouver une combinaison d'hyperparamètres permettant de bien ajuster les données. De son côté, ARIMA est également relativement rapide, ne nécessitant que **30 minutes** pour déterminer les valeurs appropriées de ses paramètres p, d et q.

Dans les sections suivantes, nous approfondirons nos expérimentations en fournissant une vue d'ensemble détaillée de la mise en œuvre des différents modèles et stratégies de prédiction sur nos données. Notre objectif est de trouver un modèle qui réponde aux exigences spécifiques de notre cas d'utilisation des véhicules autonomes : offrir une grande précision tout en maintenant des coûts raisonnables en termes de temps de prédiction et d'entraînement.

5.5 Prédictions univariées à un pas en avant

Dans cette première série d'expérimentations, nous nous intéressons à la prédiction d'un pas en avant de chaque série chronologique individuellement en développant chacun des modèles sélectionnés. Ensuite, nous enregistrons le temps total d'entraînement et le temps total de génération des prédictions sur l'ensemble de test pour tous les modèles. Ces deux facteurs revêtent une grande importance pour les comparaisons ultérieures. Enfin, nous calculons les erreurs RMSE et MAE pour chaque modèle afin de comparer leur précision. Les résultats de ces expérimentations sont présentés dans les tableaux 5.4 à 5.12, chaque tableau correspondant à une série chronologique donnée.

5.5.1 Précision des prédictions

Le modèle de persistance forme la base de notre étude, et les autres modèles sont soumis à la tâche de surpasser sa précision pour être considérés compétents. Dans l'ensemble des séries chronologiques, tous les autres modèles surpassent de manière cohérente le modèle de persistance, ce qui indique que nos modèles de prédiction apprennent effectivement à partir des données et se révèlent performants.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.02211	0.01808	-	0.00011
ARIMA	0.02171	0.01775	295.24045 (0.14762)	6.07866
LSTM	0.01885	0.01543	275.31462	0.94996
GRU	0.01944	0.01591	205.47089	0.84545
RNN	0.02049	0.01668	190.75704	0.36809
ESN	**0.01583**	**0.01362**	2.83301	0.14184

TABLE 5.4 : Prédictions univariées à un pas en avant de la série node1_delay du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.00855	0.00698	-	0.00009
ARIMA	0.00831	0.00677	331.13528 (0.16557)	6.20659
LSTM	0.00689	0.00575	169.30349	0.80933
GRU	0.00696	0.00582	156.79970	0.69090
RNN	0.00693	0.00577	206.74629	0.65463
ESN	**0.00604**	**0.00519**	2.20503	0.11188

TABLE 5.5 : Prédictions univariées à un pas en avant de la série node2_delay du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.01226	0.01005	-	0.00007
ARIMA	0.01201	0.00986	315.98457 (0.15799)	5.86463
LSTM	0.01032	0.00860	206.01040	0.92989
GRU	0.01051	0.00870	156.55630	0.86422
RNN	0.01108	0.00909	197.93322	0.34672
ESN	**0.00889**	**0.00764**	1.99874	0.13935

TABLE 5.6 : Prédictions univariées à un pas en avant de la série node3_delay du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.02142	0.01742	-	0.00011
ARIMA	0.02096	0.01731	289.59900 (0.14480)	5.82631
LSTM	0.01833	0.01515	95.26364	0.91648
GRU	0.01858	0.01526	80.54434	0.87464
RNN	0.01880	0.01544	104.34716	0.38619
ESN	**0.01536**	**0.01306**	2.04051	0.12909

TABLE 5.7 : Prédictions univariées à un pas en avant de la série node1_ram du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.03616	0.02877	-	0.00014
ARIMA	0.03479	0.02829	285.87563 (0.14294)	5.97975
LSTM	0.02964	0.02475	207.64390	0.90170
GRU	0.02925	0.02448	160.47253	0.75213
RNN	0.02894	0.02421	222.46698	0.37756
ESN	**0.02603**	**0.02238**	2.14621	0.12329

TABLE 5.8 : Prédictions univariées à un pas en avant de la série node2_ram du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.02346	0.01872	-	0.00011
ARIMA	0.02294	0.01858	283.59772 (0.14180)	5.66439
LSTM	0.01979	0.01618	164.26032	0.80022
GRU	0.01948	0.01595	152.58536	0.69669
RNN	0.01940	0.01603	189.28047	0.67344
ESN	**0.01696**	**0.01438**	2.04692	0.10552

TABLE 5.9 : Prédictions univariées à un pas en avant de la série node3_ram du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.12048	0.09816	-	0.00014
ARIMA	0.10915	0.08975	237.76282 (0.11888)	5.78672
LSTM	0.09098	0.07765	205.38146	0.92063
GRU	0.09038	0.07747	152.14188	0.84251
RNN	0.08977	0.07711	189.27849	0.35295
ESN	**0.08681**	**0.07571**	3.34907	0.12442

TABLE 5.10 : Prédictions univariées à un pas en avant de la série node1_cpu du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.17724	0.14452	-	0.00013
ARIMA	**0.12539**	**0.10858**	296.69617 (0.14835)	5.62501
LSTM	0.12788	0.10951	209.73208	1.07458
GRU	0.12790	0.10948	172.11567	1.00632
RNN	0.12836	0.10954	84.78259	0.40384
ESN	0.12613	0.10915	1.36598	0.11125

TABLE 5.11 : Prédictions univariées à un pas en avant de la série node2_cpu du premier ensemble de données.

Modèle	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	0.03505	0.02852	-	0.00009
ARIMA	0.03429	0.02791	262.20982 (0.13110)	5.62980
LSTM	0.02833	0.02388	205.57009	0.91729
GRU	0.02830	0.02378	153.42972	0.84580
RNN	0.02874	0.02401	203.39467	0.38310
ESN	**0.02569**	**0.02209**	2.11508	0.14229

TABLE 5.12 : Prédictions univariées à un pas en avant de la série node3_cpu du premier ensemble de données.

Dans la série chronologique **node1_delay**, l'ESN brille par sa précision exceptionnelle, affichant le RMSE le plus bas (0.01583) ainsi que le MAE le plus minime (0.01362). Ces résultats laissent entrevoir l'ESN comme le choix optimal pour une prédiction précise dans cette série chronologique.

En comparaison, les modèles LSTM, GRU et RNN présentent des performances similaires en termes de précision. LSTM, étant le plus complexe des trois, se traduit par une précision légèrement supérieure à celle de GRU et de RNN.

GRU, d'autre part, propose une version simplifiée du LSTM, mais sa précision est légèrement inférieure à celle du LSTM. RNN, le modèle le plus simple parmi les trois, affiche une précision légèrement moindre que ces deux modèles plus complexes.

ARIMA, pour sa part, demeure compétitif en termes de précision, bien que son niveau de précision reste inférieur à celui des autres modèles.

Dans la série chronologique **node2_delay**, l'ESN se démarque encore une fois avec le RMSE le plus bas (0.00604) et le MAE le plus faible (0.00519), attestant d'une précision remarquable dans cette série chronologique. Cependant, les modèles LSTM, GRU et RNN affichent également de bonnes performances en termes de précision. ARIMA maintient sa compétitivité en termes de précision avec un RMSE de 0.00831 et un MAE de 0.00677, mais reste en deçà des autres modèles en termes de précision.

Pour la série **node3_delay**, l'ESN se démarque à nouveau avec le RMSE le plus bas (0.00889) et le MAE le plus faible (0.00764), démontrant une précision élevée pour cette série chronologique. Les modèles LSTM, GRU et RNN affichent également de bonnes performances. ARIMA est compétitif en termes de précision (RMSE : 0.01201, MAE : 0.00986), mais demeure moins précis que les autres modèles.

Les séries chronologiques **node1_ram**, **node2_ram**, **node3_ram**, **node1_cpu**, **node2_cpu**, et **node3_cpu** présentent une tendance commune où l'ESN se distingue généralement par sa précision exceptionnelle. Les modèles LSTM, GRU et RNN rivalisent en termes de précision, tandis qu'ARIMA maintient également sa compétitivité en termes de précision. Dans la série **node2_cpu**, ARIMA surpasse légèrement l'ESN en termes de précision (RMSE : 0.12539, MAE : 0.10858) et se révèle comme le meilleur modèle en termes de précision pour cette série. Cette mise en évidence souligne la compétitivité d'ARIMA dans la prédiction de ces séries chronologiques de QoS en termes de précision.

5.5.2 Temps d'entraînement des modèles

En ce qui concerne les temps d'entraînement des modèles, nous pouvons observer les différences significatives suivantes :

- Le modèle ESN se distingue en étant nettement plus rapide à entraîner que les autres modèles, avec un temps moyen d'entraînement de seulement **2.23 secondes**, et un écart-type très faible de **0.53 secondes**. cela signifie que les temps d'entraînement sont généralement très proches les uns des autres, ce qui indique une grande stabilité dans le processus d'entraînement du modèle. Cela en fait une option attrayante pour des applications nécessitant une formation rapide et efficace.

- Les modèles LSTM, GRU et RNN présentent des temps d'entraînement relativement longs, avec des moyennes de **193.16**, **154.46** et **176.55** secondes respectivement. Bien qu'ils puissent offrir des performances de précision potentiellement élevées, il est important de noter que leur temps d'entraînement est significativement plus élevé.

- En revanche, le modèle ARIMA demande un temps d'entraînement considérablement plus long, avec une moyenne de 288.68 secondes et un écart-type notable de **25.78** secondes. Par conséquent, il n'est pas idéal pour les situations où l'efficacité temporelle de l'entraînement est cruciale. ARIMA présente un inconvénient majeur en raison de la durée substantielle de son processus d'entraînement. Contrairement aux autres modèles qui conservent leur apprentissage antérieur et ajustent progressivement leurs poids au fil du temps, ARIMA repart pratiquement de zéro à chaque nouvelle itération. En moyenne, ARIMA prend **0.1443 secondes** pour réestimer ses paramètres à chaque fois (les valeurs entre parenthèses). Cette caractéristique découle de la nature statistique d'ARIMA, qui nécessite la réestimation de ses paramètres à chaque ajustement aux données. Par conséquent, il s'avère moins adapté aux scénarios en temps réel où la rapidité de la prédiction est essentielle, car il exige des temps d'entraînement plus longs à chaque nouvelle observation.

Dans l'ensemble, le modèle ESN se distingue par sa rapidité d'entraînement. Cependant, les modèles LSTM, GRU et RNN restent des options viables à prendre en considération en fonction des contraintes temporelles spécifiques à l'application. Toutefois, dans notre cas, lié aux véhicules autonomes, où les données de QoS peuvent varier dans le temps, la fréquence de réentrainement du modèle de prédiction peut s'avérer essentielle. Ces modèles peuvent présenter un léger inconvénient en raison de leurs temps d'entraînement relativement plus longs, ce qui peut potentiellement engendrer des problèmes. Quant au modèle ARIMA, il exige un temps d'entraînement considérablement plus long par rapport aux autres modèles.

5.5.3 Temps de génération des prédictions

Le modèle de persistance, en tant que modèle de base, n'entraîne pas de temps de génération de prédictions significatif.

Dans toutes les séries chronologiques, nous observons des variations importantes en termes de temps de génération de prédictions entre les différents modèles. Voici les points saillants :

- Pour la série chronologique **node1_delay**, l'ESN présente le temps de génération de prédictions le plus bas (0.14184 s), suivi de près par le RNN (0.36809 s). Les modèles LSTM et GRU, bien qu'offrant de bonnes performances en termes de précision, sont légèrement plus lents dans la génération de prédictions (0.94996 s et 0.84545 s respectivement). ARIMA, quant à lui, est le plus lent dans cette série (6.07866 s).

- Dans la série **node2_delay**, l'ESN conserve sa rapidité, générant des prédictions en seulement 0.11188 s, suivi du RNN (0.65463 s). Les modèles LSTM et GRU sont également rapides (0.80933 s et 0.69090 s respectivement), tandis qu'ARIMA reste le plus lent (6.20659 s).

- Pour la série **node3_delay**, l'ESN maintient sa performance de vitesse exceptionnelle avec un temps de génération de prédictions de seulement 0.13935 s, suivi du RNN (0.34672 s). Les modèles LSTM et GRU, bien que rapides, sont légèrement plus lents (0.92989 s et 0.86422 s respectivement), tandis qu'ARIMA demeure le plus lent (5.86463 s).

- Dans les séries **node1_ram**, **node2_ram**, **node3_ram**, **node1_cpu**, **node2_cpu**, et **node3_cpu**, la tendance générale se maintient, où l'ESN génère des prédictions en un temps très court, suivi par les modèles LSTM, GRU et RNN. ARIMA reste le plus lent dans toutes ces séries.

En résumé, l'ESN se distingue régulièrement en étant le modèle le plus rapide pour la génération de prédictions dans l'ensemble des séries chronologiques. Les modèles LSTM, GRU et RNN suivent de près en termes de vitesse, bien que légèrement plus lents que l'ESN. ARIMA est systématiquement le modèle le plus lent pour la génération de prédictions. Cette analyse met en lumière la rapidité de l'ESN, ce qui en fait un choix optimal lorsque des réponses et des prédictions en temps réel sont essentielles.

5.5.4 Synthèse

- L'**ESN** se démarque par sa **précision exceptionnelle**, affichant des erreurs minimales dans **toutes séries chronologiques**.
- Les modèles **LSTM**, **GRU** et **RNN** rivalisent en termes de précision, bien que le LSTM soit légèrement en tête en raison de sa complexité supérieure.
- **ARIMA** conserve une compétitivité en termes de précision, mais ses performances sont légèrement inférieures à celles des autres modèles.
- En ce qui concerne les **temps d'entraînement**, l'ESN se distingue en étant **nettement plus rapide** que les autres modèles, ce qui en fait un choix attrayant pour les applications nécessitant une formation rapide et efficace.
- Les modèles **LSTM**, **GRU** et **RNN** présentent des temps d'entraînement relativement plus longs, ce qui peut être un inconvénient en fonction des contraintes temporelles de l'application.
- **ARIMA** demande un temps d'entraînement **considérablement plus long** que tous les autres modèles, le rendant moins adapté aux applications nécessitant une formation rapide.
- En ce qui concerne les **temps de génération des prédictions**, l'ESN se démarque régulièrement comme le modèle le plus rapide. Les modèles **LSTM**, **GRU** et **RNN** suivent de près en termes de vitesse, bien qu'ils soient légèrement plus lents que l'ESN.
- **ARIMA** est systématiquement le modèle **le plus lent** pour la génération de prédictions.

En résumé, l'ESN offre une combinaison de précision exceptionnelle et de rapidité d'entraînement et de génération de prédictions, en particulier adaptée aux applications nécessitant des réponses en temps réel. Les modèles LSTM, GRU et RNN présentent des performances de précision légèrement inférieure mais avec des temps d'entraînement et de génération de prédictions plus longs. ARIMA, bien qu'encore compétitif en termes de précision, est le modèle le plus lent dans toutes les cas.

5.6 Prédictions univariées à N pas en avant

Dans cette deuxième série d'expérimentations, nous nous concentrons sur la prédiction de N pas en avant. Cette prédiction dépend en réalité de la fréquence d'observation au sein de la série chronologique étudiée. Une fréquence d'observation élevée, c'est-à-dire une observation fréquente des données sur de courtes périodes, conduit à des valeurs plus précises au sein de la série chronologique. En conséquence, les prédictions deviennent également plus précises. Cependant, il arrive que nous

ayons besoin de prédire non seulement une valeur future, mais plusieurs. Prédire sur un horizon trop court, tout en maintenant une fréquence d'observation élevée, pourrait signifier que le système n'a pas suffisamment de temps pour prendre des mesures correctives. En revanche, un horizon trop long aurait pour effet de réduire la précision des prédictions. Cette baisse de précision s'explique par le fait que plus l'horizon est long, plus le modèle néglige les variations à court terme au sein de la série chronologique. Il en résulte une diminution de la précision. De plus, il est important de noter qu'il existe généralement une forte corrélation entre les valeurs successives au sein d'une série chronologique. Par conséquent, prédire plusieurs pas en avant peut négliger cette corrélation et avoir un impact négatif sur la précision des modèles.

Au cours de nos expérimentations, nous allons évaluer les performances de nos modèles sélectionnés dans le contexte de la prédiction à plusieurs pas en avant, en utilisant différents horizons de prédiction : 2 pas (équivalant à 6 secondes), 4 pas (12 secondes), 6 pas (18 secondes), 8 pas (24 secondes) et 10 pas (30 secondes) en avant.

Parallèlement, nous évaluerons ces modèles en utilisant les 5 stratégies de prédiction à plusieurs pas décrites dans la section 2.4.1. Notre étude sera axée sur la série chronologique **node1_delay**. Nous allons comparer les performances en termes de précision (RMSE et MAE), ainsi que les temps d'entraînement et de génération de prédictions pour chaque modèle avec chaque stratégie. Il convient de noter qu'en raison de sa nature particulière, le modèle ARIMA ne peut être appliqué qu'à la stratégie récursive. En effet, ARIMA est conçu pour prédire séquentiellement les valeurs d'une série, ce qui signifie qu'il est spécifiquement adapté à la prédiction des valeurs futures de la série une étape à la fois, **dans un ordre séquentiel**. Les résultats de ces expérimentations sont présentés dans les tableaux 5.13 à 5.17, chaque tableau correspondant à une stratégie de prédiction à N pas en avant.

Modèle	Horizon de Prédiction	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	2	0.02720	0.02189	-	0.00022
	4	0.03517	0.02787	-	0.00015
	6	0.04156	0.03246	-	0.00010
	8	0.04730	0.03661	-	0.00009
	10	0.05270	0.04064	-	0.00018
ARIMA	2	0.02629	0.02119	653.75044	34.64744
	4	0.03295	0.02622	1244.10167	51.95518
	6	0.03770	0.02971	1852.95325	71.21842
	8	0.04142	0.03255	2474.79536	92.01990
	10	0.04443	0.03492	3095.37520	111.59979
LSTM	2	0.02398	0.01921	185.96033	244.03145
	4	0.03148	0.02468	185.96033	479.65192
	6	0.03812	0.02959	185.96033	713.47360
	8	0.04555	0.03521	185.96033	1026.11798
	10	0.05164	0.03982	185.96033	1434.72751
GRU	2	0.02379	0.01905	160.63590	188.29514
	4	0.03096	0.02424	160.63590	367.97173
	6	0.03708	0.02875	160.63590	541.41497
	8	0.04262	0.03286	160.63590	723.98883
	10	0.04436	0.03397	160.63590	908.04708
RNN	2	0.02286	0.01835	203.59897	174.77102
	4	0.02878	0.02266	203.59897	342.23202
	6	0.03353	0.02611	203.59897	515.89146
	8	0.03770	0.02916	203.59897	689.07837
	10	0.04520	0.03481	203.59897	864.36111
ESN	2	**0.01946**	**0.01593**	1.82606	0.54838
	4	**0.02519**	**0.01995**	1.82606	0.99124
	6	**0.02974**	**0.02321**	1.82606	1.02333
	8	**0.03404**	**0.02628**	1.82606	2.65858
	10	**0.03803**	**0.02919**	1.82606	1.39220

TABLE 5.13 : Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie récursive.

Modèle	Horizon de Prédiction	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	2	0.02720	0.02189	-	0.00022
	4	0.03517	0.02787	-	0.00015
	6	0.04156	0.03246	-	0.00010
	8	0.04730	0.03661	-	0.00009
	10	0.05270	0.04064	-	0.00018
LSTM	2	0.02402	0.01939	368.63541	2.13098
	4	0.02877	0.02307	647.23179	4.31572
	6	0.03311	0.02599	1120.19067	9.66910
	8	0.03642	0.02830	1416.86246	9.09336
	10	0.04218	0.03298	1814.29116	11.02615
GRU	2	0.02470	0.02002	295.16508	2.84358
	4	0.02922	0.02353	673.59231	3.88605
	6	0.03387	0.02663	1019.08502	5.72668
	8	0.03666	0.02837	1475.91159	9.50767
	10	0.04268	0.03340	1782.99367	16.84478
RNN	2	0.02391	0.01937	267.39001	0.90377
	4	0.03112	0.02530	499.90637	1.92336
	6	0.03439	0.02689	665.87608	3.40509
	8	0.03728	0.02886	937.42033	3.94438
	10	0.04208	0.03251	1262.44483	5.39870
ESN	2	**0.01941**	**0.01598**	7.85470	0.21909
	4	**0.02517**	**0.01999**	7.15839	0.22987
	6	**0.02990**	**0.02332**	11.94157	0.28549
	8	**0.03416**	**0.02636**	15.50191	0.37918
	10	**0.03834**	**0.02939**	20.58492	0.48014

TABLE 5.14 : Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie directe.

Modèle	Horizon de Prédiction	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	2	0.02720	0.02189	-	0.00022
	4	0.03517	0.02787	-	0.00015
	6	0.04156	0.03246	-	0.00010
	8	0.04730	0.03661	-	0.00009
	10	0.05270	0.04064	-	0.00018
LSTM	2	0.02308	0.01858	115.42446	0.58381
	4	0.02935	0.02326	286.14430	1.18311
	6	0.03418	0.02675	391.17303	2.10200
	8	0.03734	0.02914	502.98660	2.32222
	10	0.04090	0.03205	622.08811	2.90129
GRU	2	0.02330	0.01870	136.87591	0.59887
	4	0.02944	0.02329	253.00411	0.91815
	6	0.03421	0.02679	445.41206	2.27884
	8	0.03723	0.02905	516.29333	2.68460
	10	**0.04087**	0.03203	572.62931	2.45680
RNN	2	0.02280	0.01835	72.18820	0.39885
	4	0.02917	0.02308	150.03004	0.86991
	6	0.03445	0.02697	231.09158	1.38626
	8	0.03718	0.02899	293.66448	5.07520
	10	0.04159	0.03256	352.98019	2.54613
ESN	2	**0.02087**	**0.01690**	0.50169	0.05832
	4	**0.02735**	**0.02157**	0.95320	0.11133
	6	**0.03199**	**0.02493**	1.65765	0.16441
	8	**0.03670**	**0.02833**	1.95928	0.22359
	10	0.04095	**0.03150**	2.70162	0.28032

TABLE 5.15 : Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie hybride DirRec.

Modèle	Horizon de Prédiction	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	2	0.02720	0.02189	-	0.00022
	4	0.03517	0.02787	-	0.00015
	6	0.04156	0.03246	-	0.00010
	8	0.04730	0.03661	-	0.00009
	10	0.05270	0.04064	-	0.00018
LSTM	2	0.02241	0.01800	94.78146	0.99485
	4	0.02686	0.02135	72.08377	1.91246
	6	0.03228	0.02534	250.09103	1.66337
	8	0.03621	0.02818	801.79241	1.15127
	10	0.03959	0.03069	56.67461	0.41469
GRU	2	0.02094	0.01703	227.07894	0.62042
	4	0.02763	0.02202	53.80654	1.06034
	6	0.03163	0.02483	269.90927	1.51047
	8	0.03629	0.02826	413.02616	0.39325
	10	0.04055	0.03147	41.79832	0.18965
RNN	2	0.02076	0.01686	143.01586	0.28063
	4	0.02788	0.02198	47.93892	0.45329
	6	0.03201	0.02526	122.15408	0.61484
	8	0.03629	0.02826	413.02616	0.39325
	10	0.04055	0.03147	41.79832	0.18965
ESN	2	**0.01939**	**0.01596**	1.74218	0.07664
	4	**0.02516**	**0.01997**	1.71317	0.09333
	6	**0.03080**	**0.02410**	7.33919	0.38094
	8	**0.03520**	**0.02723**	16.30125	1.09223
	10	**0.03893**	**0.03022**	7.05726	0.42938

TABLE 5.16 : Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie MIMO.

Modèle	Horizon de Prédiction	RMSE	MAE	Temps d'entraînement (s)	Temps de génération de prédictions (s)
Persistance	2	0.02720	0.02189	-	0.00022
	4	0.03517	0.02787	-	0.00015
	6	0.04156	0.03246	-	0.00010
	8	0.04730	0.03661	-	0.00009
	10	0.05270	0.04064	-	0.00018
LSTM	2	0.02241	0.01800	94.78146	0.99485
	4	0.02784	0.02212	229.66953	0.83705
	6	0.03281	0.02555	284.02866	2.24353
	8	0.03760	0.02910	144.28235	1.60486
	10	**0.04160**	**0.03226**	121.73754	2.14036
GRU	2	0.02094	0.01703	227.07894	0.62042
	4	0.02759	0.02192	177.47386	0.81719
	6	0.03277	0.02553	380.22341	1.56735
	8	0.03759	0.02914	149.68066	1.48651
	10	0.04179	0.03243	86.46238	2.21265
RNN	2	0.02076	0.01686	143.01586	0.28063
	4	0.02842	0.02260	166.54311	0.55423
	6	0.03282	0.02559	306.92912	0.88241
	8	0.03918	0.03052	152.43318	0.92432
	10	0.04195	0.03253	66.63897	1.18872
ESN	2	**0.01939**	**0.01596**	1.74218	0.07664
	4	**0.02706**	**0.02138**	3.11527	0.12229
	6	**0.03264**	**0.02537**	7.41965	0.38142
	8	**0.03753**	**0.02903**	4.52528	0.19782
	10	0.04223	0.03247	12.23867	0.26258

TABLE 5.17 : Performance des modèles de prédiction pour différents horizons de prédiction avec la stratégie DIRMO (s=2).

5.6.1 Précision des prédictions

Le modèle de persistance continue de servir de référence. Dans le contexte de la prédiction à plusieurs pas en avant, ce modèle génère simplement la même valeur actuelle autant de fois que la taille de l'horizon de prédiction. Cependant, nous allons maintenant examiner en détail les performances de nos modèles dans chaque stratégie de prédiction à plusieurs pas en avant :

5.6.1.1 Stratégie Recursive

Pour rappel, cette stratégie consiste à appliquer de manière répétée un modèle de prédiction à 1 pas en avant N fois pour générer N prédictions. À chaque itération, la valeur estimée précédente est utilisée comme entrée pour la prédiction suivante. Vous trouverez ci-dessous le pseudocode de l'algorithme que nous avons utilisé pour implémenter cette stratégie, présenté dans l'algorithme 5.

Algorithm 5 Stratégie de prédiction récursive

```
function PredictRecursive(model, N)
    predictions ← []                    ▷ Initialiser une liste vide pour stocker les prédictions
    for i ← 1 to N do
        prediction ← Modèle de prédiction à 1 pas en avant
        predictions.append(prediction)  ▷ Ajouter la prédiction à la liste
    end for
    return predictions                  ▷ Renvoyer la liste des prédictions
end function
```

Après avoir appliqué cette stratégie avec nos modèles sélectionnés et différents horizons de prédiction, nous pouvons observer les résultats suivants à partir du tableau 5.13 :

- Le modèle ESN se distingue en obtenant les meilleurs résultats en termes de RMSE et de MAE pour tous les horizons de prédiction, affichant un écart remarquable par rapport aux autres modèles.
- Les modèles LSTM, GRU et RNN affichent de bonnes performances, mais ils sont moins performants que l'ESN. Parmi les trois, le RNN a obtenu les meilleurs résultats pour des horizons de prédiction allant de 2 à 8, suivi du GRU, puis du LSTM. Toutefois, pour un horizon de prédiction de 10, le GRU a légèrement dépassé le RNN, tandis que le LSTM est resté le moins performant parmi les trois.
- ARIMA se révèle être un concurrent sérieux en termes de précision. Bien qu'il affiche le RMSE et le MAE les plus bas pour des horizons de prédiction de 2 et 4, il parvient à dépasser le LSTM à partir de l'horizon de prédiction 6. Cependant, sa précision demeure toujours inférieure à celle du RNN, du GRU et de l'ESN, ce dernier étant le meilleur modèle dans tous les horizons de prédiction.
- Tous les modèles ont nettement surpassé le modèle de persistance dans tous les horizons de prédiction, démontrant ainsi l'efficacité de nos modèles de prédiction pour la prédiction à plusieurs pas en avant avec la stratégie récursive.
- La principale limitation de la stratégie récursive réside dans **l'accumulation des erreurs de prédiction** au fur et à mesure que l'horizon de prédiction augmente. Cependant, il convient de noter que lorsqu'un modèle est capable de produire des prédictions avec des erreurs faibles, il

peut maintenir cette précision même avec un horizon de prédiction plus long. Cela est particulièrement évident dans le cas de l'ESN, qui se distingue en tant que modèle le plus précis pour la prédiction à un pas en avant. À mesure que l'horizon de prédiction augmente, l'écart entre sa précision et celle des autres modèles devient encore plus marqué.

5.6.1.2 Stratégie Directe

Cette stratégie implique la formation d'un ensemble de modèles de prédiction correspondant à la taille de l'horizon de prédiction. Chaque modèle génère une prédiction pour une valeur de l'horizon. L'algorithme 6 ci-dessous illustre la méthode que nous avons utilisée pour mettre en œuvre cette stratégie.

Algorithm 6 Stratégie de prédiction directe

function TRAINDIRECTMULTISTEP($train_data, model_type, horizon$)
  $models \leftarrow []$ ▷ Initialiser une liste vide pour stocker les modèles
  for $i \leftarrow 1$ to $horizon$ **do**
    $model \leftarrow$ Créer un modèle de type $model_type$
    $train_subset \leftarrow train_data[1 : N - i]$ ▷ Sous-ensemble de données d'entraînement
    $target \leftarrow$ décalage_à_gauche$(train_data, i)$ ▷ Cibles correspondant à l'horizon
    $model$.entrainer$(train_subset, target)$
    $models$.append$(model)$ ▷ Ajouter le modèle à la liste
  end for
  return $models$ ▷ Renvoyer la liste des modèles formés
end function
function PREDICTDIRECTMULTISTEP($test_data, models$)
  $predictions \leftarrow []$ ▷ Initialiser une liste vide pour stocker les prédictions
  for $i \leftarrow 1$ to taille de $test_data -$ horizon **do**
    $input_data \leftarrow test_data[i : i + horizon]$ ▷ Données d'entrée pour la prédiction
    $model \leftarrow models[i]$ ▷ Sélectionner le modèle correspondant à l'horizon actuel
    $prediction \leftarrow model$.predict$(input_data)$ ▷ Prédire une valeur
    $predictions$.append$(prediction)$ ▷ Ajouter la prédiction à la liste
  end for
  return $predictions$ ▷ Renvoyer la liste des prédictions
end function

En se basant sur les données du Tableau 5.14, qui présentent les résultats de la prédiction à plusieurs pas en avant obtenus avec nos modèles sélectionnés pour différents horizons de prédiction en utilisant cette stratégie, nous pouvons observer ce qui suit :

- Une fois de plus, le modèle ESN se distingue en affichant les meilleures performances en termes de RMSE et de MAE pour tous les horizons de prédiction, avec un écart remarquable par rapport aux autres modèles.
- Les modèles LSTM, GRU et RNN démontrent de bonnes performances, bien qu'ils soient moins performants que l'ESN. Le LSTM surpasse le GRU pour tous les horizons de prédiction, mais le RNN parvient à dépasser légèrement le LSTM dans l'horizon de prédiction de 10.
- Tous les modèles ont largement surpassé le modèle de persistance dans tous les horizons de prédiction, ce qui démontre l'efficacité de nos modèles en termes de précision.
- Si l'on compare la stratégie directe à la stratégie récursive pour le meilleur modèle en termes de précision, l'ESN, on constate que pour des horizons de prédiction de 2 et 4, la stratégie directe

offre une légère amélioration de la précision par rapport à la stratégie récursive. Cependant, à partir d'un horizon de prédiction de 6, la stratégie récursive commence à surpasser la stratégie directe, et cet écart entre les deux stratégies s'accentue avec l'augmentation de l'horizon de prédiction.

Cela s'explique par le fait que l'ESN, en tant que modèle récurrent, est capable de capturer efficacement les dépendances à court terme dans les données. Toutefois, lorsque l'horizon de prédiction atteint 6, il dépasse la capacité de l'ESN à modéliser des dépendances à long terme avec grande précision. À ce stade, il devient évident que l'ESN gère mieux la prédiction à plusieurs pas en avant en intégrant de manière progressive les prédictions précédentes.

- Cependant, en ce qui concerne les modèles LSTM, GRU et RNN, qui ont affiché des erreurs légèrement supérieures à celles de l'ESN, une tendance claire se dessine. Avec la stratégie récursive, LSTM et GRU surpassent légèrement les performances des modèles à stratégie directe, mais cette supériorité s'inverse à partir d'un horizon de prédiction de 4, où la stratégie directe prend clairement l'avantage avec un écart significatif.

  Cette même tendance se vérifie également pour le modèle RNN. Toutefois, dans son cas, la stratégie directe commence à surpasser la stratégie récursive à partir d'un horizon de prédiction de 6, marquant ainsi un changement similaire de performance.

  Ceci suggère que, contrairement à l'ESN, ces modèles sont plus compétents pour modéliser efficacement les dépendances à long terme au-delà d'un certain horizon de prédiction. Lorsque les erreurs cumulatives deviennent significatives avec la stratégie récursive, la capacité de ces modèles à saisir les dépendances à long terme conduit à une meilleure performance de la stratégie directe au-delà de cet horizon spécifique.

- Toutefois, cela ne signifie pas que le modèle ESN est incapable de saisir les dépendances à long terme. Le fait qu'il ait surpassé les autres modèles avec la stratégie directe démontre qu'il peut capturer ces dépendances avec précision, mais il excelle davantage dans la capture des dépendances à court terme.

5.6.1.3 Stratégie hybride DirRec

Cette stratégie combine les deux stratégies précédentes en utilisant un ensemble de modèles de prédiction correspondant à la taille de l'horizon de prédiction. Chaque modèle génère une prédiction pour une valeur de l'horizon tout en incorporant les prédictions générées par les étapes précédentes dans l'ensemble d'entrée. L'algorithme 7 ci-dessous illustre la mise en œuvre de cette stratégie :

L'objectif principal de cette stratégie est de combiner les avantages des deux stratégies, directe et récursive, tout en reconnaissant que cette approche peut également présenter certains inconvénients des deux. Le tableau 5.15 présente les résultats obtenus lors de l'application de nos modèles sélectionnés à différents horizons de prédiction en utilisant cette stratégie. À partir de ce tableau, nous pouvons identifier les points suivants :

- Encore une fois, l'ESN s'est avéré être le modèle le plus performant pour tous les horizons de prédiction. Cependant, cette fois-ci, l'écart de précision entre les modèles est minime. Avec un horizon de prédiction de 10, le modèle GRU a presque atteint la même précision que l'ESN, le surpassant légèrement en termes de RMSE (0.04087 pour GRU contre 0.0495 pour ESN), mais

Algorithm 7 Stratégie de prédiction DirREC

```
function TrainDirREC(train_data, model_type, horizon)
    models ← []                                   ▷ Initialiser une liste vide pour stocker les modèles
    input_data ← train_data                       ▷ Initialiser les données d'entrée
    for h ← 1 to horizon do
        model ← Créer un modèle de type model_type
        target ← décalage_à_gauche(input_data, h)   ▷ Cibles correspondant à l'horizon
        model.entrainer(input_data, target)         ▷ Entraîner le modèle
        models.append(model)                        ▷ Ajouter le modèle à la liste
        input_predictions ← model.predict(input_data)   ▷ Ajouter le modèle à la liste
        input_data ← concaténer(input_data, input_predictions)
    end for
    return models                                 ▷ Renvoyer la liste des modèles formés
end function
function PredictDirREC(test_data, models)
    predictions ← []                              ▷ Initialiser une liste vide pour stocker les prédictions
    input_data ← test_data                        ▷ Initialiser les données d'entrée avec les données de test
    for h ← 1 to horizon do
        model ← models[h]                         ▷ Sélectionner le modèle correspondant à l'horizon actuel
        prediction ← model.predict(input_data)    ▷ Prédire une valeur
        predictions.append(prediction)            ▷ Ajouter la prédiction à la liste
        input_data ← concaténer(input_data, prédiction)   ▷ Mettre à jour les données d'entrée
    end for
    return predictions                            ▷ Renvoyer la liste des prédictions
end function
```

l'ESN a maintenu un MAE inférieur à celui de GRU. Quant au modèle LSTM, il se trouve très proche des deux avec un RMSE de 0.04090.

- Tous les modèles affichent une compétitivité élevée en termes de précision, l'ESN se démarquant généralement comme le meilleur. De plus, ils surpassent nettement le modèle de persistance, ce qui démontre leurs performances. Cette compétitivité suggère que la stratégie hybride DirRec parvient à tirer parti des avantages des deux approches tout en minimisant les inconvénients.

- Le modèle RNN surpasse clairement GRU et LSTM pour des horizons de prédiction de 2 et 4. Cependant, à partir d'un horizon de 6, la tendance s'inverse, avec GRU et LSTM affichant de meilleures valeurs de RMSE et MAE. Cette inversion devient plus marquée à mesure que l'horizon de prédiction s'allonge. LSTM et GRU offrent des performances très similaires, avec de légères variations. Le LSTM a une légère avance pour des horizons de 2, 4 et 6, tandis que le GRU prend la priorité pour des horizons de 8 et 10. Cette tendance suggère que, pour des horizons de prédiction plus longs, la capacité du LSTM et du GRU à saisir des dépendances à long terme devient un atout précieux par rapport au RNN.

- En comparant les 3 stratégies avec le modèle ESN, il est clair que la stratégie DirRec a généré les résultats les moins performants pour tous les horizons de prédiction. Cette différence de performance était d'ailleurs notable par rapport aux deux premières stratégies, qui affichaient des niveaux de précision similaires.

  Cette tendance peut s'expliquer par le fait que la stratégie DirRec combine des aspects des stratégies Directe et Récursive, mais elle semble ne pas capitaliser sur les avantages spécifiques

de chacune de ces approches pour le modèle ESN. En d'autres termes, elle pourrait présenter les inconvénients des deux stratégies sans en tirer pleinement profit.

- Pour le modèle LSTM, nous pouvons observer que la stratégie DirRec se distingue en offrant les meilleures performances pour un horizon de prédiction de 2, suivie de près par la stratégie récursive, puis par la stratégie directe. Cependant, cette tendance s'inverse à mesure que l'horizon de prédiction s'allonge (4, 6, 8), où la stratégie directe devient progressivement plus performante que les autres, suivie de la stratégie DirRec, puis de la stratégie récursive. Cette inversion peut s'expliquer par la capacité du LSTM à modéliser efficacement des dépendances à long terme, une caractéristique qui devient moins cruciale pour des prédictions à court terme. Finalement, pour un horizon de prédiction de 10, la stratégie DirRec redevient la plus performante, suivie de la stratégie directe, et enfin de la stratégie récursive.

  Cela signifie que, pour des horizons de prédiction plus longs, renforcer la capacité du LSTM à capturer les dépendances à long terme en incorporant les prédictions précédentes, comme le fait la stratégie DirRec, entraîne une nette amélioration des performances du modèle.

- Pour les modèles GRU et RNN, nous observons une variabilité dans les performances en fonction de l'horizon de prédiction et de la stratégie.

- En général, il est observé que la stratégie DirRec se situe toujours entre les deux stratégies, directe et récursive, pour les modèles LSTM, GRU et RNN. Cependant, en examinant l'ensemble des résultats pour les 3 modèles, on constate que la stratégie DirRec a réussi à surpasser la stratégie récursive pour un horizon court (2), et elle a également dépassé la stratégie directe pour un horizon long (10), avec des écarts significatifs. Entre ces deux horizons, une variabilité des performances est observée entre les trois stratégies.

  Cela signifie que la stratégie DirRec a réussi à optimiser les performances des modèles LSTM, GRU et RNN, contrairement à l'ESN. En d'autres termes, elle a su tirer pleinement parti des avantages des deux stratégies, directe et récursive, lorsqu'elle a été appliquée à ces modèles.

5.6.1.4 Stratégie MIMO

Cette stratégie est la plus simple à mettre en œuvre parmi toutes les stratégies. L'idée est simple : il s'agit d'ajuster la couche de sortie des modèles (output layer) de manière à générer autant de prédictions que l'horizon de prédiction. Cette couche contient donc un nombre de neurones égal à la taille de l'horizon de prédiction. Le tableau 5.16 présente les résultats obtenus après l'application de cette stratégie pour différents horizons de prédiction, à partir desquels nous pouvons observer :

- L'ESN se distingue une fois de plus en affichant la meilleure précision en termes de RMSE et MAE pour tous les horizons de prédiction.

- Les modèles LSTM, GRU et RNN sont tous très compétitifs, affichant de bonnes précisions. Entre ces modèles, les performances sont presque similaires, avec GRU et RNN qui surpassent notablement LSTM dans un horizon de 2, tandis que LSTM les dépasse nettement avec un horizon de 10. Cette tendance peut être liée à la capacité de LSTM à capturer des dépendances à long terme, ce qui devient plus crucial dans les prédictions à plus long terme.

- Cette stratégie se distingue des 3 premières stratégies en étant une stratégie de sortie multiple, produisant simultanément toutes les prédictions pour l'ensemble de l'horizon de prédiction.

Lorsque nous comparons ses performances à celles des 3 premières stratégies en utilisant le modèle ESN, nous constatons que cette stratégie surpasse les trois premières avec des horizons de prédiction de 2 et 4. Cependant, à partir d'un horizon de prédiction de 6, les stratégies directe et récursive dépassent nettement cette stratégie. Malgré cela, cette dernière reste meilleure que la stratégie hybride DirRec, qui, pour le modèle ESN, n'a pas été en mesure de tirer parti des avantages des deux premières stratégies.

- En ce qui concerne les modèles LSTM, GRU et RNN, la stratégie MIMO a surpassé les 3 premières stratégies pour tous les horizons de prédiction, avec un écart significatif et notable.

  Cela suggère que ces modèles sont plus efficaces pour générer les prédictions à plusieurs pas en avant simultanément en utilisant une stratégie de sortie multiple. Cette approche se traduit par une amélioration significative de la précision de leurs prédictions avec la stratégie MIMO. En revanche, le modèle ESN semble mieux adapté pour générer les prédictions à plusieurs pas en avant en utilisant une stratégie de sortie unique, en particulier avec des horizons plus longs.

5.6.1.5 Stratégie hybride DIRMO

Cette stratégie combine deux stratégies : une stratégie à sortie unique, la stratégie directe, et une stratégie à sortie multiple, la stratégie MIMO. Pour mettre en œuvre cette stratégie, nous devons définir un paramètre supplémentaire, à savoir la taille du bloc "s", qui représente le nombre de valeurs générés par chaque modèle dans l'horizon de prédiction. Dans nos expérimentations, nous avons choisi une valeur de "s" égale à 2, car 2 est le plus grand diviseur commun entre les différents horizons de prédiction que nous avons sélectionnés. Par conséquent, pour chaque horizon de prédiction donné, nous créons un nombre spécifique de tâches de prédiction : 1 pour un horizon de 2, 2 pour un horizon de 4, 3 pour un horizon de 6, 4 pour un horizon de 8, et 5 pour un horizon de 10. Chaque tâche de prédiction est gérée par un modèle à sortie multiple distinct, similaire à la stratégie directe. Il convient de noter que pour un horizon de prédiction de 2, une seule tâche de prédiction est créée, ce qui équivaut à une stratégie MIMO avec un horizon de 2. C'est pourquoi nous obtenons des résultats identiques à ceux de la stratégie MIMO pour un horizon de 2. Les résultats de l'application de cette stratégie sur différents horizons de prédiction avec nos modèles sélectionnés sont présentés dans le tableau 5.17, à partir duquel nous pouvons observer les éléments suivants :

- Pour cette stratégie également, le modèle ESN a réussi à surpasser légèrement les autres modèles pour des horizons de prédiction allant jusqu'à 8. Cependant, cette fois-ci, les erreurs sont très similaires et il n'y a pas de différence significative entre eux.
- Pour un horizon de prédiction de 10, les modèles LSTM, GRU et RNN ont surpassé ESN avec une différence significative en termes de performance.
- La capacité de l'ESN à saisir les dépendances à long terme n'est pas aussi forte que sa capacité à saisir les dépendances à court terme. Cette différence est clairement visible dans la stratégie directe, où le modèle ESN montre une diminution de ses performances lorsque l'horizon de prédiction devient plus long. Cette observation est cohérente avec les résultats de la stratégie DIRMO, qui englobe également la stratégie directe, et où l'on constate que la précision de l'ESN diminue lorsque l'horizon de prédiction s'allonge.
- Les modèles LSTM, GRU et RNN produisent des résultats pratiquement identiques pour des horizons de prédiction de 4, 6 et 8. Cependant, le modèle LSTM les surpasse tous avec un

horizon de prédiction de 10. Cette tendance est similaire à ce que nous avons observé avec la stratégie MIMO pour ces modèles.

- Si l'on compare les performances des modèles avec cette stratégie par rapport aux autres stratégies, nous constatons que, en ce qui concerne le modèle ESN, cette stratégie est la moins performante en termes de précision, notamment avec des horizons de prédiction plus longs (6, 8 et 10). Sa performance commence à se dégrader à partir d'un horizon de prédiction de 4 par rapport aux autres stratégies.
- Pour les modèles LSTM, GRU et RNN, la stratégie MIMO a montré les meilleures précisions par rapport à toutes les autres stratégies, quel que soit l'horizon de prédiction. Cependant, avec la stratégie DIRMO, qui combine la MIMO avec la stratégie directe, nous avons observé que les performances de ces modèles se sont dégradées, en particulier pour les horizons plus longs. Jusqu'à un horizon de 6, cette stratégie a obtenu de meilleures précisions que les autres stratégies à sortie unique, bien que moins bonnes que la stratégie MIMO. En revanche, à partir d'un horizon de 8, les stratégies à sortie unique, notamment DirRec et Directe, ont nettement surpassé cette stratégie.
- L'explication réside dans le fait que les modèles LSTM, GRU et RNN sont plus adaptés aux stratégies à sortie multiple. Lorsque nous avons combiné une stratégie à sortie unique avec une stratégie à sortie multiple dans la stratégie DIRMO, les performances de ces modèles ont commencé à se dégrader, en particulier avec des horizons de prédiction plus longs. En revanche, le modèle ESN semble mieux s'adapter aux stratégies à sortie unique, en particulier la stratégie récursive. Cela se reflète dans ses performances, car même avec la stratégie directe, il n'est pas aussi performant pour capturer les dépendances à long terme que pour capturer les dépendances à court terme dans la stratégie récursive. Cette tendance est clairement observée lorsque nous avons combiné la stratégie directe avec la stratégie MIMO, deux stratégies avec lesquelles l'ESN ne fonctionne pas aussi bien, ce qui se traduit par les moins bonnes performances de ce modèle pour tous les horizons de prédiction avec la stratégie DIRMO.

5.6.2 Temps d'entraînement des modèles

Le modèle ESN se distingue par le temps d'entraînement le plus court dans toutes les stratégies. En revanche, les modèles LSTM, GRU et RNN exigent des temps d'entraînement relativement plus longs. Le modèle ARIMA, utilisé dans le cadre de la stratégie récursive, présente le temps d'entraînement le plus étendu de tous les modèles testés. Dans notre première série d'expérimentations, nous avons minutieusement étudié et comparé les temps d'entraînement de ces modèles. Cette fois, nous nous concentrons sur les temps d'entraînement spécifiques à chaque modèle dans le cadre des 5 stratégies de prédiction. Les observations suivantes sont extraites des données présentées dans la cinquième colonne des tableaux, allant de 5.13 à 5.17 :

- Dans la stratégie récursive, pour les modèles ESN, LSTM, GRU et RNN, un seul modèle est entraîné, et ce même modèle est utilisé pour effectuer des prédictions sur les différents horizons de prédiction. C'est la raison pour laquelle nous observons des temps de génération de modèles identiques pour tous les horizons. En revanche, pour le modèle ARIMA, un réentraînement du modèle est nécessaire pour chaque valeur de l'horizon de prédiction, ce qui explique pourquoi le temps d'entraînement augmente considérablement à mesure que l'horizon de prédiction s'allonge. De plus, tout comme pour les prédictions à un pas en avant, ARIMA exige

un réentraînement du modèle pour chaque nouvelle observation dans les données de test. Cela explique pourquoi le temps d'entraînement du modèle ARIMA est nettement supérieur à celui des autres modèles.

- Le temps d'entraînement du modèle ESN est très court, à seulement 1,82 secondes, ce qui est négligeable par rapport aux autres modèles. En revanche, les modèles LSTM, GRU et RNN nécessitent un temps d'entraînement relativement long, bien qu'il soit encore nettement inférieur à celui de ARIMA. Cependant, en comparaison avec ESN, ces modèles présentent des temps d'entraînement significativement plus longs.

- Dans la stratégie directe, il est nécessaire de former autant de modèles que l'horizon de prédiction pour chaque modèle individuel afin de réaliser des prédictions. Cela explique pourquoi les temps d'entraînement des modèles augmentent avec des horizons de prédiction plus longs. Dans cette configuration, ESN continue à présenter le temps d'entraînement le plus court, tandis que les modèles LSTM, GRU et RNN montrent des temps d'entraînement relativement plus longs.

- Lorsque l'on compare la stratégie directe à la stratégie récursive, il est clair que les temps d'entraînement sont nettement plus longs dans le cas de la stratégie directe pour tous les modèles. Cette différence s'explique par le fait que la stratégie directe implique l'entraînement de plusieurs modèles de chaque type, tandis que la stratégie récursive n'exige qu'un seul modèle pour les prédictions, ce qui réduit considérablement le temps d'entraînement.

- Dans la stratégie DirRec, nous observons une tendance similaire à celle de la stratégie directe, où le temps d'entraînement des modèles augmente à mesure que l'horizon de prédiction s'allonge. Cependant, une observation importante à partir des tableaux 5.14 et 5.15 est que le temps d'entraînement des modèles dans la stratégie DirRec est en fait inférieur à celui de la stratégie directe, malgré l'augmentation de la taille des entrées à chaque étape due aux prédictions précédentes. Cette différence s'explique principalement par les hyperparamètres spécifiques de chaque stratégie, notamment le nombre d'époques d'entraînement. Théoriquement, la stratégie DirRec nécessiterait des temps d'entraînement plus longs en raison de la croissance de la taille des entrées à chaque étape avec les prédictions précédentes.

  Il est également à noter que le modèle ESN présente le temps d'entraînement le plus court dans cette stratégie, ce qui confirme sa capacité à s'entraîner rapidement par rapport aux autres modèles.

- Dans la stratégie MIMO, une particularité est que seul un modèle est entraîné pour chaque type de modèle, quel que soit l'horizon de prédiction. Cela entraîne une variabilité des temps d'entraînement des modèles pour les horizon de prédiction. Toutefois, il est important de noter que les temps d'entraînement des modèles avec la stratégie MIMO sont généralement plus longs que dans la stratégie récursive, mais plus courts que dans les stratégies directe et DirRec.

  Cette différence de temps d'entraînement s'explique par le fait que la stratégie MIMO ne nécessite qu'un seul modèle par type, contrairement aux stratégies directe et DirRec qui requièrent plusieurs modèles. Cependant, dans la stratégie MIMO, le modèle entraîné est à sortie multiple, ce qui signifie qu'il doit apprendre à générer plusieurs prédictions simultanément, ce qui peut rendre le temps d'entraînement plus long que dans la stratégie récursive.

- Avec la stratégie DIRMO, les temps d'entraînement des modèles se révèlent très variables et ne suivent pas une tendance claire, contrairement aux stratégies directe et DirRec, où les temps d'entraînement augmentent proportionnellement à la taille de l'horizon de prédiction. Cette variabilité est également observée dans la stratégie MIMO, en raison des sorties multiples des modèles.

  Toutefois, il est important de noter que les temps de génération des modèles avec la stratégie DIRMO sont généralement relativement plus longs que dans la stratégie MIMO. Cette différence s'explique par le fait que dans la stratégie DIRMO, plusieurs modèles à sorties multiples doivent être entraînés, ce qui peut entraîner une augmentation des temps d'entraînement par rapport à la stratégie MIMO.

5.6.3 Temps de génération des prédictions

Maintenant, portons notre attention sur les temps de génération des prédictions des modèles avec les différentes stratégies. Nous pouvons faire des observations importantes à partir de la dernière colonne des tableaux 5.13 à 5.17 :

- Dans la stratégie récursive, les temps de génération des prédictions augmentent proportionnellement à l'augmentation de la taille de l'horizon de prédiction pour tous les modèles. Cette tendance est cohérente, car à chaque itération, le modèle doit ajouter la prédiction précédente pour prédire une nouvelle valeur, ce qui demande plus de temps avec un horizon de prédiction plus long.

- Même avec cette stratégie, le modèle ESN génère des prédictions plus rapidement que les autres modèles. Son temps de génération de prédictions est négligeable en comparaison avec les autres modèles.

- LSTM, GRU et RNN prennent relativement plus de temps pour générer des prédictions. Ils nécessitent plusieurs minutes pour effectuer cette opération avec la stratégie récursive. Dans des situations réelles, notamment dans le contexte des véhicules autonomes, de tels temps de génération des prédictions pourraient ne pas être tolérables. Cependant, ARIMA génère des prédictions plus rapidement que ces modèles, bien que son temps de génération reste considérablement long par rapport au modèle ESN.

- Pour les stratégies directe et DirRec, les temps de génération des prédictions sont très courts pour tous les modèles, en comparaison avec la stratégie récursive. L'ESN affiche le temps de génération de prédictions le plus court.

- Les stratégies à sorties multiples, MIMO et DIRMO, sont généralement plus rapides en termes de temps de génération des prédictions pour l'ensemble des modèles par rapport à toutes les autres stratégies. Cette efficacité s'explique par le fait que ces stratégies produisent plusieurs valeurs de l'horizon de prédiction à la fois, tandis que les stratégies à sortie unique produisent plusieurs valeurs uniques séparément.

5.6.4 Synthèse

Notre analyse des différentes stratégies de prédiction à plusieurs pas en avant, appliquées à différents modèles de prédiction, a permis de tirer les conclusions suivantes :

1. Le modèle ESN s'est révélé être le plus performant en termes de précision pour toutes les stratégies et pour tous les horizons de prédiction, avec une nette avance par rapport aux autres modèles. Ce modèle présente également les temps d'entraînement et de génération de prédictions les plus courts parmi toutes les stratégies et pour tous les horizons. Cependant, il convient de noter que l'ESN atteint ses meilleures performances avec les stratégies à sortie unique, en particulier avec la stratégie récursive pour des horizons plus longs.

2. Les modèles LSTM, GRU et RNN ont affiché de bonnes performances, bien qu'inférieures à celles de l'ESN en général. Leur précision varie en fonction de l'horizon de prédiction et de la stratégie utilisée. Contrairement à l'ESN, ces modèles sont plus performants avec des stratégies à sortie multiple. Cependant, ils présentent des coûts élevés en termes de temps, avec des temps de génération de prédictions importants dans le cas de la stratégie récursive, et des temps d'entraînement considérables dans les stratégies directe et DirRec. LSTM et GRU se distinguent par leur capacité à capturer efficacement les dépendances à long terme, en particulier avec des horizons plus longs, lorsque la stratégie directe est employée.

3. Le modèle ARIMA, lorsqu'il est utilisé avec la stratégie récursive, s'est avéré être un concurrent sérieux en termes de précision, bien qu'il soit généralement moins performant que l'ESN, surtout pour les horizons de prédiction plus longs. Cependant, il présente un inconvénient majeur en raison de son temps d'entraînement des modèles considérablement élevé, ce qui en fait un choix moins favorable pour les prédictions à plusieurs pas en avant.

4. La stratégie récursive est efficace pour des horizons de prédiction plus courts, mais elle présente un inconvénient majeur, à savoir l'accumulation d'erreurs de prédiction à mesure que l'horizon de prédiction s'allonge. Cependant, il convient de noter que certains modèles, tels que l'ESN, sont capables de maintenir une précision élevée même avec des horizons de prédiction plus longs, ce qui en fait une option intéressante pour de telles situations.

5. La stratégie directe se révèle efficace pour des horizons de prédiction plus longs, en particulier pour les modèles capables de capturer des dépendances à long terme, tels que LSTM et GRU. Cette stratégie permet à ces modèles de saisir efficacement les dépendances à long terme tout en évitant le problème de l'accumulation d'erreurs. Cependant, il est important de noter que la stratégie directe nécessite des temps d'entraînement plus longs en comparaison aux autres stratégies.

6. La stratégie hybride DirRec a réussi à combiner les avantages des stratégies directe et récursive, obtenant ainsi de bonnes performances pour certains modèles, à savoir LSTM, GRU et RNN. Cependant, cette approche n'a pas été aussi efficace pour le modèle ESN, et elle pourrait présenter les inconvénients des deux stratégies sans en tirer pleinement profit.

7. La stratégie MIMO est efficace pour réduire les temps de génération des prédictions, mais elle peut ne pas être aussi précise que les autres stratégies pour certains modèles, en particulier l'ESN. Cependant, elle semble mieux adaptée aux modèles LSTM, GRU et RNN.

8. La stratégie DIRMO présente une variabilité significative dans les performances des modèles en fonction de la configuration spécifique de chaque modèle, mais elle a généralement des temps de génération des prédictions plus longs que la stratégie MIMO.

En fin de compte, le choix de la stratégie dépendra des exigences spécifiques de la tâche de prédiction, y compris les contraintes de temps, l'horizon de prédiction et les performances souhaitées. Dans notre cas spécifié, l'ESN s'est révélé être un choix solide non seulement en termes de précision, mais aussi en termes de temps, étant le modèle le plus rapide parmi tous ceux sélectionnés. Toutefois, il est moins performant avec les stratégies à sortie multiple, MIMO et DIRMO. De plus, bien qu'il soit capable de capturer des dépendances à long terme, sa performance dans la capture des dépendances à long terme est moins impressionnante, ce qui le rend moins adapté aux stratégies directe et DirRec, en particulier avec des horizons plus longs. Dans ce contexte, la stratégie récursive se démarque comme la meilleure option pour l'ESN, en particulier avec des horizons plus longs.

Cependant, il est important de noter que, pour des horizons très longs, au-delà de 20 par exemple, les erreurs cumulatives de la stratégie récursive peuvent considérablement dégrader les performances du modèle. Ainsi, le choix du modèle et de la stratégie dépend fortement du cas d'application spécifique, en particulier de l'horizon de prédiction.

5.7 Prédictions multivariées

Dans cette troisième et dernière série d'expérimentations, notre objectif principal est d'évaluer l'impact des prédictions multivariées sur les performances des modèles. Nous examinons ces performances en termes de précision, ainsi que du temps nécessaire à l'entraînement et à la génération des prédictions.

Pour ces expérimentations, nous avons opté pour le modèle ESN, car il a montré des performances supérieures aux autres modèles.

Nous avons divisé cette série d'expérimentations en deux : la prédiction multivariée par nœud et la prédiction multivariée par caractéristique. Dans la première nous allons tenter de prédire à la fois les 3 caractéristiques d'un même nœud (délai, charge CPU et charge RAM), et dans la deuxième nous allons prédire la même caractéristique pour les trois nœuds (par exemple : le délai pour nœud1, nœud2 et nœud3).

5.7.1 Prédiction multivariée par nœud

Pendant cette première étape, nous allons tester et évaluer la prédiction multivariée des trois caractéristiques de chacun des 3 nœuds. Les résultats de cette évaluation pour chaque nœud sont présentés dans le tableau 5.18.

Nœud	Temps d'entrainement (s)	Temps de génération de prédictions (s)	Caractéristique	RMSE	MAE
Noeud 1	2.44192	0.10297	Délai	0.01624	0.01391
			Charge RAM	0.01578	0.01338
			Charge CPU	0.08619	0.07506
Noeud 2	8.07898	0.08289	Délai	0.00620	0.00531
			Charge RAM	0.02637	0.02246
			Charge CPU	0.12815	0.10996
Noeud 3	7.68983	0.08878	Délai	0.00901	0.00769
			Charge RAM	0.01711	0.01443
			Charge CPU	0.02597	0.02217

TABLE 5.18 : Résultats de la prédiction multivariée des caractéristiques de chaque nœud.

Commentaires Le modèle ESN a démontré de solides performances dans le scénario de prédiction multivariée par nœud, avec des valeurs relativement faibles de RMSE et MAE pour toutes les caractéristiques de chaque nœud. Cependant, une légère dégradation des performances par rapport au scénario de prédiction univariée, où chaque caractéristique est prédite séparément, a été observée. Par exemple, dans le cas de la prédiction du délai du nœud 1, le modèle ESN a obtenu un RMSE de **0.01583** et un MAE de **0.01362** dans le scénario univarié, tandis que dans le scénario multivarié, ces métriques se sont légèrement détériorées avec un RMSE de **0.01624** et un MAE de **0.01391**, ce qui représente une dégradation notable de la performance. Une tendance similaire a été observée pour d'autres caractéristiques de tous les nœuds, suggérant que la prédiction simultanée des caractéristiques d'un même nœud n'a pas amélioré la performance du modèle. Il semble donc qu'il n'y ait pas de corrélations ou de relations significatives entre les caractéristiques d'un même nœud qui pourraient optimiser la capture des dépendances et améliorer la précision du modèle.

Un autre aspect essentiel à prendre en compte concerne les temps d'entraînement et de génération des prédictions. Dans le scénario multivarié, un seul modèle est entraîné pour chaque nœud, ce qui prend un temps total d'entraînement de **18.21 secondes** pour l'ensemble des modèles. En revanche, dans le scénario univarié, où un modèle distinct est entraîné pour chaque caractéristique, le temps total d'entraînement de tous les modèles s'élève à **20.10 secondes**, ce qui représente une amélioration d'environ **2 secondes** en faveur du modèle multivarié. En ce qui concerne le temps de génération des prédictions, le total dans le scénario multivarié est de **0.27 secondes** pour générer des prédictions pour toutes les caractéristiques, tandis que dans le scénario univarié, il faut **1.13 secondes**, ce qui équivaut à une différence d'environ **1 seconde**. Cette réduction de 2 secondes dans le temps d'entraînement des modèles et de 1 seconde dans le temps de génération des prédictions est significative, surtout dans le contexte des véhicules autonomes, où chaque milliseconde compte.

En fin de compte, bien que la prédiction multivariée par nœud puisse offrir des avantages en termes de temps d'entraînement et de génération des prédictions, elle peut entraîner une légère dé-

gradation des performances par rapport à la prédiction univariée. Le choix entre les deux dépendra des priorités spécifiques de la tâche, en mettant en balance les compromis entre précision et efficacité temporelle.

5.7.2 Prédiction multivariée par caractéristique

Dans cette deuxième étape, nous allons évaluer la prédiction multivariée de chacune des 3 caractéristiques pour les 3 nœuds à la fois. Les résultats correspondants sont présentés dans le tableau 5.19.

Caractéristique	Temps d'entrainement (s)	Temps de génération de prédictions (s)	Nœud	RMSE	MAE
Délai	11.93710	0.57046	Nœud 1	0.01592	0.01359
			Nœud 2	0.00617	0.00526
			Nœud 3	0.00905	0.00775
RAM	10.71825	0.14634	Nœud 1	0.01556	0.01310
			Nœud 2	0.02619	0.02236
			Nœud 3	0.01725	0.01456
CPU	1.93479	0.09823	Nœud 1	0.08660	0.07498
			Nœud 2	0.12643	0.10895
			Nœud 3	0.02699	0.02300

TABLE 5.19 : Résultats de la prédiction multivariée de chaque caractéristique pour les 3 nœuds.

Commentaires Dans le scénario multivarié par caractéristique, le modèle a montré une optimisation remarquable par rapport au scénario précédent. Ce scénario a obtenu des performances pratiquement similaires à celles du modèle univarié, voire même les a dépassées dans certains cas, notamment pour la caractéristique CPU du nœud 1. Dans le scénario univarié, le modèle avait prédit cette caractéristique avec un RMSE de **0.08681** et un MAE de **0.07571**, tandis que dans le scénario multivarié par caractéristique, la précision de prédiction était de RMSE **0.08660** et MAE **0.07498**, représentant une amélioration notable de la performance. Ainsi, les caractéristiques mesurées pour les nœuds peuvent présenter des tendances similaires, ce qui peut aider le modèle à optimiser ses performances, contrairement au cas univarié où chaque caractéristique est prédite séparément.

D'autre part, le temps total nécessaire pour l'entraînement des modèles dans ce scénario multivarié par caractéristique s'est élevé à **24.59 secondes**, alors que dans le scénario univarié, il était de seulement **20.10 secondes**. En ce qui concerne la génération des prédictions, dans ce scénario multivarié, cela prend uniquement **0.82 secondes** pour générer les prédictions pour toutes les caractéristiques, tandis que dans le scénario univarié, il faut **1.13 secondes**, ce qui représente un écart de **0.3 secondes**. En résumé, dans ce scénario multivarié par caractéristique, la performance du modèle a été clairement optimisée par rapport au scénario précédent et est pratiquement similaire à celle du scénario univarié, mais cette optimisation a été obtenue au prix d'un temps d'entraînement des mo-

dèles notablement plus long. Cependant, cette stratégie a réussi à optimiser le temps de génération des prédictions par rapport au scénario univarié, bien que le premier scénario multivarié par nœud reste plus rapide pour générer les prédictions. Le choix entre ces deux scénarios dépendra donc des priorités spécifiques de la tâche, en équilibrant les compromis entre précision et efficacité temporelle.

5.7.3 Synthèse

Le choix entre les scénarios de prédiction univariée ou multivariée dépend des priorités spécifiques de la tâche. Le scénario multivarié par nœud offre des gains en efficacité temporelle, tandis que le scénario multivarié par caractéristique permet d'optimiser la précision du modèle au prix de temps d'entraînement plus longs. Il est donc essentiel de prendre en compte ces compromis entre précision et efficacité temporelle en fonction des besoins de la tâche.

Les résultats ont montré que, dans ce cas particulier, la prédiction multivariée, bien qu'apportant des avantages en termes de temps d'entraînement et de génération des prédictions, n'a pas significativement amélioré la précision du modèle. Au contraire, elle a principalement optimisé les performances en termes de temps.

5.8 Réalisation de l'API

Pour la mise en œuvre de notre API, nous avons utilisé la même logique de code ainsi que les mêmes bibliothèques que celles déjà implémentées lors de nos tests précédents, en utilisant le langage de programmation Python pour le backend. Pour le développement de l'API, nous avons choisi FastAPI, qui est un framework web moderne, rapide (haute performance) basé sur Python, conçu spécifiquement pour la création d'API [70]. De plus, nous avons utilisé Uvicorn comme environnement serveur pour exécuter le backend de notre API [80]. La Figure 5.2 représente le schéma de notre API et les technologies associées utilisées dans sa mise en œuvre.

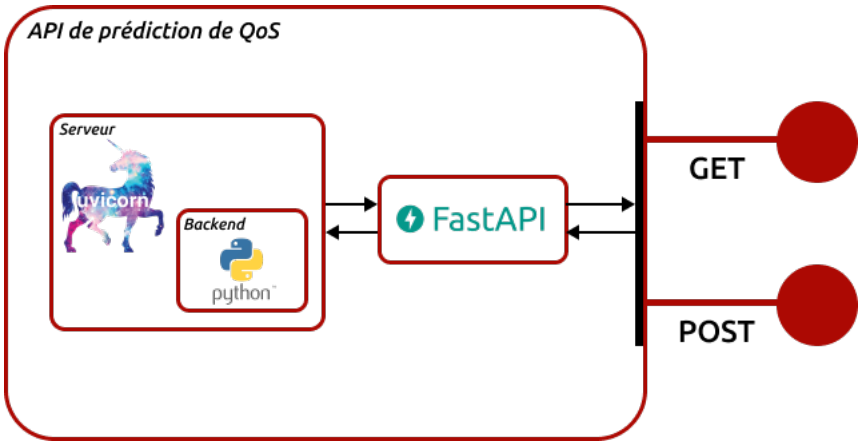


FIGURE 5.2 : Schéma de l'API et des technologies associées

Afin de présenter et de tester les interactions avec notre API, nous avons développé une interface web conviviale permettant d'interagir avec l'API. Cette interface utilise les fonctionnalités de l'API et offre des prédictions en temps réel. Pour développer cette interface, nous avons utilisé *Streamlit*,

qui est un framework frontend open-source en Python [78]. La Figure 5.3 présente une capture de l'interface web développée pour cette expérience.

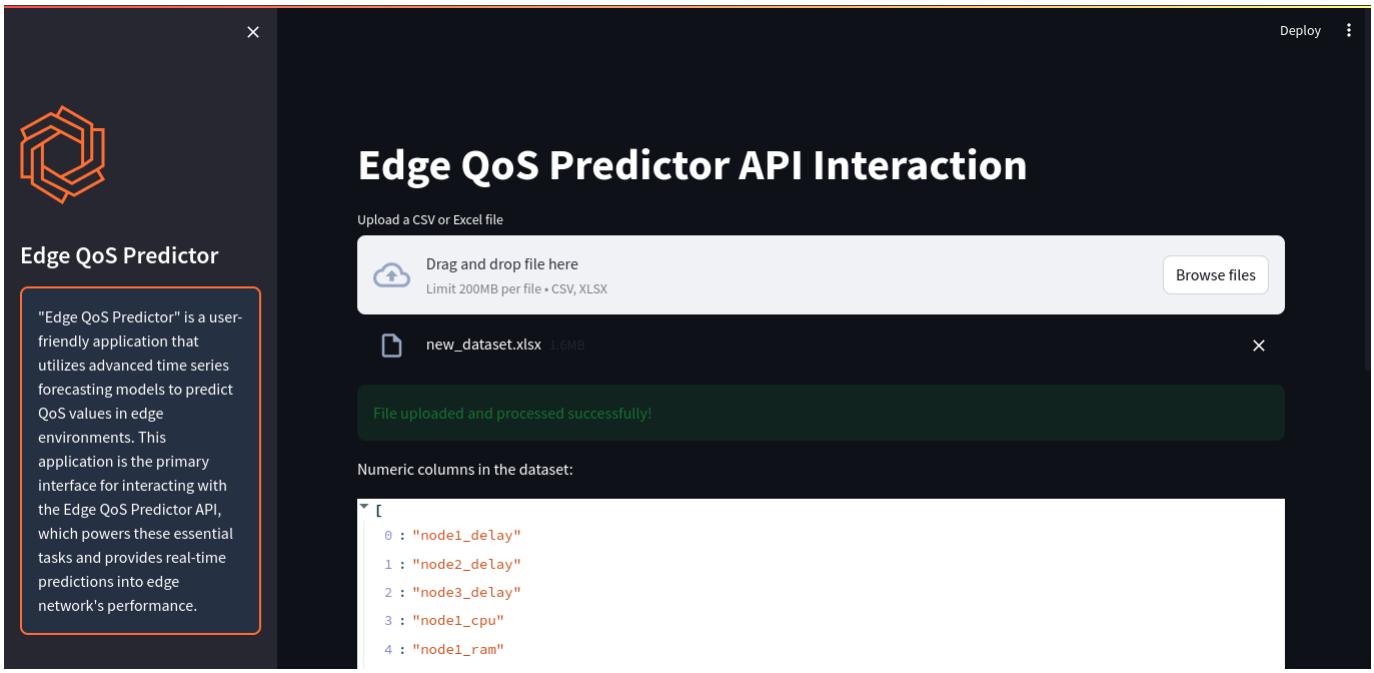


FIGURE 5.3 : Capture de l'interface web

5.9 Conclusion

Après avoir mené minutieusement trois séries d'expérimentations approfondies portant sur plusieurs modèles (considérés comme les plus prometteurs dans le domaine de la prédiction des séries chronologiques), nous avons réussi à identifier un modèle qui répond efficacement à nos besoins en matière de prédiction de la qualité de service (QoS) pour les véhicules autonomes. Le modèle ESN offre un équilibre satisfaisant entre la précision des prédictions et le coût en termes de temps. Bien que les autres modèles présentent des performances compétitives en termes de précision, ils ne parviennent pas à répondre aux exigences temporelles de notre application. De plus, le processus d'optimisation des hyperparamètres, qui peut s'avérer très coûteux en termes de temps, ne prend que quelques minutes dans le cas de l'ESN, nous permettant ainsi de disposer rapidement d'un modèle prêt à générer des prédictions précises. Donc, le modèle ESN constitue un choix idéal pour des applications en temps réel telles que la nôtre.

La prédiction à plusieurs pas en avant peut entraîner une dégradation des performances des modèles de prédiction. Cependant, dans certains cas, il est nécessaire de prédire plusieurs pas en avant pour permettre au système de prendre les mesures appropriées. Dans ce contexte spécifique, les prédictions multivariées ont réussi à optimiser les performances du modèle en termes de temps, mais elles n'ont pas apporté d'amélioration significative en termes de précision. Le choix entre la prédiction univariée et multivariée dépend donc fortement des besoins spécifiques de la tâche : **Privilégie-t-on la précision à tout prix, ou peut-on tolérer une légère perte de précision en échange d'un gain de temps ?**

Conclusion Générale

Ce travail a traité la problématique cruciale de la garantie d'une QoS suffisante pour les véhicules autonomes dans des environnements Edge. Notre approche proactive visait à anticiper les éventuelles dégradations de la QoS au niveau des nœuds Edge.

L'objectif principal de cette étude était de répondre à la question fondamentale : "Comment anticiper une éventuelle dégradation de la QoS pour les nœuds Edge dans des environnements de véhicules autonomes ?" Pour cela, nous avons développé et évalué différents modèles de prédiction de la QoS.

Nous avons constaté que le modèle ESN (Echo State Network) se distingue en proposant un équilibre satisfaisant entre la précision des prédictions et le coût en termes de temps. Il répond aux exigences temporelles essentielles des applications en temps réel, telles que celles des véhicules autonomes.

De plus, nous avons développé une API robuste qui fournit une interface flexible pour configurer, entraîner et utiliser les modèles de prédiction, ce qui la rend essentielle pour le système global.

Nous avons également examiné la prédiction univariée par rapport à la prédiction multivariée. Les prédictions multivariées se sont révélées utiles pour améliorer les performances en termes de temps, bien que pour ces données particulières, elles n'aient pas conduit à des améliorations significatives de précision. Le choix entre les deux dépend donc des besoins spécifiques de la tâche.

Cette étude a apporté plusieurs contributions significatives à la compréhension et à la résolution des problèmes liés à la prédiction de la QoS, que ce soit dans notre cadre spécifique pour les véhicules autonomes dans des environnements Edge, ou de manière générale :

1. Nous avons mis en place un processus de développement et d'évaluation de modèles de prédiction de la QoS, qui comprend le prétraitement des données, la sélection de modèles, la modélisation et une évaluation rigoureuse.

2. Pour rendre ce processus accessible et utilisable, nous avons encapsulé l'ensemble des étapes et des modèles dans une API robuste.

3. Nous avons conclu que le modèle ESN se distingue par son efficacité dans la prédiction précise de la QoS tout en respectant les contraintes temporelles essentielles pour les véhicules autonomes. Ce modèle offre donc un excellent équilibre entre précision et rapidité.

4. Nous avons évalué la pertinence de la prédiction univariée et multivariée, ce qui fournit des indications pour choisir en fonction des objectifs de précision et de rapidité.

5. Nous avons comparé et évalué rigoureusement les différentes stratégies de prédiction à plusieurs pas en avant, et nous avons constaté qu'il n'y a pas de meilleure stratégie, chaque modèle peut performer de manière optimale avec une stratégie, selon ces caractéristique et sa propre architecture.

Perspectives pour des développements futurs

Tests en conditions réelles

La validation des modèles dans des environnements réels de véhicules autonomes, en mettant en œuvre l'architecture envisagée, reste une étape cruciale pour leur adoption. Cette phase permettra de démontrer leur efficacité et leur capacité à fonctionner dans des situations du monde réel.

Intégration de données contextuelles

Une direction prometteuse serait l'intégration de données contextuelles, telles que les conditions météorologiques, le trafic routier, etc. Cette approche pourrait significativement améliorer la qualité des prédictions, en particulier en ce qui concerne les délais de communication. Elle ouvre la voie à l'amélioration des prédictions pour les séries chronologiques, tout en offrant des possibilités d'application à d'autres domaines où la prédiction d'atteinte de seuils est pertinente.

Bibliographie

[1] L. Antão, R. Pinto, J. Reis, and G. Gonçalves. Requirements for testing and validating the industrial internet of things. In *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pages 110–115, 2018.

[2] P. Bajdor and M. Starostka-Patyk. Smart city : A bibliometric analysis of conceptual dimensions and areas. *Energies*, 14(14), 2021.

[3] J. Barker. Machine learning in m4 : What makes a good unstructured model ? *International Journal of Forecasting*, 36(1) :150–155, 2020. M4 Competition.

[4] S. Barmpounakis, L. Magoula, N. Koursioumpas, R. Khalili, J. M. Perdomo, and R. P. Manjunath. Lstm-based qos prediction for 5g-enabled connected and automated mobility applications. In *2021 IEEE 4th 5G World Forum (5GWF)*, pages 436–440, 2021.

[5] S. Barmpounakis, N. Maroulis, N. Koursioumpas, A. Kousaridas, A. Kalamari, P. Kontopoulos, and N. Alonistioti. Ai-driven, qos prediction for v2x communications in beyond 5g systems. *Computer Networks*, 217 :109341, 2022.

[6] S. A. Bhat, I. B. Sofi, and C.-Y. Chi. Edge computing and its convergence with blockchain in 5g and beyond : Security, challenges, and opportunities. *IEEE Access*, 8 :205340–205373, 2020.

[7] H. Borges, R. Akbarinia, and F. Masseglia. Anomaly Detection in Time Series. In *Transactions on Large-Scale Data- and Knowledge-Centered Systems L*, volume LNCS. TLDKS -12930 of *Lecture Notes in Computer Science. Transactions on Large-Scale Data- and Knowledge-Centered Systems*, pages 46–62. 2021.

[8] V. Cerqueira, L. Torgo, and C. Soares. Machine learning vs statistical methods for time series forecasting : Size matters, 2019.

[9] S. Ding, Y. Li, D. Wu, Y. Zhang, and S. Yang. Time-aware cloud service recommendation using similarity-enhanced collaborative filtering and arima model. *Decision Support Systems*, 107 :103–115, 2018.

[10] R. DiPietro and G. D. Hager. Chapter 21 - deep learning : Rnns and lstm. In S. K. Zhou, D. Rueckert, and G. Fichtinger, editors, *Handbook of Medical Image Computing and Computer Assisted Intervention*, The Elsevier and MICCAI Society Book Series, pages 503–519. Academic Press, 2020.

[11] Y. Dong and C.-Y. J. Peng. Principled missing data methods for researchers. *SpringerPlus*, 2(1) :222, 2013.

[12] ETSI. Multi-access edge computing (mec); study on mec support for v2x use cases. https://www.etsi.org/deliver/etsi_gr/mec/001_099/022/02.01.01_60/gr_mec022v020101p.pdf, 2018.

[13] Y.-Y. Fanjiang, Y. Syu, and W.-L. Huang. Time series qos forecasting for web services using multi-predictor-based genetic programming. *IEEE Transactions on Services Computing*, 15(3) :1423–1435, 2022.

[14] Y.-Y. Fanjiang, Y. Syu, and J.-Y. Kuo. Search based approach to forecasting qos attributes of web services using genetic programming. *Information and Software Technology*, 80 :158–174, 2016.

[15] L. Gao, T. H. Luan, B. Liu, W. Zhou, and S. Yu. *Fog Computing and Its Applications in 5G*, pages 571–593. Springer International Publishing, Cham, 2017.

[16] R. Gao, L. Du, O. Duru, and K. F. Yuen. Time series forecasting based on echo state network and empirical wavelet transformation. *Applied Soft Computing*, 102 :107111, 2021.

[17] M. Hamze. *Autonomie, sécurité et QoS de bout en bout dans un environnement de Cloud Computing*. Theses, Université de Bourgogne, Dec. 2015.

[18] S. Haytamy and F. Omara. A deep learning based framework for optimizing cloud consumer qos-based service composition. *Computing*, 102(5) :1117–1137, May 2020.

[19] R. J. Hyndman and G. Athanasopoulos. *Forecasting : principles and practice*. OTexts, 2018.

[20] H. Ibrahim, C. K. Loo, and F. Alnajjar. Bidirectional parallel echo state network for speech emotion recognition. *Neural Computing and Applications*, 34(20) :17581–17599, Oct 2022.

[21] S. Jeschke, C. Brecher, T. Meisen, D. Özdemir, and T. Eschert. *Industrial Internet of Things and Cyber Manufacturing Systems*, pages 3–19. Springer International Publishing, Cham, 2017.

[22] H. Jin, P. Zhang, H. Dong, Y. Zhu, and A. Bouguettaya. Privacy-aware forecasting of quality of service in mobile edge computing. *IEEE Transactions on Services Computing*, 16(1) :478–492, 2023.

[23] B. João, C. Souza, and F. Serralvo. A systematic review of smart cities and the internet of things as a research topic belmiro do nascimento joão ź crisomar lobo de souza ź a systematic review of smart cities and the internet of things as a research topic belmiro do nascimento joão | crisomar lobo de souza. *Cadernos EBAPE.BR*, 17 :1–16, 01 2020.

[24] L. U. Khan, I. Yaqoob, N. Tran, S. Kazmi, T. Nguyen Dang, and C. S. Hong. Edge-computing-enabled smart cities : A comprehensive survey. *IEEE Internet of Things Journal*, PP, 04 2020.

[25] A. Kousaridas, R. P. Manjunath, J. Perdomo, C. Zhou, E. Zielinski, S. Schmitz, and A. Pfadler. Qos prediction for 5g connected and automated driving. *IEEE Communications Magazine*, 59(9) :58–64, 2021.

[26] J. Li, X. Luo, Y. Xia, Y. Han, and Q.-s. Zhu. A time series and reduction-based model for modeling and qos prediction of service compositions. *Concurrency and Computation : Practice and Experience*, 27, 01 2015.

[27] S.-C. Lin, Y. Zhang, C.-H. Hsu, M. Skach, M. Haque, L. Tang, and J. Mars. The architectural implications of autonomous driving : Constraints and acceleration. *ACM SIGPLAN Notices*, 53 :751–766, 03 2018.

[28] Z. C. Lipton, J. Berkowitz, and C. Elkan. A critical review of recurrent neural networks for sequence learning, 2015.

[29] C. Liu, S. C. Hoi, P. Zhao, and J. Sun. Online arima algorithms for time series prediction. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), Feb. 2016.

[30] J. S. Malak, M. Mohsenzadeh, and M. A. Seyyedi. Web service qos prediction based on multi agents. In *2009 International Conference on Computer Technology and Development*, volume 1, pages 265–269, 2009.

[31] D. Minovski, C. Åhlund, and K. Mitra. Modeling quality of iot experience in autonomous vehicles. *IEEE Internet of Things Journal*, 7(5) :3833–3849, 2020.

[32] S. Mistry, A. Bouguettaya, and H. Dong. *Service Providers' Long-Term QoS Prediction Model*, pages 111–122. Springer International Publishing, Cham, 2018.

[33] A. Musaddiq, R. Ali, R. Bajracharya, Y. Qadri, F. Al-Turjman, and S. W. Kim. *Trends, Issues, and Challenges in the Domain of IoT-Based Vehicular Cloud Network*, pages 49–64. 04 2020.

[34] A. Newell and S. K. Card. The prospects for psychological science in human-computer interaction. *HumanComputer Interaction*, 1(3) :209–242, 1985.

[35] A. Nikitas, K. Michalakopoulou, E. T. Njoya, and D. Karampatzakis. Artificial intelligence, transport and the smart city : Definitions and dimensions of a new mobility era. *Sustainability*, 12(7), 2020.

[36] H. Nourikhah, K. Akbari, and M. Kalantari. Modeling and predicting measured response time of cloud-based web services using long-memory time series. *The Journal of Supercomputing*, 71 :673–696, 02 2014.

[37] L. Pacheco, H. Oliveira, D. Rosário, E. Cerqueira, L. Villas, and T. Braun. Service migration for connected autonomous vehicles. In *2020 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–6, 2020.

[38] M. Peixeiro. *Time Series Forecasting in Python*. 2022.

[39] F. Raissi, S. Yangui, and F. Camps. Autonomous cars, 5g mobile networks and smart cities : Beyond the hype. In *2019 IEEE 28th International Conference on Enabling Technologies : Infrastructure for Collaborative Enterprises (WETICE)*, pages 180–185, 2019.

[40] M. M. Rana and K. Hossain. Connected and autonomous vehicles and infrastructures : A literature review. *International Journal of Pavement Research and Technology*, 16(2) :264–284, Mar 2023.

[41] G. A. Rob J Hyndman. Forecasting : principles and practice. https://otexts.com/fpp2/, 2017.

[42] K. Rong and P. Bailis. ASAP : automatic smoothing for attention prioritization in streaming time series visualization. *CoRR*, abs/1703.00983, 2017.

[43] D. Sabella, A. Vaillant, P. Kuure, U. Rauschenbach, and F. Giust. Mobile-edge computing architecture : The role of mec in the internet of things. *IEEE Consumer Electronics Magazine*, 5(4) :84–91, 2016.

[44] D. Sahoo, N. Sood, U. Rani, G. Abraham, V. Dutt, and A. Dileep. Comparative analysis of multi-step time-series forecasting for network load dataset. In *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pages 1–7, 2020.

[45] S. A. A. Shah, E. Ahmed, M. Imran, and S. Zeadally. 5g for vehicular communications. *IEEE Communications Magazine*, 56(1) :111–117, 2018.

[46] G. Shmueli, P. C. Bruce, I. Yahav, N. R. Patel, and K. C. Lichtendahl Jr. *Data mining for business analytics : concepts, techniques, and applications in R*. John Wiley & Sons, 2017.

[47] R. Shrestha, S. Y. Nam, R. Bajracharya, and S. Kim. Evolution of v2x communication and integration of blockchain for security enhancements. *Electronics*, 9 :1338, 08 2020.

[48] E. Spiliotis, S. Makridakis, A.-A. Semenoglou, and V. Assimakopoulos. Comparison of statistical and machine learning methods for daily sku demand forecasting. *Operational Research*, 22(3) :3037–3061, Jul 2022.

[49] X. Sun, S. Wang, Y. Xia, and W. Zheng. Predictive-trend-aware composition of web services with time-varying quality-of-service. *IEEE Access*, 8 :1910–1921, 2020.

[50] Y. Syu, J.-Y. Kuo, and Y.-Y. Fanjiang. Time series forecasting for dynamic quality of web services : An empirical study. *Journal of Systems and Software*, 134 :279–303, 2017.

[51] Y. Syu and C.-M. Wang. Qos time series modeling and forecasting for web services : A comprehensive survey. *IEEE Transactions on Network and Service Management*, 18(1) :926–944, 2021.

[52] Y. Syu, C.-M. Wang, and Y.-Y. Fanjiang. Modeling and forecasting of time-aware dynamic qos attributes for cloud services. *IEEE Transactions on Network and Service Management*, 16(1) :56–71, 2019.

[53] S. B. Taieb, G. Bontempi, A. Atiya, and A. Sorjamaa. A review and comparison of strategies for multi-step ahead time series forecasting based on the nn5 forecasting competition, 2011.

[54] M. Tozak, S. Taskin, and I. Yildirim. Smart grids and smart cities - a systematic mapping study. pages 227–233, 06 2019.

[55] Z. ur Rehman, O. K. Hussain, F. K. Hussain, E. Chang, and T. Dillon. User-side qos forecasting and management of cloud services. *World Wide Web*, 18(6) :1677–1716, Nov 2015.

[56] S. Wang, X. Zhang, Y. Zhang, L. Wang, J. Yang, and W. Wang. A survey on mobile edge networks : Convergence of computing, caching and communications. *IEEE Access*, PP :1–1, 03 2017.

[57] T. Wei, P. Zhang, H. Dong, H. Jin, and A. Bouguettaya. Mobility-aware proactive qos monitoring for mobile edge computing. In J. Troya, B. Medjahed, M. Piattini, L. Yao, P. Fernández, and A. Ruiz-Cortés, editors, *Service-Oriented Computing*, pages 134–142, Cham, 2022. Springer Nature Switzerland.

[58] T. Wei, P. Zhang, H. Dong, H. Jin, and A. Bouguettaya. Mobility-aware proactive qos monitoring forÂ mobile edge computing. In J. Troya, B. Medjahed, M. Piattini, L. Yao, P. Fernández, and A. Ruiz-Cortés, editors, *Service-Oriented Computing*, pages 134–142, Cham, 2022. Springer Nature Switzerland.

[59] G. White and S. Clarke. Short-term qos forecasting at the edge for reliable service applications. *IEEE Transactions on Services Computing*, 15(2) :1089–1102, 2022.

[60] G. White, A. Palade, and S. Clarke. Forecasting qos attributes using lstm networks. In *2018 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2018.

[61] Q. Wu, G. Ding, Y. Xu, S. Feng, Z. Du, J. Wang, and K. Long. Cognitive internet of things : A new paradigm beyond connection. *IEEE Internet of Things Journal*, 1(2) :129–143, 2014.

[62] Y. Xu, Y. Shi, Y. Ge, S. Chen, and L. Wang. Informer-based qos prediction for v2x communication : A method with verification using reality field test data. *Computer Networks*, page 109958, 2023.

[63] Z. Ye, S. Mistry, A. Bouguettaya, and H. Dong. Long-term qos-aware cloud service composition using multivariate time series analysis. *IEEE Transactions on Services Computing*, 9(3) :382–393, 2016.

[64] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. Dive into deep learning. *arXiv preprint arXiv :2106.11342*, 2021.

[65] P. Zhang, H. Jin, H. Dong, W. Song, and L. Wang. La-lmrbf : Online and long-term web service qos forecasting. *IEEE Transactions on Services Computing*, 14(6) :1809–1823, 2021.

Webographie

[66] A. Harsha. The ultimate showdown : Rnn vs lstm vs gru which is the best ? https://www.shiksha.com/online-courses/articles/rnn-vs-gru-vs-lstm/, 2023.

[67] S. Kumar. Statistical vs machine learning vs deep learning modeling for time series forecasting. https://blog.reachsumit.com/posts/2022/12/stats-vs-ml-for-ts/, 2022.

[68] M. Macharla. The 6 levels of vehicle autonomy explained. https://www.synopsys.com/automotive/autonomous-driving-levels.html.

[69] M. Macharla. Self driving cars and iot. https://iot4beginners.com/self-driving-cars-and-iot/, 2020.

[70] Fastapi. https://fastapi.tiangolo.com/.

[71] Google colaboratory. https://colab.google/.

[72] Keras api reference. https://keras.io/api/.

[73] Kubernetes documentation. https://kubernetes.io/docs/home/.

[74] Organigramme. https://www.laas.fr/public/fr/organigramme.

[75] Pandas. https://pandas.pydata.org/.

[76] Python 3.11.5 documentation. https://www.python.org/.

[77] Qu'est-ce que l'iot ? https://www.sap.com/suisse/products/artificial-intelligence/what-is-iot-internet-of-things.html.

[78] Streamlit. https://streamlit.io/.

[79] Tensorflow. https://www.tensorflow.org/overview.

[80] Uvicorn. https://www.uvicorn.org/.

[81] 2016 fatal motor vehicle crashes : Overview. https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/812456, 2017.

[82] echo state network (esn). https://www.aiforanyone.org/glossary/echo-state-network, 2018.

[83] Comprendre le fonctionnement d'un lstm et d'un gru en schémas. https://penseeartificielle.fr/comprendre-lstm-gru-fonctionnement-schema/, 2019.

[84] La ville intelligente pour mieux vivre en milieu urbain. https://fr.euronews.com/next/2019/10/17/la-ville-intelligente-pour-mieux-vivre-en-milieu-urbain, 2019.

[85] Véhicules autonomes et connectés, les défis actuels et les voies de recherche. https://www.inria.fr/sites/default/files/2019-10/inrialivreblancvac-180529073843.pdf, 2019.

[86] 5g is here, and were all moving to the edge. https://www.avnet.com/wps/portal/apac/resources/article/5g-we-are-all-moving-to-the-edge/, 2021.

[87] Power of recurrent neural networks (rnn) : Revolutionizing ai. https://www.simplilearn.com/tutorials/deep-learning-tutorial/rnn, 2023.